Plotly

Knowledge-Base Reference

Version 7.1.0

Figure structure, Plotly Express and graph objects, updating and styling figures, subplots, templates, rendering and export, core chart types and financial charts, plus an API quick reference.

Source: official documentation, https://plotly.com/python/
Content © 2016-2026 Plotly Technologies Inc. (MIT License)
Compiled October 29, 2026

Contents

About this reference **4**

Getting started with Plotly **5**
- Overview . . . 5
- Installation . . . 5
- Static Image Export . . . 6
- Where to next? . . . 6

The figure data structure **7**
- Overview . . . 7
- Figures as Trees of Attributes . . . 7
- The Top-Level `data` Attribute . . . 8
- The Top-Level `layout` Attribute . . . 8
- The Top-Level `frames` Attribute . . . 9
- The `config` Object . . . 9
- Positioning With Paper, Container Coordinates, or Axis Domain Coordinates . . . 9
- 2D Cartesian Trace Types and Subplots . . . 10
- 3D, Polar, Ternary and Smith Trace Types and Subplots . . . 10
- Map Trace Types and Subplots . . . 11
- Traces Which Are Their Own Subplots . . . 11
- Carpet Trace Types and Subplots . . . 12
- Trace Types, Legends and Color Bars . . . 12

Creating and updating figures **13**
- Figures As Dictionaries . . . 13
- Figures as Graph Objects . . . 13
- Creating Figures . . . 15
- Updating Figures . . . 16
- Overwrite Existing Properties When Using Update Methods . . . 22
- Other Update Methods . . . 23

Plotly Express **25**
- Overview . . . 25
- High-Level Features . . . 25
- Gallery . . . 26
- Part to Whole Charts . . . 28
- Images and Heatmaps . . . 30
- Outline Maps . . . 30

Styling Plotly Express figures **33**
- Styling Figures made with Plotly Express . . . 33
- Built-in Plotly Express Styling Arguments . . . 33
- Updating or Modifying Figures made with Plotly Express . . . 34
- How Plotly Express Works with Templates . . . 35
- Setting Plotly Express Styling Defaults . . . 35

Displaying figures (renderers) **37**
- Displaying Figures . . . 37

Subplots **43**
Subplots and Plotly Express . . . 43
Subplots with Shared Colorscale . . . 48

Theming and templates **54**
Theming and templates . . . 54
Using built-in themes . . . 54
Creating themes . . . 56

Interactive HTML export **63**
Interactive vs Static Export . . . 63
Controlling the size of the HTML file . . . 63
Inserting Plotly Output into HTML using a Jinja2 Template . . . 63
Reference . . . 65

Static image export **66**
Write Image to a File . . . 66
Get Image as Bytes . . . 68
Specify Image Dimensions and Scale . . . 68
Image Export Settings . . . 69

Scatter plots **71**
Scatter plots with Plotly Express . . . 71
Line plots with Plotly Express . . . 72
Scatter and line plots with go.Scatter . . . 75

Line charts **80**
Line Plots with plotly.express . . . 80
Line Plots with column encoding color . . . 80
Data Order in Line Charts . . . 80
Connected Scatterplots . . . 81
Line charts with markers . . . 81
Line plots on Date axes . . . 81
Sparklines with Plotly Express . . . 81
Line Plot with go.Scatter . . . 82

Bar charts **89**
Bar chart with Plotly Express . . . 89
Colored Bars . . . 90
Stacked vs Grouped Bars . . . 90
Aggregating into Single Colored Bars . . . 90
Bar Charts with Text . . . 91
Pattern Fills . . . 92
Facetted subplots . . . 92
Stacked Bar Chart . . . 93
Bar Chart with Relative Barmode . . . 93
Grouped Stacked Bar Chart . . . 93
Stacked Bar Chart From Aggregating a DataFrame . . . 94
Bar Chart with Hover Text . . . 95
Bar Chart with Direct Labels . . . 96
Controlling text fontsize with uniformtext . . . 96
Rotated Bar Chart Labels . . . 96
Customizing Individual Bar Colors . . . 97
Customizing Individual Bar Widths . . . 97
Using a scatterplot to wrap long bars into multiple columns . . . 98
Customizing Individual Bar Base . . . 100

Rounded Bars . . . 100
Colored and Styled Bar Chart . . . 101
Bar Chart with Sorted or Ordered Categories . . . 102
Horizontal Bar Charts . . . 103
Bar Charts With Multicategory Axis Type . . . 103
Reference . . . 103

Histograms **104**
Histograms with Plotly Express . . . 104
Histograms with go.Histogram . . . 107

Box plots **112**
Box Plot with `plotly.express` . . . 112
Box plot with go.Box . . . 113

Heatmaps **121**
Heatmaps with Plotly Express . . . 121
Controlling Aspect Ratio . . . 122
Customizing the axes and labels on a heatmap . . . 122
Display an xarray image with px.imshow . . . 122
Basic Heatmap with `plotly.graph_objects` . . . 123
Heatmap with Categorical Axis Labels . . . 123
Heatmap with Unequal Block Sizes . . . 123
Heatmap with Datetime Axis . . . 124
Text on Heatmap Points . . . 124
Heatmap and datashader . . . 125

Candlestick charts **126**
Simple Candlestick with Pandas . . . 126
Candlestick without Rangeslider . . . 126
Adding Customized Text and Annotations . . . 126
Custom Candlestick Colors . . . 127
Simple Example with `datetime` Objects . . . 127
Reference . . . 128

OHLC charts **129**
Custom Hovertext . . . 131

API quick reference **132**
Plotly Express . . . 132
Graph objects and layout . . . 149
The available keys are: . . . 153
Output . . . 155

About this reference

This knowledge-base reference collects core chapters of the official **Plotly 7.1.0** documentation (https://plotly.com/python/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the Markdown files the website is built from), taken from the `v7.1.0` release tag of github.com/plotly/plotly.py. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 7.1.0 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Note on charts. Plotly charts on plotly.com are interactive and are drawn by running each page's code when the site is built, so they are not part of the documentation source and are not included here. Every code example is included; running one reproduces its chart. Sections that only embed a Dash app demo and sign-up banners were omitted.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. plotly.py and its documentation are distributed under the MIT License, Copyright (c) 2016-2026 Plotly Technologies Inc. Redistribution with this notice is permitted. Full text: https://github.com/plotly/plotly.py/blob/main/LICENSE.txt

Chapter sources

- Getting started with Plotly: https://plotly.com/python/getting-started/
- The figure data structure: https://plotly.com/python/figure-structure/
- Creating and updating figures: https://plotly.com/python/creating-and-updating-figures/
- Plotly Express: https://plotly.com/python/plotly-express/
- Styling Plotly Express figures: https://plotly.com/python/styling-plotly-express/
- Displaying figures (renderers): https://plotly.com/python/renderers/
- Subplots: https://plotly.com/python/subplots/
- Theming and templates: https://plotly.com/python/templates/
- Interactive HTML export: https://plotly.com/python/interactive-html-export/
- Static image export: https://plotly.com/python/static-image-export/
- Scatter plots: https://plotly.com/python/line-and-scatter/
- Line charts: https://plotly.com/python/line-charts/
- Bar charts: https://plotly.com/python/bar-charts/
- Histograms: https://plotly.com/python/histograms/
- Box plots: https://plotly.com/python/box-plots/
- Heatmaps: https://plotly.com/python/heatmaps/
- Candlestick charts: https://plotly.com/python/candlestick-charts/
- OHLC charts: https://plotly.com/python/ohlc-charts/

Getting started with Plotly

Source: https://plotly.com/python/getting-started/

Overview

The `plotly` Python library is an interactive, open-source plotting library that supports over 40 unique chart types covering a wide range of statistical, financial, geographic, scientific, and 3-dimensional use-cases.

Built on top of the Plotly JavaScript library (plotly.js), `plotly` enables Python users to create beautiful interactive web-based visualizations that can be displayed in Jupyter notebooks, saved to standalone HTML files, or served as part of pure Python-built web applications using Dash. The `plotly` Python library is sometimes referred to as "plotly.py" to differentiate it from the JavaScript library.

Thanks to deep integration with our Kaleido image export utility, `plotly` also provides great support for non-web contexts including desktop editors (e.g. QtConsole, Spyder, PyCharm) and static document publishing (e.g. exporting notebooks to PDF with high-quality vector images).

This Getting Started guide explains how to install `plotly` and related optional pages. Once you've installed, you can use our documentation in three main ways:

1. You jump right in to **examples** of how to make basic charts, statistical charts, scientific charts, financial charts, maps, and 3-dimensional charts.
2. If you prefer to learn about the **fundamentals** of the library first, you can read about the structure of figures, how to create and update figures, how to display figures, how to theme figures with templates, how to export figures to various formats and about Plotly Express, the high-level API for doing all of the above.
3. You can check out our exhaustive **reference** guides: the Python API reference or the Figure Reference

For information on using Python to build web applications containing plotly figures, see the *Dash User Guide*.

We also encourage you to join the Plotly Community Forum if you want help with anything related to `plotly`.

Installation

`plotly` may be installed using `pip`:

```
$ pip install plotly
```

or `conda`:

```
$ conda install -c conda-forge plotly
```

If you want to use Plotly Express, install its required dependencies with:

```
pip install plotly[express]
```

You'll also need to install a supported dataframe library.

Static Image Export

plotly.py supports static image export, using the `kaleido` package (version 1.0.0 or greater).

Kaleido has minimal dependencies and can be installed using pip…

```
$ pip install --upgrade kaleido
```

or conda.

```
$ conda install -c plotly python-kaleido
```

Kaleido requires Chrome or Chromium to generate images. By default, Kaleido will use the Chrome or Chromium version already installed on your system. If you don't have it installed or Kaleido can't find it, you may need to install it by running the following command:

```
plotly_get_chrome
```

Where to next?

Once you've installed, you can use our documentation in three main ways:

1. You jump right in to **examples** of how to make basic charts, statistical charts, scientific charts, financial charts, maps, and 3-dimensional charts.
2. If you prefer to learn about the **fundamentals** of the library first, you can read about the structure of figures, how to create and update figures, how to display figures, how to theme figures with templates, how to export figures to various formats and about Plotly Express, the high-level API for doing all of the above.
3. You can check out our exhaustive **reference** guides: the Python API reference or the Figure Reference

For information on using Python to build web applications containing plotly figures, see the *Dash User Guide*.

We also encourage you to join the Plotly Community Forum if you want help with anything related to `plotly`.

The figure data structure

Source: https://plotly.com/python/figure-structure/

Overview

The `plotly` Python package exists to create, manipulate and render graphical figures (i.e. charts, plots, maps and diagrams) represented by data structures also referred to as figures. The rendering process uses the Plotly.js JavaScript library under the hood although Python developers using this module very rarely need to interact with the Javascript library directly, if ever. Figures can be represented in Python either as dicts or as instances of the `plotly.graph_objects.Figure` class, and are serialized as text in JavaScript Object Notation (JSON) before being passed to Plotly.js.

> Note: the recommended entry-point into the plotly package is the high-level plotly.express module, also known as Plotly Express, which consists of Python functions which return fully-populated `plotly.graph_objects.Figure` objects. This page exists to document the architecture of the data structure that these objects represent, for users who wish to understand more about how to customize them, or assemble them from other `plotly.graph_objects` components.

Viewing the underlying data structure for any `plotly.graph_objects.Figure` object, including those returned by Plotly Express, can be done via `print(fig)` or, in JupyterLab, with the special `fig.show("json")` renderer. Figures also support `fig.to_dict()` and `fig.to_json()` methods. `print()`ing the figure will result in the often-verbose `layout.template` key being represented as ellipses `'...'` for brevity.

```
import plotly.express as px

fig = px.line(x=["a","b","c"], y=[1,3,2], title="sample figure")
print(fig)
fig.show()
```

Figures as Trees of Attributes

Plotly.js supports inputs adhering to a well-defined schema, whose overall architecture is explained in this page and which is exhaustively documented in the Figure Reference (which is itself generated from a machine-readable JSON representation of the schema). Figures are represented as trees with named nodes called "attributes". The root node of the tree has three top-level attributes: `data`, `layout` and `frames` (see below).

Attributes are referred to in text and in the Figure Reference by their full "path" i.e. the dot-delimited concatenation of their parents. For example `"layout.width"` refers to the attribute whose key is `"width"` inside a dict which is the value associated with a key `"layout"` at the root of the figure. If one of the parents is a list rather than a dict, a set of brackets is inserted in the path when referring to the attribute in the abstract, e.g. `"layout.annotations[].text"`. Finally, as explained below, the top-level "data" attribute defines a list of typed objects called "traces" with the schema dependent upon the type, and these attributes' paths are listed in the Figure Reference as `"data[type=scatter].name"`.

The `plotly.graph_objects` module contains an automatically-generated hierarchy of Python classes which represent non-leaf attributes in the figure schema and provide a Pythonic API

for them. When manipulating a `plotly.graph_objects.Figure` object, attributes can be set either directly using Python object attributes e.g. `fig.layout.title.font.family="Open Sans"` or using update methods and "magic underscores" e.g. `fig.update_layout(title_font_family="Open Sans")`

When building a figure, it is *not necessary to populate every attribute* of every object. At render-time, the JavaScript layer will compute default values for each required unspecified attribute, depending upon the ones that are specified, as documented in the Figure Reference. An example of this would be `layout.xaxis.range`, which may be specified explicitly, but if not will be computed based on the range of `x` values for every trace linked to that axis. The JavaScript layer will ignore unknown attributes or malformed values, although the `plotly.graph_objects` module provides Python-side validation for attribute values. Note also that if the `layout.template` key is present (as it is by default) then default values will be drawn first from the contents of the template and only if missing from there will the JavaScript layer infer further defaults. The built-in template can be disabled by setting `layout.template="none"`.

The Top-Level `data` Attribute

The first of the three top-level attributes of a figure is `data`, whose value must be a list of dicts referred to as "traces".

- Each trace has one of more than 40 possible types (see below for a list organized by subplot type, including e.g. `scatter`, `bar`, `pie`, `surface`, `choropleth` etc), and represents a set of related graphical marks in a figure. Each trace must have a `type` attribute which defines the other allowable attributes.
- Each trace is drawn on a single subplot whose type must be compatible with the trace's type, or is its own subplot (see below).
- Traces may have a single legend entry, with the exception of pie and funnelarea traces (see below).
- Certain trace types support continuous color, with an associated colorbar, which can be controlled by attributes either within the trace, or within the layout when using the coloraxis attribute.

The Top-Level `layout` Attribute

The second of the three top-level attributes of a figure is `layout`, whose value is referred to in text as "the layout" and must be a dict, containing attributes that control positioning and configuration of non-data-related parts of the figure such as:

- Dimensions and margins, which define the bounds of "paper coordinates" (see below)
- Figure-wide defaults: templates, fonts, colors, hover-label and modebar defaults
- Title and legend (positionable in container and/or paper coordinates)
- Color axes and associated color bars (positionable in paper coordinates)
- Subplots of various types on which can be drawn multiple traces and which are positioned in paper coordinates:
  - `xaxis`, `yaxis`, `xaxis2`, `yaxis3` etc: X and Y cartesian axes, the intersections of which are cartesian subplots
  - `scene`, `scene2`, `scene3` etc: 3d scene subplots
  - `ternary`, `ternary2`, `ternary3`, `polar`, `polar2`, `polar3`, `geo`, `geo2`, `geo3`, `map`, `map2`, `map3`, `smith`, `smith2` etc: ternary, polar, geo, map or smith subplots
- Non-data marks which can be positioned in paper coordinates, or in data coordinates linked to 2d cartesian subplots:
  - `annotations`: textual annotations with or without arrows
  - `shapes`: lines, rectangles, ellipses or open or closed paths
  - `images`: background or decorative images

- Controls which can be positioned in paper coordinates and which can trigger Plotly.js functions when interacted with by a user:
  - `updatemenus`: single buttons, toggles and dropdown menus
  - `sliders`: slider controls

The Top-Level `frames` Attribute

The third of the three top-level attributes of a figure is `frames`, whose value must be a list of dicts that define sequential frames in an animated plot. Each frame contains its own data attribute as well as other parameters. Animations are usually triggered and controlled via controls defined in layout.sliders and/or layout.updatemenus

The `config` Object

At render-time, it is also possible to control certain figure behaviors which are not considered part of the figure proper i.e. the behavior of the "modebar" and how the figure relates to mouse actions like scrolling etc. The object that contains these options is called the `config`, and has its own documentation page. It is exposed in Python as the `config` keyword argument of the `.show()` method on `plotly.graph_objects.Figure` objects.

Positioning With Paper, Container Coordinates, or Axis Domain Coordinates

Various figure components configured within the layout of the figure support positioning attributes named `x` or `y`, whose values may be specified in "paper coordinates" (sometimes referred to as "plot fractions" or "normalized coordinates"). Examples include `layout.xaxis.domain` or `layout.legend.x` or `layout.annotation[].x`.

Positioning in paper coordinates is *not* done in absolute pixel terms, but rather in terms relative to a coordinate system defined with an origin `(0,0)` at `(layout.margin.l, layout.margin.b)` and a point `(1,1)` at `(layout.width-layout.margin.r, layout.height-layout.margin.t)` (note: `layout.margin` values are pixel values, as are `layout.width` and `layout.height`). Paper coordinate values less than 0 or greater than 1 are permitted, and refer to areas within the plot margins.

To position an object in "paper" coordinates, the corresponding axis reference is set to `"paper"`. For instance a shape's `xref` attribute would be set to `"paper"` so that the `x` value of the shape refers to its position in paper coordinates.

Note that the contents of the `layout.margin` attribute are by default computed based on the position and dimensions of certain items like the title or legend, and may be made dependent on the position and dimensions of tick labels as well when setting the `layout.xaxis.automargin` attribute to `True`. This has the effect of automatically increasing the margin values and therefore shrinking the physical area defined between the `(0,0)` and `(1,1)` points. Positioning certain items at paper coordinates less than 0 or greater than 1 will also trigger this behavior. The `layout.width` and `layout.height`, however, are taken as givens, so a figure will never grow or shrink based on its contents.

The figure title may be positioned using "container coordinates" which have `(0,0)` and `(1,1)` anchored at the bottom-left and top-right of the figure, respectively, and therefore are independent of the values of layout.margin.

Furthermore, shapes, annotations, and images can be placed relative to an axis's domain so that, for instance, an `x` value of `0.5` would place the object halfway along the x-axis, regardless of the domain as specified in the `layout.xaxis.domain` attribute. This behavior can be specified

by adding `' domain'` to the axis reference in the axis referencing attribute of the object. For example, setting `yref = 'y2 domain'` for a shape will refer to the length and position of the axis named `y2`.

2D Cartesian Trace Types and Subplots

The most commonly-used kind of subplot is a two-dimensional Cartesian subplot. Traces compatible with these subplots support `xaxis` and `yaxis` attributes whose values must refer to corresponding objects in the layout portion of the figure. For example, if `xaxis="x"`, and `yaxis="y"` (which is the default) then this trace is drawn on the subplot at the intersection of the axes configured under `layout.xaxis` and `layout.yaxis`, but if `xaxis="x2"` and `yaxis="y3"` then the trace is drawn at the intersection of the axes configured under `layout.xaxis2` and `layout.yaxis3`. Note that attributes such as `layout.xaxis` and `layout.xaxis2` etc do not have to be explicitly defined, in which case default values will be inferred. Multiple traces of different types can be drawn on the same subplot.

X- and Y-axes support the `type` attribute, which enables them to represent continuous values (`type="linear"`, `type="log"`), temporal values (`type="date"`) or categorical values (`type="category"`, `type="multicategory"`). Axes can also be overlaid on top of one another to create dual-axis or multiple-axis charts. 2-d cartesian subplots lend themselves very well to creating "small multiples" figures, also known as facet or trellis plots.

The following trace types are compatible with 2d-cartesian subplots via the `xaxis` and `yaxis` attributes:

- scatter-like trace types: `scatter` and `scattergl`, which can be used to draw scatter plots, line plots and curves, time-series plots, bubble charts, dot plots and filled areas and also support error bars
- `bar`, `funnel`, `waterfall`: bar-like trace types which can also be used to draw timelines and Gantt charts
- `histogram`: an *aggregating* bar-like trace type
- `box` and `violin`: 1-dimensional distribution-like trace types
- `histogram2d` and `histogram2dcontour`: 2-dimensional distribution-like density trace types
- `image`, `heatmap` and `contour`: matrix trace types
- `ohlc` and `candlestick`: stock-like trace types
- `carpet`: a special trace type for building carpet plots, in that other traces can use as subplots (see below)
- `splom`: multi-dimensional scatter plots which implicitly refer to many 2-d cartesian subplots at once.

3D, Polar, Ternary and Smith Trace Types and Subplots

Beyond 2D cartesian subplots, figures can include three-dimensional cartesian subplots, polar subplots, ternary subplots and smith subplots. The following trace types support attributes named `scene`, `polar`, `smith` or `ternary`, whose values must refer to corresponding objects in the layout portion of the figure i.e. `ternary="ternary2"` etc. Note that attributes such as `layout.scene` and `layout.ternary2` etc do not have to be explicitly defined, in which case default values will be inferred. Multiple traces of a compatible type can be placed on the same subplot.

The following trace types are compatible with 3D subplots via the `scene` attribute, which contains special camera controls:

- `scatter3d`, which can be used to draw individual markers, 3d bubble charts and lines and curves
- `surface` and `mesh`: 3d surface trace types

- `cone` and `streamtube`: 3d vector field trace types
- `volume` and `isosurface`: 3d volume trace types

The following trace types are compatible with polar subplots via the `polar` attribute:

- scatter-like trace types: `scatterpolar` and `scatterpolargl`, which can be used to draw individual markers, curves and filled areas (i.e. radar or spider charts)
- `barpolar`: useful for wind roses and other polar bar charts

The following trace types are compatible with ternary subplots via the `ternary` attribute:

- `scatterternary`, which can be used to draw individual markers, curves and filled areas

The following trace types are compatible with smith subplots via the `smith` attribute:

- `scattersmith`, which can be used to draw individual markers, curves and filled areas

Map Trace Types and Subplots

Figures can include two different types of map subplots: geo subplots for outline maps and tile-based maps. The following trace types support attributes named `geo` or `map`, whose values must refer to corresponding objects in the layout i.e. `geo="geo2"` etc. Note that attributes such as `layout.geo2` and `layout.map` etc do not have to be explicitly defined, in which case default values will be inferred. Multiple traces of a compatible type can be placed on the same subplot.

The following trace types are compatible with geo subplots via the `geo` attribute:

- `scattergeo`, which can be used to draw individual markers, line and curves and filled areas on outline maps
- `choropleth`: colored polygons on outline maps

The following trace types are compatible with tile map subplots via the `map` attribute:

- `scattermap`, which can be used to draw individual markers, lines and curves and filled areas on tile maps
- `choroplethmap`: colored polygons on tile maps
- `densitymap`: density heatmaps on tile maps

Traces Which Are Their Own Subplots

Certain trace types cannot share subplots, and hence have no attribute to map to a corresponding subplot in the layout. Instead, these traces are their own subplot and support a `domain` attribute for position, which enables the trace to be positioned in paper coordinates (see below). With the exception of `pie` and `funnelarea`, such traces also do not support legends (see below)

The following trace types are their own subplots and support a domain attribute:

- `pie` and `funnelarea`: one-level part-to-whole relationships with legend items
- `sunburst` and `treemap`: hierarchical multi-level part-to-whole relationships
- `parcoords` and `parcats`: continuous and categorical multidimensional figures with parallel coordinates and parallel sets
- `sankey`: flow diagrams
- `table`: text-based tables
- `indicator`: big numbers, gauges, and bullet charts

Carpet Trace Types and Subplots

Certain trace types use traces of type `carpet` as a subplot. These support a `carpet` attribute whose value must match the value of the `carpet` attribute of the `carpet` trace they are to be drawn on. Multiple compatible traces can be placed on the same `carpet` trace.

The following trace types are compatible with `carpet` trace subplots via the `carpet` attribute:

- `scattercarpet`, which can be used to draw individual markers, curves and filled areas
- `contourcarpet`

Trace Types, Legends and Color Bars

Traces of most types can be optionally associated with a single legend item in the legend. Whether or not a given trace appears in the legend is controlled via the `showlegend` attribute. Traces which are their own subplots (see above) do not support this, with the exception of traces of type `pie` and `funnelarea` for which every distinct color represented in the trace gets a separate legend item. Users may show or hide traces by clicking or double-clicking on their associated legend item. Traces that support legend items also support the `legendgroup` attribute, and all traces with the same legend group are treated the same way during click/double-click interactions.

The fact that legend items are linked to traces means that when using discrete color, a figure must have one trace per color in order to get a meaningful legend. Plotly Express has robust support for discrete color to make this easy.

Traces which support continuous color can also be associated with color axes in the layout via the `coloraxis` attribute. Multiple traces can be linked to the same color axis. Color axes have a legend-like component called color bars. Alternatively, color axes can be configured within the trace itself.

Creating and updating figures

Source: https://plotly.com/python/creating-and-updating-figures/

The `plotly` Python package exists to create, manipulate and render graphical figures (i.e. charts, plots, maps and diagrams) represented by data structures also referred to as figures. The rendering process uses the Plotly.js JavaScript library under the hood although Python developers using this module very rarely need to interact with the Javascript library directly, if ever. Figures can be represented in Python either as dicts or as instances of the `plotly.graph_objects.Figure` class, and are serialized as text in JavaScript Object Notation (JSON) before being passed to Plotly.js.

> Note: the recommended entry-point into the plotly package is the high-level plotly.express module, also known as Plotly Express, which consists of Python functions which return fully-populated `plotly.graph_objects.Figure` objects. This page exists to document the structure of the data structure that these objects represent for users who wish to understand more about how to customize them, or assemble them from other `plotly.graph_objects` components.

Figures As Dictionaries

At a low level, figures can be represented as dictionaries and displayed using functions from the `plotly.io` module. The `fig` dictionary in the example below describes a figure. It contains a single `bar` trace and a title.

```
fig = dict({
    "data": [{"type": "bar",
              "x": [1, 2, 3],
              "y": [1, 3, 2]}],
    "layout": {"title": {"text": "A Figure Specified By Python Dictionary"}}
})

# To display the figure defined by this dict, use the low-level plotly.io.show function
import plotly.io as pio

pio.show(fig)
```

Figures as Graph Objects

The `plotly.graph_objects` module provides an automatically-generated hierarchy of classes called "graph objects" that may be used to represent figures, with a top-level class `plotly.graph_objects.Figure`.

> Note that the *recommended alternative* to working with Python dictionaries is to create entire figures at once using Plotly Express and to manipulate the resulting `plotly.graph_objects.Figure` objects as described in this page, wherever possible, rather than to assemble figures bottom-up from underlying graph objects. See "When to use Graph Objects".

Graph objects have several benefits compared to plain Python dictionaries.

1. Graph objects provide precise data validation. If you provide an invalid property name or an invalid property value as the key to a graph object, an exception will be raised with a helpful error message describing the problem. This is not the case if you use plain Python dictionaries and lists to build your figures.
2. Graph objects contain descriptions of each valid property as Python docstrings, with a full API reference available. You can use these docstrings in the development environment of your choice to learn about the available properties as an alternative to consulting the online Full Reference.
3. Properties of graph objects can be accessed using both dictionary-style key lookup (e.g. `fig["layout"]`) or class-style property access (e.g. `fig.layout`).
4. Graph objects support higher-level convenience functions for making updates to already constructed figures (`.update_layout()`, `.add_trace()` etc) as described below.
5. Graph object constructors and update methods accept "magic underscores" (e.g. `go.Figure(layout_title_text="The Title")` rather than `dict(layout=dict(title=dict(text="The Title")))`) for more compact code, as described below.
6. Graph objects support attached rendering (`.show()`) and exporting functions (`.write_image()`) that automatically invoke the appropriate functions from the `plotly.io` module.

Below you can find an example of one way that the figure in the example above could be specified using a graph object instead of a dictionary.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Bar(x=[1, 2, 3], y=[1, 3, 2])],
    layout=go.Layout(
        title=go.layout.Title(text="A Figure Specified By A Graph Object")
    )
)

fig.show()
```

You can also create a graph object figure from a dictionary representation by passing the dictionary to the `go.Figure` constructor.

```
import plotly.graph_objects as go

dict_of_fig = dict({
    "data": [{"type": "bar",
              "x": [1, 2, 3],
              "y": [1, 3, 2]}],
    "layout": {"title": {"text": "A Figure Specified By A Graph Object With A Dictionary"}}
})

fig = go.Figure(dict_of_fig)

fig.show()
```

Converting Graph Objects To Dictionaries and JSON

Graph objects can be turned into their Python dictionary representation using the `fig.to_dict()` method. You can also retrieve the JSON string representation of a graph object using the `fig.to_json()` method.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Bar(x=[1, 2, 3], y=[1, 3, 2])],
```

```
    layout=go.Layout(height=600, width=800)
)

fig.layout.template = None # to slim down the output

print("Dictionary Representation of A Graph Object:\n\n" + str(fig.to_dict()))
print("\n\n")
print("JSON Representation of A Graph Object:\n\n" + str(fig.to_json()))
print("\n\n")
```

Creating Figures

This section summarizes several ways to create new graph object figures with the `plotly.py` graphing library.

> The *recommended way* to create figures and populate them is to use Plotly Express but this page documents various other options for completeness

Plotly Express

Plotly Express (included as the `plotly.express` module) is a high-level data visualization API that produces fully-populated graph object figures in single function-calls.

```
import plotly.express as px

df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", title="A Plotly Express Figure")

# If you print the figure, you'll see that it's just a regular figure with data and layout
# print(fig)

fig.show()
```

Graph Objects `Figure` Constructor

As demonstrated above, you can build a complete figure by passing trace and layout specifications to the `plotly.graph_objects.Figure` constructor. These trace and layout specifications can be either dictionaries or graph objects.

In the following example, the traces are specified using graph objects and the layout is specified as a dictionary.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Bar(x=[1, 2, 3], y=[1, 3, 2])],
    layout=dict(title=dict(text="A Figure Specified By A Graph Object"))
)

fig.show()
```

Figure Factories

Figure factories (included in `plotly.py` in the `plotly.figure_factory` module) are functions that produce graph object figures, often to satisfy the needs of specialized domains. Here's an exam-

ple of using the `create_quiver()` figure factory to construct a graph object figure that displays a 2D quiver plot.

```
import numpy as np
import plotly.figure_factory as ff

x1,y1 = np.meshgrid(np.arange(0, 2, .2), np.arange(0, 2, .2))
u1 = np.cos(x1)*y1
v1 = np.sin(x1)*y1

fig = ff.create_quiver(x1, y1, u1, v1)

fig.show()
```

Make Subplots

The `plotly.subplots.make_subplots()` function produces a graph object figure that is preconfigured with a grid of subplots that traces can be added to. The `add_trace()` function will be discussed more below.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_trace(go.Scatter(y=[4, 2, 1], mode="lines"), row=1, col=1)
fig.add_trace(go.Bar(y=[2, 1, 3]), row=1, col=2)

fig.show()
```

Updating Figures

Regardless of how a graph object figure was constructed, it can be updated by adding additional traces to it and modifying its properties.

Adding Traces

New traces can be added to a graph object figure using the `add_trace()` method. This method accepts a graph object trace (an instance of `go.Scatter`, `go.Bar`, etc.) and adds it to the figure. This allows you to start with an empty figure, and add traces to it sequentially. The `append_trace()` method does the same thing, although it does not return the figure.

```
import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Bar(x=[1, 2, 3], y=[1, 3, 2]))

fig.show()
```

You can also add traces to a figure produced by a figure factory or Plotly Express.

```
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
```

```
                title="Using The add_trace() method With A Plotly Express Figure")

fig.add_trace(
    go.Scatter(
        x=[2, 4],
        y=[4, 8],
        mode="lines",
        line=go.scatter.Line(color="gray"),
        showlegend=False)
)
fig.show()
```

Adding Traces To Subplots

If a figure was created using `plotly.subplots.make_subplots()`, then supplying the `row` and `col` arguments to `add_trace()` can be used to add a trace to a particular subplot.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_trace(go.Scatter(y=[4, 2, 1], mode="lines"), row=1, col=1)
fig.add_trace(go.Bar(y=[2, 1, 3]), row=1, col=2)

fig.show()
```

This also works for figures created by Plotly Express using the `facet_row` and or `facet_col` arguments.

```
import plotly.express as px
import plotly.graph_objects as go

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", facet_col="species",
                 title="Adding Traces To Subplots Within A Plotly Express Figure")

reference_line = go.Scatter(x=[2, 4],
                            y=[4, 8],
                            mode="lines",
                            line=go.scatter.Line(color="gray"),
                            showlegend=False)

fig.add_trace(reference_line, row=1, col=1)
fig.add_trace(reference_line, row=1, col=2)
fig.add_trace(reference_line, row=1, col=3)

fig.show()
```

Add Trace Convenience Methods

As an alternative to the `add_trace()` method, graph object figures have a family of methods of the form `add_{trace}` (where `{trace}` is the name of a trace type) for constructing and adding traces of each trace type.

Here is the previous subplot example, adapted to add the scatter trace using `fig.add_scatter()` and to add the bar trace using `fig.add_bar()`.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 1], mode="lines", row=1, col=1)
fig.add_bar(y=[2, 1, 3], row=1, col=2)

fig.show()
```

Magic Underscore Notation

To make it easier to work with nested properties, graph object constructors and many graph object methods support magic underscore notation.

This allows you to reference nested properties by joining together multiple nested property names with underscores.

For example, specifying the figure title in the figure constructor *without* magic underscore notation requires setting the `layout` argument to `dict(title=dict(text="A Chart"))`.

Similarly, setting the line color of a scatter trace requires setting the `marker` property to `dict(color="crimson")`.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Scatter(y=[1, 3, 2], line=dict(color="crimson"))],
    layout=dict(title=dict(text="A Graph Objects Figure Without Magic Underscore Notation"))
)

fig.show()
```

With magic underscore notation, you can accomplish the same thing by passing the figure constructor a keyword argument named `layout_title_text`, and by passing the `go.Scatter` constructor a keyword argument named `line_color`.

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Scatter(y=[1, 3, 2], line_color="crimson")],
    layout_title_text="A Graph Objects Figure With Magic Underscore Notation"
)

fig.show()
```

Magic underscore notation is supported throughout the graph objects API, and it can often significantly simplify operations involving deeply nested properties.

Note: When you see keyword arguments with underscores passed to a graph object constructor or method, it is almost always safe to assume that it is an application of magic underscore notation. We have to say “almost always” rather than “always” because there are a few property names in the plotly schema that contain underscores: error_x, error_y, error_z, copy_xstyle, copy_ystyle, copy_zstyle, paper_bgcolor, and plot_bgcolor. These were added back in the early days of the library (2012-2013) before we standardized on banning underscores from property names.

Updating Figure Layouts

Graph object figures support an `update_layout()` method that may be used to update multiple nested properties of a figure's layout.

Here is an example of updating the text and font size of a figure's title using `update_layout()`.

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Bar(x=[1, 2, 3], y=[1, 3, 2]))

fig.update_layout(title_text="Using update_layout() With Graph Object Figures",
                  title_font_size=30)

fig.show()
```

Note that the following `update_layout()` operations are equivalent:

```
fig.update_layout(title_text="update_layout() Syntax Example",
                  title_font_size=30)

fig.update_layout(title_text="update_layout() Syntax Example",
                  title_font=dict(size=30))


fig.update_layout(title=dict(text="update_layout() Syntax Example"),
                             font=dict(size=30))

fig.update_layout({"title": {"text": "update_layout() Syntax Example",
                             "font": {"size": 30}}})

fig.update_layout(title=go.layout.Title(text="update_layout() Syntax Example",
                                        font=go.layout.title.Font(size=30)))
```

Updating Traces

Graph object figures support an `update_traces()` method that may be used to update multiple nested properties of one or more of a figure's traces.

To show some examples, we will start with a figure that contains `bar` and `scatter` traces across two subplots.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 3.5], mode="markers",
                marker=dict(size=20, color="LightSeaGreen"),
                name="a", row=1, col=1)

fig.add_bar(y=[2, 1, 3],
            marker=dict(color="MediumPurple"),
            name="b", row=1, col=1)

fig.add_scatter(y=[2, 3.5, 4], mode="markers",
                marker=dict(size=20, color="MediumPurple"),
                name="c", row=1, col=2)

fig.add_bar(y=[1, 3, 2],
            marker=dict(color="LightSeaGreen"),
            name="d", row=1, col=2)
```

```
fig.show()
```

Note that both `scatter` and `bar` traces have a `marker.color` property to control their coloring. Here is an example of using `update_traces()` to modify the color of all traces.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 3.5], mode="markers",
                marker=dict(size=20, color="LightSeaGreen"),
                name="a", row=1, col=1)

fig.add_bar(y=[2, 1, 3],
            marker=dict(color="MediumPurple"),
            name="b", row=1, col=1)

fig.add_scatter(y=[2, 3.5, 4], mode="markers",
                marker=dict(size=20, color="MediumPurple"),
                name="c", row=1, col=2)

fig.add_bar(y=[1, 3, 2],
            marker=dict(color="LightSeaGreen"),
            name="d", row=1, col=2)

fig.update_traces(marker=dict(color="RoyalBlue"))

fig.show()
```

The `update_traces()` method supports a `selector` argument to control which traces should be updated. Only traces with properties that match the selector will be updated. Here is an example of using a selector to only update the color of the `bar` traces.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 3.5], mode="markers",
                marker=dict(size=20, color="LightSeaGreen"),
                name="a", row=1, col=1)

fig.add_bar(y=[2, 1, 3],
            marker=dict(color="MediumPurple"),
            name="b", row=1, col=1)

fig.add_scatter(y=[2, 3.5, 4], mode="markers",
                marker=dict(size=20, color="MediumPurple"),
                name="c", row=1, col=2)

fig.add_bar(y=[1, 3, 2],
            marker=dict(color="LightSeaGreen"),
            name="d", row=1, col=2)

fig.update_traces(marker=dict(color="RoyalBlue"),
                  selector=dict(type="bar"))

fig.show()
```

Magic underscore notation can be used in the selector to match nested properties. Here is an

example of updating the color of all traces that were formally colored `"MediumPurple"`.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 3.5], mode="markers",
                marker=dict(size=20, color="LightSeaGreen"),
                name="a", row=1, col=1)

fig.add_bar(y=[2, 1, 3],
            marker=dict(color="MediumPurple"),
            name="b", row=1, col=1)

fig.add_scatter(y=[2, 3.5, 4], mode="markers",
                marker=dict(size=20, color="MediumPurple"),
                name="c", row=1, col=2)

fig.add_bar(y=[1, 3, 2],
            marker=dict(color="LightSeaGreen"),
            name="d", row=1, col=2)

fig.update_traces(marker_color="RoyalBlue",
                  selector=dict(marker_color="MediumPurple"))

fig.show()
```

For figures with subplots, the `update_traces()` method also supports `row` and `col` arguments to control which traces should be updated. Only traces in the specified subplot row and column will be updated. Here is an example of updating the color of all traces in the second subplot column.

```
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2)

fig.add_scatter(y=[4, 2, 3.5], mode="markers",
                marker=dict(size=20, color="LightSeaGreen"),
                name="a", row=1, col=1)

fig.add_bar(y=[2, 1, 3],
            marker=dict(color="MediumPurple"),
            name="b", row=1, col=1)

fig.add_scatter(y=[2, 3.5, 4], mode="markers",
                marker=dict(size=20, color="MediumPurple"),
                name="c", row=1, col=2)

fig.add_bar(y=[1, 3, 2],
            marker=dict(color="LightSeaGreen"),
            name="d", row=1, col=2)

fig.update_traces(marker=dict(color="RoyalBlue"),
                  col=2)

fig.show()
```

The `update_traces()` method can also be used on figures produced by figure factories or Plotly Express. Here's an example of updating the regression lines produced by Plotly Express to be dotted.

```
import pandas as pd
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
                 facet_col="species", trendline="ols", title="Using update_traces() With Plotly Express
↪  Figures")

fig.update_traces(
    line=dict(dash="dot", width=4),
    selector=dict(type="scatter", mode="lines"))

fig.show()
```

Overwrite Existing Properties When Using Update Methods

`update_layout()` and `update_traces()` have an `overwrite` keyword argument, defaulting to False, in which case updates are applied recursively to the *existing* nested property structure. When set to True, the prior value of existing properties is overwritten with the provided value.

In the example below, the red color of markers is overwritten when updating `marker` in `update_traces()` with `overwrite=True`. Note that setting instead `marker_opacity` with the magic underscore would not overwrite `marker_color` because properties would be overwritten starting only at the level of `marker.opacity`.

```
import plotly.graph_objects as go

fig = go.Figure(go.Bar(x=[1, 2, 3], y=[6, 4, 9],
                       marker_color="red")) # will be overwritten below

fig.update_traces(overwrite=True, marker={"opacity": 0.4})

fig.show()
```

Conditionally Updating Traces

Suppose the updates that you want to make to a collection of traces depend on the current values of certain trace properties. The `update_traces()` method cannot handle this situation, but the `for_each_trace()` method can!

As its first argument, the `for_each_trace()` method accepts a function that accepts and updates one trace at a time. Like `update_traces()`, `for_each_trace()` also accepts `selector`, `row`, and `col` arguments to control which traces should be considered.

Here is an example of using `for_each_trace()` to convert the only markers for the `"setosa"` to square symbols in a Plotly Express Figure.

Note that this is possible because Plotly Express figures are made up of a separate trace for each column in the input data frame

```
import pandas as pd
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
```

```
                 title="Conditionally Updating Traces In A Plotly Express Figure With for_each_trace()")

fig.for_each_trace(
    lambda trace: trace.update(marker_symbol="square") if trace.name == "setosa" else (),
)

fig.show()
```

Updating Figure Axes

Graph object figures support `update_xaxes()` and `update_yaxes()` methods that may be used to update multiple nested properties of one or more of a figure's axes. Here is an example of using `update_xaxes()` to disable the vertical grid lines across all subplots in a figure produced by Plotly Express.

```
import pandas as pd
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
                 facet_col="species", title="Using update_xaxes() With A Plotly Express Figure")

fig.update_xaxes(showgrid=False)

fig.show()
```

There are also `for_each_xaxis()` and `for_each_yaxis()` methods that are analogous to the `for_each_trace()` method described above. For non-cartesian subplot types (e.g. polar), there are additional `update_{type}` and `for_each_{type}` methods (e.g. `update_polar()`, `for_each_polar()`).

Other Update Methods

Figures created with the plotly.py graphing library also support:

- the `update_layout_images()` method in order to update background layout images,
- `update_annotations()` in order to update annotations,
- and `update_shapes()` in order to update shapes.

Chaining Figure Operations

All of the figure update operations described above are methods that return a reference to the figure being modified. This makes it possible to chain multiple figure modification operations together into a single expression.

Here is an example of a chained expression that creates:

- a faceted scatter plot with OLS trend lines using Plotly Express,
- sets the title font size using `update_layout()`,
- disables vertical grid lines using `update_xaxes()`,
- updates the width and dash pattern of the trend lines using `update_traces()`,
- and then displays the figure using `show()`.

```
import plotly.express as px

df = px.data.iris()
```

```
(px.scatter(df, x="sepal_width", y="sepal_length", color="species",
            facet_col="species", trendline="ols",
            title="Chaining Multiple Figure Operations With A Plotly Express Figure")
 .update_layout(title_font_size=24)
 .update_xaxes(showgrid=False)
 .update_traces(
     line=dict(dash="dot", width=4),
     selector=dict(type="scatter", mode="lines"))
).show()
```

Property Assignment

Trace and layout properties can be updated using property assignment syntax. Here is an example of setting the figure title using property assignment.

```
import plotly.graph_objects as go
fig = go.Figure(data=go.Bar(x=[1, 2, 3], y=[1, 3, 2]))
fig.layout.title.text = "Using Property Assignment Syntax With A Graph Object Figure"
fig.show()
```

And here is an example of updating the bar outline using property assignment.

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Bar(x=[1, 2, 3], y=[1, 3, 2]))

fig.data[0].marker.line.width = 4
fig.data[0].marker.line.color = "black"

fig.show()
```

Plotly Express

Source: https://plotly.com/python/plotly-express/

Overview

The `plotly.express` module (usually imported as `px`) contains functions that can create entire figures at once, and is referred to as Plotly Express or PX. Plotly Express is a built-in part of the `plotly` library, and is the recommended starting point for creating most common figures. Every Plotly Express function uses graph objects internally and returns a `plotly.graph_objects.Figure` instance. Throughout the `plotly` documentation, you will find the Plotly Express way of building figures at the top of any applicable page, followed by a section on how to use graph objects to build similar figures. Any figure created in a single function call with Plotly Express could be created using graph objects alone, but with between 5 and 100 times more code.

Plotly Express provides more than 30 functions for creating different types of figures. The API for these functions was carefully designed to be as consistent and easy to learn as possible, making it easy to switch from a scatter plot to a bar chart to a histogram to a sunburst chart throughout a data exploration session. *Scroll down for a gallery of Plotly Express plots, each made in a single function call.*

Here is a talk from the SciPy 2021 conference that gives a good introduction to Plotly Express and Dash:

Plotly Express currently includes the following functions:

- **Basics**: `scatter`, `line`, `area`, `bar`, `funnel`, `timeline`
- **Part-of-Whole**: `pie`, `sunburst`, `treemap`, `icicle`, `funnel_area`
- **1D Distributions**: `histogram`, `box`, `violin`, `strip`, `ecdf`
- **2D Distributions**: `density_heatmap`, `density_contour`
- **Matrix or Image Input**: `imshow`
- **3-Dimensional**: `scatter_3d`, `line_3d`
- **Multidimensional**: `scatter_matrix`, `parallel_coordinates`, `parallel_categories`
- **Tile Maps**: `scatter_map`, `line_map`, `choropleth_map`, `density_map`
- **Outline Maps**: `scatter_geo`, `line_geo`, `choropleth`
- **Polar Charts**: `scatter_polar`, `line_polar`, `bar_polar`
- **Ternary Charts**: `scatter_ternary`, `line_ternary`

High-Level Features

The Plotly Express API in general offers the following features:

- **A single entry point into `plotly`**: just `import plotly.express as px` and get access to all the plotting functions, plus built-in demo datasets under `px.data` and built-in color scales and sequences under `px.color`. Every PX function returns a `plotly.graph_objects.Figure` object, so you can edit it using all the same methods like `update_layout` and `add_trace`.
- **Sensible, Overridable Defaults**: PX functions will infer sensible defaults wherever possible, and will always let you override them.
- **Flexible Input Formats**: PX functions accept input in a variety of formats, from `lists` and `dicts` to long-form or wide-form `DataFrames` to `numpy` arrays and `xarrays` to GeoPandas `GeoDataFrames`.

- **Automatic Trace and Layout configuration**: PX functions will create one trace per animation frame for each unique combination of data values mapped to discrete color, symbol, line-dash, facet-row and/or facet-column. Traces' `legendgroup` and `showlegend` attributes are set such that only one legend item appears per unique combination of discrete color, symbol and/or line-dash. Traces are automatically linked to a correctly-configured subplot of the appropriate type.
- **Automatic Figure Labelling**: PX functions label axes, legends and colorbars based in the input `DataFrame` or `xarray`, and provide extra control with the `labels` argument.
- **Automatic Hover Labels**: PX functions populate the hover-label using the labels mentioned above, and provide extra control with the `hover_name` and `hover_data` arguments.
- **Styling Control**: PX functions read styling information from the default figure template, and support commonly-needed cosmetic controls like `category_orders` and `color_discrete_map` to precisely control categorical variables.
- **Uniform Color Handling**: PX functions automatically switch between continuous and categorical color based on the input type.
- **Faceting**: the 2D-cartesian plotting functions support row, column and wrapped facetting with `facet_row`, `facet_col` and `facet_col_wrap` arguments.
- **Marginal Plots**: the 2D-cartesian plotting functions support marginal distribution plots with the `marginal`, `marginal_x` and `marginal_y` arguments.
- **A Pandas backend**: the 2D-cartesian plotting functions are available as a Pandas plotting backend so you can call them via `df.plot()`.
- **Trendlines**: `px.scatter` supports built-in trendlines with accessible model output.
- **Animations**: many PX functions support simple animation support via the `animation_frame` and `animation_group` arguments.
- **Automatic WebGL switching**: for sufficiently large scatter plots, PX will automatically use WebGL for hardware-accelerated rendering.

Gallery

The following set of figures is just a sampling of what can be done with Plotly Express.

Scatter, Line, Area and Bar Charts

Read more about scatter plots and discrete color.

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species")
fig.show()
```

Read more about trendlines and templates and marginal distribution plots.

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", marginal_y="violin",
           marginal_x="box", trendline="ols", template="simple_white")
fig.show()
```

Read more about error bars.

```
import plotly.express as px
df = px.data.iris()
df["e"] = df["sepal_width"]/100
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", error_x="e", error_y="e")
fig.show()
```

Read more about bar charts.

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color="smoker", barmode="group")
fig.show()
```

```
import plotly.express as px
df = px.data.medals_long()

fig = px.bar(df, x="medal", y="count", color="nation",
             pattern_shape="nation", pattern_shape_sequence=[".", "x", "+"])
fig.show()
```

Read more about facet plots.

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color="smoker", barmode="group", facet_row="time",
↪  facet_col="day",
       category_orders={"day": ["Thur", "Fri", "Sat", "Sun"], "time": ["Lunch", "Dinner"]})
fig.show()
```

Read more about scatterplot matrices (SPLOMs).

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter_matrix(df, dimensions=["sepal_width", "sepal_length", "petal_width", "petal_length"],
↪  color="species")
fig.show()
```

Read more about parallel coordinates and parallel categories, as well as continuous color.

```
import plotly.express as px
df = px.data.iris()
fig = px.parallel_coordinates(df, color="species_id", labels={"species_id": "Species",
                 "sepal_width": "Sepal Width", "sepal_length": "Sepal Length",
                 "petal_width": "Petal Width", "petal_length": "Petal Length", },
                   color_continuous_scale=px.colors.diverging.Tealrose, color_continuous_midpoint=2)
fig.show()
```

```
import plotly.express as px
df = px.data.tips()
fig = px.parallel_categories(df, color="size", color_continuous_scale=px.colors.sequential.Inferno)
fig.show()
```

Read more about hover labels.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.scatter(df.query("year==2007"), x="gdpPercap", y="lifeExp", size="pop", color="continent",
           hover_name="country", log_x=True, size_max=60)
fig.show()
```

Read more about animations.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.scatter(df, x="gdpPercap", y="lifeExp", animation_frame="year", animation_group="country",
           size="pop", color="continent", hover_name="country", facet_col="continent",
           log_x=True, size_max=45, range_x=[100,100000], range_y=[25,90])
```

```
fig.show()
```

Read more about line charts.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.line(df, x="year", y="lifeExp", color="continent", line_group="country", hover_name="country",
        line_shape="spline", render_mode="svg")
fig.show()
```

Read more about area charts.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.area(df, x="year", y="pop", color="continent", line_group="country")
fig.show()
```

Read more about timeline/Gantt charts.

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame([
    dict(Task="Job A", Start='2009-01-01', Finish='2009-02-28', Resource="Alex"),
    dict(Task="Job B", Start='2009-03-05', Finish='2009-04-15', Resource="Alex"),
    dict(Task="Job C", Start='2009-02-20', Finish='2009-05-30', Resource="Max")
])

fig = px.timeline(df, x_start="Start", x_end="Finish", y="Resource", color="Resource")
fig.show()
```

Read more about funnel charts.

```
import plotly.express as px
data = dict(
    number=[39, 27.4, 20.6, 11, 2],
    stage=["Website visit", "Downloads", "Potential customers", "Requested price", "Invoice sent"])
fig = px.funnel(data, x='number', y='stage')
fig.show()
```

Part to Whole Charts

Read more about pie charts.

```
import plotly.express as px
df = px.data.gapminder().query("year == 2007").query("continent == 'Europe'")
df.loc[df['pop'] < 2.e6, 'country'] = 'Other countries' # Represent only large countries
fig = px.pie(df, values='pop', names='country', title='Population of European continent')
fig.show()
```

Read more about sunburst charts.

```
import plotly.express as px

df = px.data.gapminder().query("year == 2007")
fig = px.sunburst(df, path=['continent', 'country'], values='pop',
                  color='lifeExp', hover_data=['iso_alpha'])
fig.show()
```

Read more about treemaps.

```
import plotly.express as px
import numpy as np
df = px.data.gapminder().query("year == 2007")
fig = px.treemap(df, path=[px.Constant('world'), 'continent', 'country'], values='pop',
                 color='lifeExp', hover_data=['iso_alpha'])
fig.show()
```

Read more about icicle charts.

```
import plotly.express as px
import numpy as np
df = px.data.gapminder().query("year == 2007")
fig = px.icicle(df, path=[px.Constant('world'), 'continent', 'country'], values='pop',
                 color='lifeExp', hover_data=['iso_alpha'])
fig.show()
```

Distributions

Read more about histograms.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", y="tip", color="sex", marginal="rug", hover_data=df.columns)
fig.show()
```

Read more about box plots.

```
import plotly.express as px
df = px.data.tips()
fig = px.box(df, x="day", y="total_bill", color="smoker", notched=True)
fig.show()
```

Read more about violin plots.

```
import plotly.express as px
df = px.data.tips()
fig = px.violin(df, y="tip", x="smoker", color="sex", box=True, points="all", hover_data=df.columns)
fig.show()
```

Read more about Empirical Cumulative Distribution Function (ECDF) charts.

```
import plotly.express as px
df = px.data.tips()
fig = px.ecdf(df, x="total_bill", color="sex")
fig.show()
```

Read more about strip charts.

```
import plotly.express as px
df = px.data.tips()
fig = px.strip(df, x="total_bill", y="time", orientation="h", color="smoker")
fig.show()
```

Read more about density contours, also known as 2D histogram contours.

```
import plotly.express as px
df = px.data.iris()
fig = px.density_contour(df, x="sepal_width", y="sepal_length")
fig.show()
```

Read more about density heatmaps, also known as 2D histograms.

```
import plotly.express as px
df = px.data.iris()
fig = px.density_heatmap(df, x="sepal_width", y="sepal_length", marginal_x="rug", marginal_y="histogram")
fig.show()
```

Images and Heatmaps

Read more about heatmaps and images.

```
import plotly.express as px
data=[[1, 25, 30, 50, 1], [20, 1, 60, 80, 30], [30, 60, 1, 5, 20]]
fig = px.imshow(data,
                labels=dict(x="Day of Week", y="Time of Day", color="Productivity"),
                x=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],
                y=['Morning', 'Afternoon', 'Evening']
               )
fig.update_xaxes(side="top")
fig.show()
```

```
import plotly.express as px
from skimage import io
img = io.imread('https://user-images.githubusercontent.com/72614349/179115668-2630e3e4-3a9f-4c88-9494-
↪ 3412e606450a.jpg')
fig = px.imshow(img)
fig.show()
```

Tile Maps

Read more about tile maps and point on tile maps.

```
import plotly.express as px
df = px.data.carshare()
fig = px.scatter_map(df, lat="centroid_lat", lon="centroid_lon", color="peak_hour", size="car_hours",
                  color_continuous_scale=px.colors.cyclical.IceFire, size_max=15, zoom=10,
                  map_style="carto-positron")
fig.show()
```

Read more about tile map GeoJSON choropleths.

```
import plotly.express as px

df = px.data.election()
geojson = px.data.election_geojson()

fig = px.choropleth_map(df, geojson=geojson, color="Bergeron",
                           locations="district", featureidkey="properties.district",
                           center={"lat": 45.5517, "lon": -73.7073},
                           map_style="carto-positron", zoom=9)
fig.show()
```

Outline Maps

Read more about outline symbol maps.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.scatter_geo(df, locations="iso_alpha", color="continent", hover_name="country", size="pop",
               animation_frame="year", projection="natural earth")
fig.show()
```

Read more about choropleth maps.

```
import plotly.express as px
df = px.data.gapminder()
fig = px.choropleth(df, locations="iso_alpha", color="lifeExp", hover_name="country",
↪  animation_frame="year", range_color=[20,80])
fig.show()
```

Polar Coordinates

Read more about polar plots.

```
import plotly.express as px
df = px.data.wind()
fig = px.scatter_polar(df, r="frequency", theta="direction", color="strength", symbol="strength",
            color_discrete_sequence=px.colors.sequential.Plasma_r)
fig.show()
```

Read more about radar charts.

```
import plotly.express as px
df = px.data.wind()
fig = px.line_polar(df, r="frequency", theta="direction", color="strength", line_close=True,
            color_discrete_sequence=px.colors.sequential.Plasma_r)
fig.show()
```

Read more about polar bar charts.

```
import plotly.express as px
df = px.data.wind()
fig = px.bar_polar(df, r="frequency", theta="direction", color="strength", template="plotly_dark",
            color_discrete_sequence= px.colors.sequential.Plasma_r)
fig.show()
```

3D Coordinates

Read more about 3D scatter plots.

```
import plotly.express as px
df = px.data.election()
fig = px.scatter_3d(df, x="Joly", y="Coderre", z="Bergeron", color="winner", size="total",
↪  hover_name="district",
                 symbol="result", color_discrete_map = {"Joly": "blue", "Bergeron": "green",
↪  "Coderre":"red"})
fig.show()
```

Ternary Coordinates

Read more about ternary charts.

```
import plotly.express as px
df = px.data.election()
```

```
fig = px.scatter_ternary(df, a="Joly", b="Coderre", c="Bergeron", color="winner", size="total",
↪  hover_name="district",
                  size_max=15, color_discrete_map = {"Joly": "blue", "Bergeron": "green",
↪  "Coderre":"red"} )
fig.show()
```

Styling Plotly Express figures

Source: https://plotly.com/python/styling-plotly-express/

Styling Figures made with Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data. Every Plotly Express function returns a `plotly.graph_objects.Figure` object whose `data` and `layout` has been pre-populated according to the provided arguments.

> You can style and customize figures made with Plotly Express *in all the same ways* as you can style figures made more manually by explicitly assembling `graph_objects` into a figure.

More specifically, here are the 4 ways you can style and customize figures made with Plotly Express:

1. Control common parameters like width & height, titles, labeling and colors using built-in Plotly Express function arguments
2. Updating the figure attributes using update methods or by directly setting attributes
3. Using Plotly's theming/templating mechanism via the `template` argument to every Plotly Express function
4. Setting default values for common parameters using `px.defaults`

Built-in Plotly Express Styling Arguments

Many common styling options can be set directly in the `px` function call. Every Plotly Express function accepts the following arguments:

- `title` to set the figure title
- `width` and `height` to set the figure dimensions
- `template` to set many styling parameters at once (see below for more details)
- `labels` to override the default axis and legend labels behaviour, which is to use the data frame column name if available, and otherwise to use the label name itself like "x", "y", "color" etc. `labels` accepts a `dict` whose keys are the label to rename and whose values are the desired labels. These labels appear in axis labels, legend and color bar titles, and in hover labels.
- `category_orders` to override the default category ordering behaviour, which is to use the order in which the data appears in the input. `category_orders` accepts a `dict` whose keys are the column name to reorder and whose values are a `list` of values in the desired order. These orderings apply everywhere categories appear: in legends, on axes, in bar stacks, in the order of facets, in the order of animation frames etc.
- `hover_data` and `hover_name` to control which attributes appear in the hover label and how they are formatted.
- Various color-related attributes such as `color_continuous_scale`, `color_range`, `color_discrete_sequence` and/or `color_discrete_map` set the colors used in the figure. `color_discrete_map` accepts a dict whose keys are values mapped to `color` and whose values are the desired CSS colors.

To illustrate each of these, here is a simple, default figure made with Plotly Express. Note the default orderings for the x-axis categories and the usage of lowercase & snake_case data frame columns for axis labelling.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="day", y="total_bill", color="sex")
fig.show()
```

Here is the same figure, restyled by adding some extra parameters to the initial Plotly Express call:

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="day", y="total_bill", color="sex",
             title="Receipts by Payer Gender and Day of Week",
             width=600, height=400,
             labels={ # replaces default labels by column name
                 "sex": "Payer Gender",  "day": "Day of Week", "total_bill": "Receipts"
             },
             category_orders={ # replaces default order by column name
                 "day": ["Thur", "Fri", "Sat", "Sun"], "sex": ["Male", "Female"]
             },
             color_discrete_map={ # replaces default color mapping by value
                 "Male": "RebeccaPurple", "Female": "MediumPurple"
             },
             template="simple_white"
             )
fig.show()
```

Updating or Modifying Figures made with Plotly Express

If none of the built-in Plotly Express arguments allow you to customize the figure the way you need to, you can use the `update_*` and `add_*` methods on the `plotly.graph_objects.Figure` object returned by the PX function to make any further modifications to the figure. This approach is the one used throughout the Plotly.py documentation to customize axes, control legends and colorbars, add shapes and annotations etc.

Here is the same figure as above, with some additional customizations to the axes and legend via `.update_yaxes()`, and `.update_layout()`, as well as some annotations added via `.add_shape()` and `.add_annotation()`.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="day", y="total_bill", color="sex",
             title="Receipts by Payer Gender and Day of Week vs Target",
             width=600, height=400,
             labels={"sex": "Payer Gender",  "day": "Day of Week", "total_bill": "Receipts"},
             category_orders={"day": ["Thur", "Fri", "Sat", "Sun"], "sex": ["Male", "Female"]},
             color_discrete_map={"Male": "RebeccaPurple", "Female": "MediumPurple"},
             template="simple_white"
             )

fig.update_yaxes( # the y-axis is in dollars
    tickprefix="$", showgrid=True
)

fig.update_layout( # customize font and legend orientation & position
    font_family="Rockwell",
    legend=dict(
        title=None, orientation="h", y=1, yanchor="bottom", x=0.5, xanchor="center"
    )
```

```
)

fig.add_shape( # add a horizontal "target" line
    type="line", line_color="salmon", line_width=3, opacity=1, line_dash="dot",
    x0=0, x1=1, xref="paper", y0=950, y1=950, yref="y"
)

fig.add_annotation( # add a text callout with arrow
    text="below target!", x="Fri", y=400, arrowhead=1, showarrow=True
)

fig.show()
```

How Plotly Express Works with Templates

Plotly has a theming system based on templates and figures created with Plotly Express interact smoothly with this system:

- Plotly Express methods will use the default template if one is set in `plotly.io` (by default, this is set to `plotly`) or in `plotly.express.defaults` (see below)
- The template in use can always be overridden via the `template` argument to every PX function
- The default `color_continuous_scale` will be the value of `layout.colorscales.sequential` in the template in use, unless it is overridden via the corresponding function argument or via `plotly.express.defaults` (see below)
- The default `color_discrete_sequence` will be the value of `layout.colorway` in the template in use, unless it is overridden via the corresponding function argument or via `plotly.express.defaults` (see below)

By way of example, in the following figure, simply setting the `template` argument will automatically change the default continuous color scale, even though we have not specified `color_continuous_scale` directly.

```
import plotly.express as px
df = px.data.iris()
fig = px.density_heatmap(df, x="sepal_width", y="sepal_length", template="seaborn")
fig.show()
```

Setting Plotly Express Styling Defaults

Plotly Express supports a simple default-configuration system via the `plotly.express.defaults` singleton object. The values of the properties set on this object are used for the rest of the active session in place of `None` as the default values for any argument to a PX function with a matching name:

- `width` and `height` can be set once globally for all Plotly Express functions
- `template` can override the setting of `plotly.io.templates.default` for all Plotly Express functions
- `color_continuous_scale` and `color_discrete_scale` can override the contents of the template in use for all Plotly Express functions that accept these arguments
- `line_dash_sequence`, `symbol_sequence` and `size_max` can be set once globally for all Plotly Express functions that accept these arguments

To illustrate this “defaults hierarchy”, in the following example:

- we set the Plotly-wide default template to `simple_white`, but
- we override the default template for Plotly Express to be `ggplot2`, but

- we also set the default `color_continuous_scale`, and
- we set the default `height` and `width` to 400 by 600, but
- we override the default `width` to 400 via the function argument.

As a result, any figure produced with Plotly Express thereafter uses the `ggplot2` settings for all attributes except for the continuous color scale (visible because `simple_white` doesn't set a plot background, and neither the `simple_white` nor `ggplot2` template uses `Blackbody` as a color scale), and uses the Plotly Express defaults for height but not width (visible because the figure height is the same as the figure width, despite the default).

```
import plotly.express as px
import plotly.io as pio

pio.templates.default = "simple_white"

px.defaults.template = "ggplot2"
px.defaults.color_continuous_scale = px.colors.sequential.Blackbody
px.defaults.width = 600
px.defaults.height = 400

df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="sepal_length", width=400)
fig.show()
```

Displaying figures (renderers)

Source: https://plotly.com/python/renderers/

Displaying Figures

Plotly's Python graphing library, `plotly.py`, gives you a wide range of options for how and where to display your figures.

In general, there are six different approaches you can take in order to display `plotly` figures:

- Using the `renderers` framework in the context of a script or notebook (the main topic of this page)
- Using Plotly Studio to generate charts using natural language
- Using Dash in a web app context
- Using a `FigureWidget` rather than a `Figure` in an `ipywidgets` context
- By exporting to an HTML file and loading that file in a browser immediately or later
- By rendering the figure to a static image file using Kaleido such as PNG, JPEG, SVG or PDF and loading the resulting file in any viewer

Each of the first four approaches is discussed below.

Displaying Figures Using The `renderers` Framework

The renderers framework is a flexible approach for displaying `plotly.py` figures in a variety of contexts. To display a figure using the renderers framework, you call the `.show()` method on a graph object figure, or pass the figure to the `plotly.io.show` function. With either approach, `plotly.py` will display the figure using the current default renderer(s).

```
import plotly.graph_objects as go
fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
    layout_title_text="A Figure Displayed with fig.show()"
)
fig.show()
```

In most situations, you can omit the call to `.show()` and allow the figure to display itself.

```
import plotly.graph_objects as go
fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
    layout_title_text="A Figure Displaying Itself"
)
fig
```

> To be precise, figures will display themselves using the current default renderer when the two following conditions are true. First, the last expression in a cell must evaluate to a figure. Second, `plotly.py` must be running from within an `IPython` kernel.

In many contexts, an appropriate renderer will be chosen automatically and you will not need to perform any additional configuration. These contexts include the classic Jupyter Notebook, JupyterLab, Visual Studio Code notebooks, Google Colab, Kaggle notebooks, Azure notebooks, and the Python interactive shell.

Additional contexts are supported by choosing a compatible renderer including QtConsole, Spyder, and more.

Next, we will show how to configure the default renderer. After that, we will describe all of the built-in renderers and discuss why you might choose to use each one.

> Note: The `renderers` framework is a generalization of the `plotly.offline.iplot` and `plotly.offline.plot` functions that were the recommended way to display figures prior to `plotly.py` version 4. These functions have been reimplemented using the `renderers` framework and are still supported for backward compatibility, but they will not be discussed here.

Setting The Default Renderer

The current and available renderers are configured using the `plotly.io.renderers` configuration object. Display this object to see the current default renderer and the list of all available renderers.

```
import plotly.io as pio
pio.renderers
```

The default renderer that you see when you display `pio.renderers` might be different than what is shown here. This is because `plotly.py` attempts to autodetect an appropriate renderer at startup. You can change the default renderer by assigning the name of an available renderer to the `pio.renderers.default` property. For example, to switch to the `'browser'` renderer, which opens figures in a tab of the default web browser, you would run the following.

```
import plotly.io as pio
pio.renderers.default = "browser"
```

> Note: Default renderers persist for the duration of a single session. For example, if you set a default renderer in an `IPython` kernel, that default won't persist across kernel restarts.

It is also possible to set the default renderer using a system environment variable. At startup, `plotly.py` checks for the existence of an environment variable named `PLOTLY_RENDERER`. If this environment variable is set to the name of an available renderer, this renderer is set as the default.

Overriding The Default Renderer

You can override the default renderer temporarily by passing the name of an available renderer as the `renderer` keyword argument to a figure's `show()` method. For example, to use the `svg` renderer (described later) without changing the default renderer, set `renderer="svg"`:

```
import plotly.graph_objects as go
fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
    layout_title_text="A Figure Displayed with the 'svg' Renderer"
)
fig.show(renderer="svg")
```

Built-in Renderers

In this section, we will describe the built-in renderers so that you can choose the one(s) that best suit your needs.

Interactive Renderers

Interactive renderers display figures using the plotly.js JavaScript library and are fully interactive, supporting pan, zoom, hover tooltips, etc.

`notebook`

This renderer is intended for use in the classic Jupyter Notebook (not JupyterLab). The full plotly.js JavaScript library bundle is added to the notebook the first time a figure is rendered, so this renderer will work without an internet connection.

This renderer is a good choice for notebooks that will be exported to HTML files (Either using nbconvert or the "Download as HTML" menu action) because the exported HTML files will work without an internet connection.

> Note: Adding the plotly.js bundle to the notebook adds a few megabytes to the notebook size. If you can count on always having an internet connection, you may want to consider using the `notebook_connected` renderer if notebook size is a constraint.

`notebook_connected`

This renderer is the same as `notebook` renderer, except the plotly.js JavaScript library bundle is loaded from an online CDN location. This saves a few megabytes in notebook size, but an internet connection is required in order to display figures that are rendered this way.

This renderer is a good choice for notebooks that will be shared with nbviewer since users must have an active internet connection to access nbviewer in the first place.

`kaggle` and `azure`

These are aliases for `notebook_connected` because this renderer is a good choice for use with Kaggle Notebooks and Azure Notebooks.

`colab`

This is a custom renderer for use with Google Colab.

`browser`

This renderer will open a figure in a browser tab using the default web browser. This renderer can only be used when the Python kernel is running locally on the same machine as the web browser, so it is not compatible with Jupyter Hub or online notebook services.

> Implementation Note 1: In this context, the "default browser" is the browser that is chosen by the Python `webbrowser` module.

> Implementation Note 2: The `browser` renderer works by setting up a single use local webserver on a local port. Since the webserver is shut down as soon as the figure is served to the browser, the figure will not be restored if the browser is refreshed.

`firefox`, `chrome`, and `chromium`

These renderers are the same as the `browser` renderer, but they force the use of a particular browser.

`iframe` and `iframe_connected`

These renderers write figures out as standalone HTML files and then display `iframe` elements that reference these HTML files. The `iframe` renderer will include the plotly.js JavaScript bundle in each HTML file that is written, while the `iframe_connected` renderer includes only a reference to an online CDN location from which to load plotly.js. Consequently, the `iframe_connected` renderer outputs files that are smaller than the `iframe` renderer, but it requires an internet connection while the `iframe` renderer can operate offline.

This renderer may be useful when working with notebooks that contain lots of large figures. When using the `notebook` or `notebook_connected` renderer, all of the data for all of the figures in a notebook are stored inline in the notebook itself. If this would result in a prohibitively large notebook size, an `iframe` or `iframe_connected` renderer could be used instead. With the `iframe` renderers, the figure data are stored in the individual HTML files rather than in the notebook itself, resulting in a smaller notebook size.

> Implementation Note: The HTML files written by the `iframe` renderers are stored in a sub-directory named `iframe_figures`. The HTML files are given names based on the execution number of the notebook cell that produced the figure. This means that each time a notebook kernel is restarted, any prior HTML files will be overwritten. This also means that you should not store multiple notebooks using an `iframe` renderer in the same directory, because this could result in figures from one notebook overwriting figures from another notebook.

`plotly_mimetype`

The `plotly_mimetype` renderer creates a specification of the figure (called a MIME-type bundle), and requests that the current user interface displays it. User interfaces that support this renderer include JupyterLab, nteract, and the Visual Studio Code notebook interface.

`jupyterlab`, `nteract`, and `vscode`

These are aliases for `plotly_mimetype` since this renderer is a good choice when working in JupyterLab, nteract, and the Visual Studio Code notebook interface. Note that in VSCode Notebooks, the version of Plotly.js that is used to render is provided by the vscode-python extension and often trails the latest version by several weeks, so the latest features of `plotly` may not be available in VSCode right away. The situation is similar for Nteract.

Static Image Renderers

A set of renderers is provided for displaying figures as static images. See the Static Image Export page for more information on getting set up.

`png`, `jpeg`, and `svg`

These renderers display figures as static `.png`, `.jpeg`, and `.svg` files, respectively. These renderers are useful for user interfaces that do not support inline HTML output, but do support inline static images. Examples include the QtConsole, Spyder, and the PyCharm notebook interface.

```
import plotly.graph_objects as go
fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
    layout_title_text="A Figure Displayed with the 'png' Renderer"
)
fig.show(renderer="png")
```

PDF

This renderer displays figures as static PDF files. This is especially useful for notebooks that will be exported to PDF files using the LaTeX export capabilities of `nbconvert`.

Other Miscellaneous Renderers

JSON

In editors that support it (JupyterLab, nteract, and the Visual Studio Code notebook interface), this renderer displays the JSON representation of a figure in a collapsible interactive tree structure. This can be very useful for examining the structure of complex figures.

Multiple Renderers

You can specify that multiple renderers should be used by joining their names on `"+"` characters. This is useful when writing code that needs to support multiple contexts. For example, if a notebook specifies a default renderer string of `"notebook+plotly_mimetype+pdf"`, then this notebook would be able to run in the classic Jupyter Notebook as well as in JupyterLab, and it would support being exported to PDF using `nbconvert`.

Customizing Built-In Renderers

Most built-in renderers have configuration options to customize their behavior. To view a description of a renderer, including its configuration options, access the renderer object using dictionary-style key lookup on the `plotly.io.renderers` configuration object and then display it. Here is an example of accessing and displaying the `png` renderer.

```
import plotly.io as pio
png_renderer = pio.renderers["png"]
png_renderer
```

From this output, you can see that the `png` renderer supports 3 properties: `width`, `height`, and `scale`. You can customize these properties by assigning new values to them.

Here is an example that customizes the `png` renderer to change the resulting image size, sets the `png` renderer as the default, and then displays a figure.

```
import plotly.io as pio
png_renderer = pio.renderers["png"]
png_renderer.width = 500
png_renderer.height = 500

pio.renderers.default = "png"

import plotly.graph_objects as go
fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
    layout_title_text="A Figure Displayed with the 'png' Renderer"
)
fig.show()
```

You can also override the values of renderer parameters temporarily by passing them as keyword arguments to the `show()` method. For example

```
import plotly.graph_objects as go

fig = go.Figure(
    data=[go.Bar(y=[2, 1, 3])],
```

```
    layout_title_text="A Figure Displayed with the 'png' Renderer"
)
fig.show(renderer="png", width=800, height=300)
```

Displaying figures in Plotly Studio

Use Plotly Studio to build data apps with Plotly figures using natural language and AI. Describe the charts you want to Plotly Studio, which generates them within a Dash app that you can publish to Plotly Cloud or Dash Enterprise.

Displaying Figures Using `ipywidgets`

Plotly figures can be displayed in ipywidgets contexts using `plotly.graph_objects.FigureWidget` objects. `FigureWidget` is a figure graph object (just like `plotly.graph_objects.Figure`), so you can add traces to it and update it just like a regular `Figure`. But `FigureWidget` is also an `ipywidgets` object, which means that you can display it alongside other `ipywidgets` to build user interfaces right in the notebook.

See the Plotly FigureWidget Overview for more information on integrating `plotly.py` figures with `ipywidgets`.

It is important to note that `FigureWidget` does not use the renderers framework discussed above, so you should not use the `plotly.io.show` function on `FigureWidget` objects.

Performance

No matter the approach chosen to display a figure, the figure data structure is first (automatically, internally) serialized into a JSON string before being transferred from the Python context to the browser (or to an HTML file first or to Kaleido for static image export).

New in v5.0

The default JSON serialization mechanism can be slow for figures with many data points or with large `numpy` arrays or data frames. **If the `orjson` package is installed**, `plotly` will use that instead of the built-in `json` package, which can lead to **5-10x** speedups for large figures.

Once a figure is serialized to JSON, it must be rendered by a browser, either immediately in the user's browser, at some later point if the figure is exported to HTML, or immediately in Kaleido's internal headless browser for static image export. Rendering time is generally proportional to the total number of data points in the figure, the number of traces and the number of subplots. In situations where rendering performance is slow, we recommend considering the use of `plotly` WebGL traces to exploit GPU-accelerated rendering in the browser, or using the Datashader library to do Python-side rendering before using `px.imshow()` to render the figure.

Subplots

Source: https://plotly.com/python/subplots/

Subplots and Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures.

Plotly Express does not support arbitrary subplot capabilities, instead it supports faceting by a given data dimension, and it also supports marginal charts to display distribution information.

This page documents the usage of the lower-level `plotly.subplots` module and the `make_subplots` function it exposes to construct figures with arbitrary subplots. **Plotly Express faceting uses `make_subplots` internally** so adding traces to Plotly Express facets works just as documented here, with `fig.add_trace(..., row=<R>, col=<C>)`.

Simple Subplot

Figures with subplots are created using the `make_subplots` function from the `plotly.subplots` module.

Here is an example of creating a figure that includes two `scatter` traces which are side-by-side since there are 2 columns and 1 row in the subplot layout.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=1, cols=2)

fig.add_trace(
    go.Scatter(x=[1, 2, 3], y=[4, 5, 6]),
    row=1, col=1
)

fig.add_trace(
    go.Scatter(x=[20, 30, 40], y=[50, 60, 70]),
    row=1, col=2
)

fig.update_layout(height=600, width=800, title_text="Side By Side Subplots")
fig.show()
```

Stacked Subplots

Here is an example of creating a figure with subplots that are stacked on top of each other since there are 3 rows and 1 column in the subplot layout.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=3, cols=1)

fig.add_trace(go.Scatter(
```

```
    x=[3, 4, 5],
    y=[1000, 1100, 1200],
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=[2, 3, 4],
    y=[100, 110, 120],
), row=2, col=1)

fig.add_trace(go.Scatter(
    x=[0, 1, 2],
    y=[10, 11, 12]
), row=3, col=1)


fig.update_layout(height=600, width=600, title_text="Stacked Subplots")
fig.show()
```

Multiple Subplots

Here is an example of creating a 2 x 2 subplot grid and populating each subplot with a single `scatter` trace.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=2, cols=2, start_cell="bottom-left")

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]),
              row=1, col=1)

fig.add_trace(go.Scatter(x=[20, 30, 40], y=[50, 60, 70]),
              row=1, col=2)

fig.add_trace(go.Scatter(x=[300, 400, 500], y=[600, 700, 800]),
              row=2, col=1)

fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]),
              row=2, col=2)

fig.show()
```

Multiple Subplots with Titles

The `subplot_titles` argument to `make_subplots` can be used to position text annotations as titles for each subplot.

Here is an example of adding subplot titles to a 2 x 2 subplot grid of scatter traces.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    subplot_titles=("Plot 1", "Plot 2", "Plot 3", "Plot 4"))

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]),
              row=1, col=1)
```

```
fig.add_trace(go.Scatter(x=[20, 30, 40], y=[50, 60, 70]),
              row=1, col=2)

fig.add_trace(go.Scatter(x=[300, 400, 500], y=[600, 700, 800]),
              row=2, col=1)

fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]),
              row=2, col=2)

fig.update_layout(height=500, width=700,
                  title_text="Multiple Subplots with Titles")

fig.show()
```

Customizing the Subplot Title Font

New in 6.8

Use the `font` argument of `make_subplots` to customize the font used for `subplot_titles`, `column_titles`, `row_titles`, `x_title`, and `y_title`. The argument accepts a dict with any of the standard font attributes (`family`, `size`, `color`, `weight`, etc.).

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=1, cols=2,
    subplot_titles=("Plot 1", "Plot 2"),
    font=dict(family="Courier New, monospace", size=20, color="RebeccaPurple"),
)

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]), row=1, col=1)
fig.add_trace(go.Scatter(x=[20, 30, 40], y=[50, 60, 70]), row=1, col=2)

fig.show()
```

Subplots with Annotations

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=1, cols=2)

fig.add_trace(
    go.Scatter(
        x=[1, 2, 3],
        y=[4, 5, 6],
        mode="markers+text",
        text=["Text A", "Text B", "Text C"],
        textposition="bottom center"
    ),
    row=1, col=1
)

fig.add_trace(
    go.Scatter(
```

```
        x=[20, 30, 40],
        y=[50, 60, 70],
        mode="markers+text",
        text=["Text D", "Text E", "Text F"],
        textposition="bottom center"
    ),
    row=1, col=2
)

fig.update_layout(height=600, width=800, title_text="Subplots with Annotations")

fig.show()
```

Customize Subplot Column Widths and Row Heights

The `column_widths` argument to `make_subplots` can be used to customize the relative widths of the columns in a subplot grid. It should be set to a list of numbers with a length that matches the `cols` argument. These number will be normalized, so that they sum to 1, and used to compute the relative widths of the subplot grid columns. The `row_heights` argument serves the same purpose for controlling the relative heights of rows in the subplot grid.

Here is an example of creating a figure with two scatter traces in side-by-side subplots. The left subplot is set to be wider than the right one.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=1, cols=2, column_widths=[0.7, 0.3])

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]),
              row=1, col=1)

fig.add_trace(go.Scatter(x=[20, 30, 40], y=[50, 60, 70]),
              row=1, col=2)

fig.show()
```

Customizing Subplot Axes

After a figure with subplots is created using the `make_subplots` function, its axis properties (title, font, range, grid style, etc.) can be customized using the `update_xaxes` and `update_yaxes` graph object figure methods. By default, these methods apply to all of the x axes or y axes in the figure. The `row` and `col` arguments can be used to control which axes are targeted by the update.

Here is an example that creates a figure with a 2 x 2 subplot grid, populates each subplot with a scatter trace, and then updates the x and y axis titles for each subplot individually.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Initialize figure with subplots
fig = make_subplots(
    rows=2, cols=2, subplot_titles=("Plot 1", "Plot 2", "Plot 3", "Plot 4")
)

# Add traces
fig.add_trace(go.Scatter(x=[1, 2, 3], y=[4, 5, 6]), row=1, col=1)
fig.add_trace(go.Scatter(x=[20, 30, 40], y=[50, 60, 70]), row=1, col=2)
```

```
fig.add_trace(go.Scatter(x=[300, 400, 500], y=[600, 700, 800]), row=2, col=1)
fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]), row=2, col=2)

# Update xaxis properties
fig.update_xaxes(title_text="xaxis 1 title", row=1, col=1)
fig.update_xaxes(title_text="xaxis 2 title", range=[10, 50], row=1, col=2)
fig.update_xaxes(title_text="xaxis 3 title", showgrid=False, row=2, col=1)
fig.update_xaxes(title_text="xaxis 4 title", type="log", row=2, col=2)

# Update yaxis properties
fig.update_yaxes(title_text="yaxis 1 title", row=1, col=1)
fig.update_yaxes(title_text="yaxis 2 title", range=[40, 80], row=1, col=2)
fig.update_yaxes(title_text="yaxis 3 title", showgrid=False, row=2, col=1)
fig.update_yaxes(title_text="yaxis 4 title", row=2, col=2)

# Update title and height
fig.update_layout(title_text="Customizing Subplot Axes", height=700)

fig.show()
```

Subplots with Shared X-Axes

The `shared_xaxes` argument to `make_subplots` can be used to link the x axes of subplots in the resulting figure. The `vertical_spacing` argument is used to control the vertical spacing between rows in the subplot grid.

Here is an example that creates a figure with 3 vertically stacked subplots with linked x axes. A small vertical spacing value is used to reduce the spacing between subplot rows.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=3, cols=1,
                    shared_xaxes=True,
                    vertical_spacing=0.02)

fig.add_trace(go.Scatter(x=[0, 1, 2], y=[10, 11, 12]),
              row=3, col=1)

fig.add_trace(go.Scatter(x=[2, 3, 4], y=[100, 110, 120]),
              row=2, col=1)

fig.add_trace(go.Scatter(x=[3, 4, 5], y=[1000, 1100, 1200]),
              row=1, col=1)

fig.update_layout(height=600, width=600,
                  title_text="Stacked Subplots with Shared X-Axes")
fig.show()
```

Subplots with Shared Y-Axes

The `shared_yaxes` argument to `make_subplots` can be used to link the y axes of subplots in the resulting figure.

Here is an example that creates a figure with a 2 x 2 subplot grid, where the y axes of each row are linked.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=2, cols=2, shared_yaxes=True)

fig.add_trace(go.Scatter(x=[1, 2, 3], y=[2, 3, 4]),
              row=1, col=1)

fig.add_trace(go.Scatter(x=[20, 30, 40], y=[5, 5, 5]),
              row=1, col=2)

fig.add_trace(go.Scatter(x=[2, 3, 4], y=[600, 700, 800]),
              row=2, col=1)

fig.add_trace(go.Scatter(x=[4000, 5000, 6000], y=[7000, 8000, 9000]),
              row=2, col=2)

fig.update_layout(height=600, width=600,
                  title_text="Multiple Subplots with Shared Y-Axes")
fig.show()
```

Subplots with Shared Colorscale

To share colorscale information in multiple subplots, you can use coloraxis.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(rows=1, cols=2, shared_yaxes=True)

fig.add_trace(go.Bar(x=[1, 2, 3], y=[4, 5, 6],
                     marker=dict(color=[4, 5, 6], coloraxis="coloraxis")),
              1, 1)

fig.add_trace(go.Bar(x=[1, 2, 3], y=[2, 3, 5],
                     marker=dict(color=[2, 3, 5], coloraxis="coloraxis")),
              1, 2)

fig.update_layout(coloraxis=dict(colorscale='Bluered_r'), showlegend=False)
fig.show()
```

Custom Sized Subplot with Subplot Titles

The `specs` argument to `make_subplots` is used to configure per-subplot options. `specs` must be a 2-dimension list with dimensions that match those provided as the `rows` and `cols` arguments. The elements of `specs` may either be `None`, indicating no subplot should be initialized starting with this grid cell, or a dictionary containing subplot options. The `colspan` subplot option specifies the number of grid columns that the subplot starting in the given cell should occupy. If unspecified, `colspan` defaults to 1.

Here is an example that creates a 2 by 2 subplot grid containing 3 subplots. The subplot `specs` element for position (2, 1) has a `colspan` value of 2, causing it to span the full figure width. The subplot `specs` element for position (2, 2) is `None` because no subplot begins at this location in the grid.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go
```

```
fig = make_subplots(
    rows=2, cols=2,
    specs=[[{}, {}],
           [{"colspan": 2}, None]],
    subplot_titles=("First Subplot","Second Subplot", "Third Subplot"))

fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2]),
                  row=1, col=1)

fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2]),
                  row=1, col=2)
fig.add_trace(go.Scatter(x=[1, 2, 3], y=[2, 1, 2]),
                  row=2, col=1)

fig.update_layout(showlegend=False, title_text="Specs with Subplot Title")
fig.show()
```

Multiple Custom Sized Subplots

If the `print_grid` argument to `make_subplots` is set to `True`, then a text representation of the subplot grid will be printed.

Here is an example that uses the `rowspan` and `colspan` subplot options to create a custom subplot layout with subplots of mixed sizes. The `print_grid` argument is set to `True` so that the subplot grid is printed to the screen.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=5, cols=2,
    specs=[[{}, {"rowspan": 2}],
           [{}, None],
           [{"rowspan": 2, "colspan": 2}, None],
           [None, None],
           [{}, {}]],
    print_grid=True)

fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(1,1)"), row=1, col=1)
fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(1,2)"), row=1, col=2)
fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(2,1)"), row=2, col=1)
fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(3,1)"), row=3, col=1)
fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(5,1)"), row=5, col=1)
fig.add_trace(go.Scatter(x=[1, 2], y=[1, 2], name="(5,2)"), row=5, col=2)

fig.update_layout(height=600, width=600, title_text="specs examples")
fig.show()
```

Subplots Types

By default, the `make_subplots` function assumes that the traces that will be added to all subplots are 2-dimensional cartesian traces (e.g. `scatter`, `bar`, `histogram`, `violin`, etc.). Traces with other subplot types (e.g. `scatterpolar`, `scattergeo`, `parcoords`, etc.) are supported by specifying the `type` subplot option in the `specs` argument to `make_subplots`.

Here are the possible values for the `type` option:

- `"xy"`: 2D Cartesian subplot type for scatter, bar, etc. This is the default if no `type` is specified.
- `"scene"`: 3D Cartesian subplot for scatter3d, cone, etc.
- `"polar"`: Polar subplot for scatterpolar, barpolar, etc.
- `"ternary"`: Ternary subplot for scatterternary.
- `"map"`: Map subplot for scattermap.
- `"domain"`: Subplot type for traces that are individually positioned. pie, parcoords, parcats, etc.
- trace type: A trace type name (e.g. `"bar"`, `"scattergeo"`, `"carpet"`, `"mesh"`, etc.) which will be used to determine the appropriate subplot type for that trace.

Here is an example that creates and populates a 2 x 2 subplot grid containing 4 different subplot types.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    specs=[[{"type": "xy"}, {"type": "polar"}],
           [{"type": "domain"}, {"type": "scene"}]],
)

fig.add_trace(go.Bar(y=[2, 3, 1]),
              row=1, col=1)

fig.add_trace(go.Barpolar(theta=[0, 45, 90], r=[2, 3, 1]),
              row=1, col=2)

fig.add_trace(go.Pie(values=[2, 3, 1]),
              row=2, col=1)

fig.add_trace(go.Scatter3d(x=[2, 3, 1], y=[0, 0, 0],
                           z=[0.5, 1, 2], mode="lines"),
              row=2, col=2)

fig.update_layout(height=700, showlegend=False)

fig.show()
```

As an alternative to providing the name of a subplot type (e.g. `"xy"`, `"polar"`, `"domain"`, `"scene"`, etc), the `type` option may also be set to a string containing the name of a trace type (e.g. `"bar"`, `"barpolar"`, `"pie"`, `"scatter3d"`, etc.), which will be used to determine the subplot type that is compatible with that trace.

Here is the example above, modified to specify the subplot types using trace type names.

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=2, cols=2,
    specs=[[{"type": "bar"}, {"type": "barpolar"}],
           [{"type": "pie"}, {"type": "scatter3d"}]],
)

fig.add_trace(go.Bar(y=[2, 3, 1]),
              row=1, col=1)

fig.add_trace(go.Barpolar(theta=[0, 45, 90], r=[2, 3, 1]),
              row=1, col=2)
```

```
fig.add_trace(go.Pie(values=[2, 3, 1]),
              row=2, col=1)

fig.add_trace(go.Scatter3d(x=[2, 3, 1], y=[0, 0, 0],
                           z=[0.5, 1, 2], mode="lines"),
              row=2, col=2)

fig.update_layout(height=700, showlegend=False)

fig.show()
```

Side by Side Subplot (low-level API)

```
import plotly.graph_objects as go

trace1 = go.Scatter(
    x=[1, 2, 3],
    y=[4, 5, 6]
)
trace2 = go.Scatter(
    x=[20, 30, 40],
    y=[50, 60, 70],
    xaxis="x2",
    yaxis="y2"
)
data = [trace1, trace2]
layout = go.Layout(
    xaxis=dict(
        domain=[0, 0.7]
    ),
    xaxis2=dict(
        domain=[0.8, 1]
    ),
    yaxis2=dict(
        anchor="x2"
    )
)
fig = go.Figure(data=data, layout=layout)
fig.show()
```

Subplots with shared axes (low-level API)

```
import plotly.graph_objects as go

trace1 = go.Scatter(
    x=[1, 2, 3],
    y=[2, 3, 4]
)
trace2 = go.Scatter(
    x=[20, 30, 40],
    y=[5, 5, 5],
    xaxis="x2",
    yaxis="y"
)
trace3 = go.Scatter(
```

```
    x=[2, 3, 4],
    y=[600, 700, 800],
    xaxis="x",
    yaxis="y3"
)
trace4 = go.Scatter(
    x=[4000, 5000, 6000],
    y=[7000, 8000, 9000],
    xaxis="x4",
    yaxis="y4"
)
data = [trace1, trace2, trace3, trace4]
layout = go.Layout(
    xaxis=dict(
        domain=[0, 0.45]
    ),
    yaxis=dict(
        domain=[0, 0.45]
    ),
    xaxis2=dict(
        domain=[0.55, 1]
    ),
    xaxis4=dict(
        domain=[0.55, 1],
        anchor="y4"
    ),
    yaxis3=dict(
        domain=[0.55, 1]
    ),
    yaxis4=dict(
        domain=[0.55, 1],
        anchor="x4"
    )
)
fig = go.Figure(data=data, layout=layout)
fig.show()
```

Stacked Subplots with a Shared X-Axis (low-level API)

```
import plotly.graph_objects as go

trace1 = go.Scatter(
    x=[0, 1, 2],
    y=[10, 11, 12]
)
trace2 = go.Scatter(
    x=[2, 3, 4],
    y=[100, 110, 120],
    yaxis="y2"
)
trace3 = go.Scatter(
    x=[3, 4, 5],
    y=[1000, 1100, 1200],
    yaxis="y3"
)
data = [trace1, trace2, trace3]
layout = go.Layout(
    yaxis=dict(
```

```
        domain=[0, 0.33]
    ),
    legend=dict(
        traceorder="reversed"
    ),
    yaxis2=dict(
        domain=[0.33, 0.66]
    ),
    yaxis3=dict(
        domain=[0.66, 1]
    )
)
fig = go.Figure(data=data, layout=layout)
fig.show()
```

Setting Subplots on a Figure Directly

new in 4.13

Subplots can be added to an already existing figure, provided it doesn't already have subplots. `go.Figure.set_subplots` accepts all the same arguments as `plotly.subplots.make_subplots`.

```
import plotly.graph_objects as go
fig = go.Figure().set_subplots(2, 3, horizontal_spacing=0.1)
```

is equivalent to:

```
from plotly.subplots import make_subplots
fig = make_subplots(2, 3, horizontal_spacing=0.1)
```

Reference

All of the x-axis properties are found here: https://plotly.com/python/reference/layout/xaxis/ All of the y-axis properties are found here: https://plotly.com/python/reference/layout/yaxis/

```
from plotly.subplots import make_subplots
help(make_subplots)
```

Theming and templates

Source: *https://plotly.com/python/templates/*

Theming and templates

The Plotly Python library comes pre-loaded with several themes that you can get started using right away, and it also provides support for creating and registering your own themes.

> Note on terminology: Theming generally refers to the process of defining default styles for visual elements. Themes in plotly are implemented using objects called templates. Templates are slightly more general than traditional themes because in addition to defining default styles, templates can pre-populate a figure with visual elements like annotations, shapes, images, and more. In the documentation we will refer to the overall process of defining default styles as theming, and when it comes to the plotly API we will talk about how themes are implemented using templates.

Using built-in themes

View available themes

To see information about the available themes and the current default theme, display the `plotly.io.templates` configuration object like this.

```
import plotly.io as pio
pio.templates
```

From this, we can see that the default theme is `"plotly"`, and we can see the names of several additional themes that we can choose from.

Specifying themes in Plotly Express

All Plotly Express functions accept a `template` argument that can be set to the name of a registered theme (or to a `Template` object as discussed later in this section). Here is an example of using Plotly Express to build and display the same scatter plot with six different themes.

```
import plotly.express as px

df = px.data.gapminder()
df_2007 = df.query("year==2007")

for template in ["plotly", "plotly_white", "plotly_dark", "ggplot2", "seaborn", "simple_white", "none"]:
    fig = px.scatter(df_2007,
                     x="gdpPercap", y="lifeExp", size="pop", color="continent",
                     log_x=True, size_max=60,
                     template=template, title="Gapminder 2007: '%s' theme" % template)
    fig.show()
```

Specifying themes in graph object figures

The theme for a particular graph object figure can be specified by setting the `template` property of the figure's `layout` to the name of a registered theme (or to a `Template` object as discussed later in this section). Here is an example of constructing a surface plot and then displaying it with each of six themes.

```
import plotly.graph_objects as go
import pandas as pd

z_data = pd.read_csv(
↪  "https://raw.githubusercontent.com/plotly/datasets/master/api_docs/mt_bruno_elevation.csv")

fig = go.Figure(
    data=go.Surface(z=z_data.values),
    layout=go.Layout(
        title=dict(text="Mt Bruno Elevation"),
        width=500,
        height=500,
    ))

for template in ["plotly", "plotly_white", "plotly_dark", "ggplot2", "seaborn", "simple_white", "none"]:
    fig.update_layout(template=template, title=dict(text="Mt Bruno Elevation: '%s' theme" % template))
    fig.show()
```

Specifying a default theme

If a theme is not provided to a Plotly Express function or to a graph object figure, then the default theme is used. The default theme starts out as `"plotly"`, but it can be changed by setting the `plotly.io.templates.default` property to the name of a registered theme.

Here is an example of changing to default theme to `"plotly_white"` and then constructing a scatter plot with Plotly Express without providing a template.

> Note: Default themes persist for the duration of a single session, but they do not persist across sessions. If you are working in an IPython kernel, this means that default themes will persist for the life of the kernel, but they will not persist across kernel restarts.

```
import plotly.io as pio
import plotly.express as px

pio.templates.default = "plotly_white"

df = px.data.gapminder()
df_2007 = df.query("year==2007")

fig = px.scatter(df_2007,
                 x="gdpPercap", y="lifeExp", size="pop", color="continent",
                 log_x=True, size_max=60,
                 title="Gapminder 2007: current default theme")
fig.show()
```

Disable default theming

If you do not wish to use any of the new themes by default, or you want your figures to look exactly the way they did prior to plotly.py version 4, you can disable default theming by setting the default theme to `"none"`.

```
import plotly.io as pio
pio.templates.default = "none"
```

Creating themes

Representing themes with Template objects

Themes in plotly.py are represented by instances of the `Template` class from the `plotly.graph_objects.layout` module. A `Template` is a graph object that contains two top-level properties: `layout` and `data`. These template properties are described in their own sections below.

The template layout property

The `layout` property of a template is a graph object with the exact same structure as the `layout` property of a figure. When you provide values for properties of the template's `layout`, these values will be used as the defaults in any figure that this template is applied to.

Here is an example that creates a template that sets the default title font to size 24 Rockwell, and then constructs a graph object figure with this template.

```
import plotly.graph_objects as go

large_rockwell_template = dict(
    layout=go.Layout(title_font=dict(family="Rockwell", size=24))
)

fig = go.Figure()
fig.update_layout(title=dict(text="Figure Title"),
                  template=large_rockwell_template)
fig.show()
```

> Note: this example uses magic underscore notation to write `go.Layout(title=dict(font=dict(...)))` as `go.Layout(title_font=dict(...))`

The template data property

The `data` property of a template is used to customize the default values of the properties of traces that are added to a figure that the template is applied to. This `data` property holds a graph object, with type `go.layout.template.Data`, that has a property named after each supported trace type. These trace type properties are then assigned lists or tuples of graph object traces of the corresponding type.

Here is an example that creates a template that sets the default scatter trace markers to be size 20 diamonds, and then constructs a graph object figure with this template.

```
import plotly.graph_objects as go

diamond_template = go.layout.Template()
diamond_template.data.scatter = [go.Scatter(marker=dict(symbol="diamond", size=20))]

fig = go.Figure()
fig.update_layout(template=diamond_template)
fig.add_scatter(y=[2, 1, 3], mode="markers")
fig.show()
```

If a trace type property is set to a list of more than one trace, then the default properties are cycled as more traces are added to the figure. Here is an example that creates a template that cycles the default marker symbol for scatter traces, and then constructs a figure that uses this template.

```
import plotly.graph_objects as go

symbol_template = go.layout.Template()
symbol_template.data.scatter = [
    go.Scatter(marker=dict(symbol="diamond", size=10)),
    go.Scatter(marker=dict(symbol="square", size=10)),
    go.Scatter(marker=dict(symbol="circle", size=10)),
]

fig = go.Figure()
fig.update_layout(template=symbol_template)
fig.add_scatter(y=[1, 2, 3], mode="markers", name="first")
fig.add_scatter(y=[2, 3, 4], mode="markers", name="second")
fig.add_scatter(y=[3, 4, 5], mode="markers", name="third")
fig.add_scatter(y=[4, 5, 6], mode="markers", name="forth")
fig.show()
```

Note that because we built the template with a list of 3 scatter trace graph objects (one each for the diamond, square, and circle symbols), the forth scatter trace in the figure cycles around and takes on the defaults specified in the first template trace (The diamond symbol).

Theming object tuple properties

Some properties in the figure hierarchy are specified as tuples of objects. For example, the text annotations for a graph object figure can be stored as a tuple of `go.layout.Annotation` objects in the `annotations` property of the figure's layout.

To use a template to configure the default properties of all of the elements in an object tuple property (e.g. `layout.annotations`), use the `*defaults` property in the template that corresponds to the tuple property (e.g. `layout.template.layout.annotationdefaults`). The `*defaults` template property should be set to a single graph object that matches the type of the elements of the corresponding tuple. The properties of this `*defaults` object in the template will be applied to all elements of the object tuple in the figure that the template is applied to.

Here is an example that creates a template that specifies the default annotation text color, and then constructs a figure that uses this template.

```
import plotly.graph_objects as go

annotation_template = go.layout.Template()
annotation_template.layout.annotationdefaults = dict(font=dict(color="crimson"))

fig = go.Figure()
fig.update_layout(
     template=annotation_template,
     annotations=[
         dict(text="Look Here", x=1, y=1),
         dict(text="Look There", x=2, y=2)
     ]
 )
fig.show()
```

Including tuple elements in a theme

The previous section described how to use a template to customize the default properties of tuple elements that are added to a figure that the template is applied to. This is useful for styling, for example, any annotations, shapes, or images that will eventually be added to the figure.

It is also possible for a template to define tuple elements that should be included, as is, in any figure that the template is applied to. This is done by specifying a list of one or more tuple element graph objects (e.g. `go.layout.Annotation` objects) as the value of the corresponding tuple property in the template (e.g. at `template.layout.annotations`). Note that the `name` property of these tuple element graph objects must be set to a unique non-empty string.

Here is an example that creates a template that adds a large semi-transparent "DRAFT" watermark to the middle of the figure, and constructs a figure using this template.

```
import plotly.graph_objects as go

draft_template = go.layout.Template()
draft_template.layout.annotations = [
    dict(
        name="draft watermark",
        text="DRAFT",
        textangle=-30,
        opacity=0.1,
        font=dict(color="black", size=100),
        xref="paper",
        yref="paper",
        x=0.5,
        y=0.5,
        showarrow=False,
    )
]

fig=go.Figure()
fig.update_layout(template=draft_template)
fig.show()
```

Customizing theme tuple elements in a figure

The previous section described how a template can be used to add default tuple element graph objects (e.g. annotations, shapes, or images) to a figure. The properties of these default tuple elements can be customized from within the figure by adding an tuple element with a `templateitemname` property that matches the `name` property of the template object.

Here is an example, using the same `draft_template` defined above, that customizes the watermark from within the figure to read "CONFIDENTIAL" rather than "DRAFT".

```
import plotly.graph_objects as go

draft_template = go.layout.Template()
draft_template.layout.annotations = [
    dict(
        name="draft watermark",
        text="DRAFT",
        textangle=-30,
        opacity=0.1,
        font=dict(color="black", size=100),
        xref="paper",
        yref="paper",
        x=0.5,
```

```
        y=0.5,
        showarrow=False,
    )
]

fig = go.Figure()
fig.update_layout(
    template=draft_template,
    annotations=[
        dict(
            templateitemname="draft watermark",
            text="CONFIDENTIAL",
        )
    ]
)
fig.show()
```

Registering themes as named templates

The examples above construct and configure a `Template` object and then pass that object as the template specification to graph object figures (as the `layout.template` property) or Plotly Express functions (as the `template` keyword argument). It is also possible to register custom templates by name so that the name itself can be used to refer to the template. To register a template, use dictionary-style assignment to associate the template object with a name in the `plotly.io.templates` configuration object.

Here is an example of registering the draft watermark template from the previous sections as a template named `"draft"`. Then a graph object figure is created with the draft template specified by name.

```
import plotly.graph_objects as go
import plotly.io as pio

pio.templates["draft"] = go.layout.Template(
    layout_annotations=[
        dict(
            name="draft watermark",
            text="DRAFT",
            textangle=-30,
            opacity=0.1,
            font=dict(color="black", size=100),
            xref="paper",
            yref="paper",
            x=0.5,
            y=0.5,
            showarrow=False,
        )
    ]
)

fig = go.Figure()
fig.update_layout(template="draft")
fig.show()
```

> Note: this example uses magic underscore notation to write `go.layout.Template(layout=dict(annotations=[...]))` as `go.layout.Template(layout_annotations=[...])`

It is also possible to set your own custom template as the default so that you do not need to pass

it by name when constructing graph object figures or calling Plotly Express functions.

```
import plotly.graph_objects as go
import plotly.io as pio

pio.templates["draft"] = go.layout.Template(
    layout_annotations=[
        dict(
            name="draft watermark",
            text="DRAFT",
            textangle=-30,
            opacity=0.1,
            font=dict(color="black", size=100),
            xref="paper",
            yref="paper",
            x=0.5,
            y=0.5,
            showarrow=False,
        )
    ]
)
pio.templates.default = "draft"

fig = go.Figure()
fig.show()
```

Combining themes

You may have noticed that figures displayed with the custom templates defined above do not have the gray background and white gridlines that are part of the default styling of figures created with plotly.py. The reason for this is that the default styling is specified in a template named `"plotly"`, and specifying a custom template overrides the default `"plotly"` template.

If you want the styling of a custom template to be applied on top of the default styling of the `"plotly"` template, then you will need to combine the custom template with the `"plotly"` template. Multiple registered templates (whether built-in or user-defined) can be combined by specifying a template string that contains multiple template names joined on `"+"` characters.

Here is an example of setting the default template to be a combination of the built-in `"plotly"` template and the custom `"draft"` template from the previous example.

```
import plotly.graph_objects as go
import plotly.io as pio

pio.templates["draft"] = go.layout.Template(
    layout_annotations=[
        dict(
            name="draft watermark",
            text="DRAFT",
            textangle=-30,
            opacity=0.1,
            font=dict(color="black", size=100),
            xref="paper",
            yref="paper",
            x=0.5,
            y=0.5,
            showarrow=False,
        )
    ]
)
```

```
pio.templates.default = "plotly+draft"

fig = go.Figure()
fig.show()
```

Combining themes is also supported by Plotly Express

```
import plotly.io as pio
import plotly.graph_objects as go
import plotly.express as px

pio.templates["draft"] = go.layout.Template(
    layout_annotations=[
        dict(
            name="draft watermark",
            text="DRAFT",
            textangle=-30,
            opacity=0.1,
            font=dict(color="black", size=100),
            xref="paper",
            yref="paper",
            x=0.5,
            y=0.5,
            showarrow=False,
        )
    ]
)
pio.templates.default = "plotly+draft"

df = px.data.gapminder()
df_2007 = df.query("year==2007")

fig = px.scatter(df_2007,
                 x="gdpPercap", y="lifeExp", size="pop", color="continent",
                 log_x=True, size_max=60,
                 title="Gapminder 2007: current default theme")
fig.show()
```

<!- #region ->

Saving and distributing custom themes

The easiest way to save and distribute a custom template is to make a `*.py` file that creates and registers the template when it is imported. Here is an example of the contents of a file called `my_themes.py` that creates and registers the `"draft"` template when it is imported

my_themes.py

```
import plotly.graph_objects as go
import plotly.io as pio

pio.templates["draft"] = go.layout.Template(
    layout_annotations=[
        dict(
            name="draft watermark",
            text="DRAFT",
            textangle=-30,
            opacity=0.1,
```

```
            font=dict(color="black", size=100),
            xref="paper",
            yref="paper",
            x=0.5,
            y=0.5,
            showarrow=False,
        )
    ]
)
```

To get access to the `"draft"` template, import the `my_themes` module.

```
import my_themes
import plotly.io as pio
pio.templates.default = "draft"
...
```

> Note: In order for the import to succeed, the `my_themes.py` file must be on Python's module search path. See https://docs.python.org/3/tutorial/modules.html#the-module-search-path for more information.

<!- #endregion ->

Examining built-in themes

It may be useful to examine the contents and structure of the built-in templates when creating your own custom templates. The `Template` graph object for a registered template can be loaded using dictionary-style key access on the `plotly.io.templates` configuration object. Here is an example of loading the `Template` graph object for the `"plotly"` template, and then displaying the value of the template's `layout` property.

```
import plotly.io as pio
plotly_template = pio.templates["plotly"]
plotly_template.layout
```

Interactive HTML export

Source: https://plotly.com/python/interactive-html-export/

Interactive vs Static Export

Plotly figures are interactive when viewed in a web browser: you can hover over data points, pan and zoom axes, and show and hide traces by clicking or double-clicking on the legend. You can export figures either to static image file formats like PNG, JPEG, SVG or PDF or you can export them to HTML files which can be opened in a browser. This page explains how to do the latter.

<!– #region –> ### Saving to an HTML file

Any figure can be saved as an HTML file using the `write_html` method. These HTML files can be opened in any web browser to access the fully interactive figure.

```
import plotly.express as px

fig = px.scatter(x=range(10), y=range(10))
fig.write_html("path/to/file.html")
```

<!– #endregion –>

Controlling the size of the HTML file

By default, the resulting HTML file is a fully self-contained HTML file which can be uploaded to a web server or shared via email or other file-sharing mechanisms. The downside to this approach is that the file is very large (5Mb+) because it contains an inlined copy of the Plotly.js library required to make the figure interactive. This can be controlled via the `include_plotlyjs` argument (see below).

Inserting Plotly Output into HTML using a Jinja2 Template

You can insert Plotly output and text related to your data into HTML templates using Jinja2. Use `.to_html` to send the HTML to a Python string variable rather than using `write_html` to send the HTML to a disk file. Use the `full_html=False` option to output just the code necessary to add a figure to a template. We do not want to output a full HTML page, as the template will define the rest of the page's structure — for example, the page's `HTML` and `BODY` tags. First create a template file containing HTML and a Jinja variable, `{{ fig }}`. We use Python to replace the Jinja variable with our graphic `fig`.

<!– #region –>

```
&lt;!DOCTYPE html&gt;
&lt;html&gt;
&lt;head&gt;
&lt;meta charset="utf-8" /&gt;  &lt;!--It is necessary to use the UTF-8 encoding with plotly graphics to get e.g
&lt;meta name="viewport" content="width=device-width, initial-scale=1.0" /&gt;
&lt;/head&gt;
```

```
&lt;body&gt;
&lt;h1&gt;Here's a Plotly graph!&lt;/h1&gt;
{{ fig }}
&lt;p&gt;And here's some text after the graph.&lt;/p&gt;
&lt;/body&gt;
&lt;/html&gt;
```

Then use the following Python to replace `{{ fig }}` in the template with HTML that will display the Plotly figure "fig":

```
import plotly.express as px
from jinja2 import Template

data_canada = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(data_canada, x='year', y='pop')

output_html_path=r"/path/to/output.html"
input_template_path = r"/path/to/template.html"

plotly_jinja_data = {"fig":fig.to_html(full_html=False)}
#consider also defining the include_plotlyjs parameter to point to an external Plotly.js as described
 ↪  above

with open(output_html_path, "w", encoding="utf-8") as output_file:
    with open(input_template_path) as template_file:
        j2_template = Template(template_file.read())
        output_file.write(j2_template.render(plotly_jinja_data))
```

The height of the figure can be made responsive by setting `autosize` to `True` and `height` to `None` in the layout, and `responsive` to `True` in the config.

```
import plotly.express as px
from jinja2 import Template

data_canada = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(data_canada, x='year', y='pop')

output_html_path=r"/path/to/output.html"
input_template_path = r"/path/to/template.html"

fig.update_layout(autosize=True, height=None)
fig_html = fig.to_html(
    config={"responsive": True},
    default_height="80vh",
    full_html=False,
)
plotly_jinja_data = {"fig": fig_html}
#consider also defining the include_plotlyjs parameter to point to an external Plotly.js as described
 ↪  above

with open(output_html_path, "w", encoding="utf-8") as output_file:
    with open(input_template_path) as template_file:
        j2_template = Template(template_file.read())
        output_file.write(j2_template.render(plotly_jinja_data))
```

<!- #endregion ->

Reference

See the following links for parameter information and HTML figure export options:

- `plotly.io.to_html()`
- `plotly.io.write_html()`

Static image export

Source: https://plotly.com/python/static-image-export/

This page demonstrates how to export interactive Plotly figures to static image formats like PNG, JPEG, SVG, and PDF. If you want to export Plotly figures to HTML to retain interactivity, see the Interactive HTML Export page

<!– #region –> ## Install Dependencies

Kaleido

Static image generation requires Kaleido version 1.0.0 or greater.

Install Kaleido with pip:

```
$ pip install --upgrade kaleido
```

or with conda:

```
$ conda install -c conda-forge python-kaleido
```

Chrome

Kaleido uses Chrome for static image generation. Starting with Kaleido version 1.0.0, Chrome is not included in the package itself; instead, Kaleido looks for a compatible version of Chrome (or Chromium) already installed on the machine on which it's running.

If you don't have Chrome installed, you can install it directly from Google following the instructions for your operating system.

Plotly also provides a CLI for installing Chrome from the command line.

Run `plotly_get_chrome` to install Chrome.

You can also install Chrome from Python using `plotly.io.get_chrome()`

```
import plotly.io as pio

pio.get_chrome()
```

See the **Additional Information on Browsers with Kaleido** section below for more details on browser compatibility for Kaleido.

<!– #endregion –>

Write Image to a File

Plotly figures have a `write_image` method to write a figure to a file. `write_image` supports PNG, JPEG, WebP, SVG, and PDF formats.

To export a figure using `write_image`, call `write_image` on the figure, and pass as an argument the filename where you want to save the figure. The file format is inferred from the extension:

Raster Formats

PNG ~~~python import plotly.express as px data_canada = px.data.gapminder().query("country == 'Canada' ") fig = px.bar(data_canada, x='year', y='pop') fig.write_image("fig1.png") ~~~

JPEG

```
...
fig.write_image("images/fig1.jpeg")
```

WebP

```
...
fig.write_image("images/fig1.webp")
```

Vector Formats

SVG ~~~python ... fig.write_image("images/fig1.svg") ~~~

PDF

```
...
fig.write_image("images/fig1.pdf")
```

Note: Figures containing WebGL traces (i.e. of type `scattergl`, `scatter3d`, `surface`, `mesh3d`, `scatterpolargl`, `cone`, `streamtube`, `splom`, or `parcoords`) that are exported in a vector format will include encapsulated rasters, instead of vectors, for some parts of the image. This is a fundamental limitation of WebGL, which renders to a grid of pixels rather than to individual shapes.

Note that Plotly Express automatically uses WebGL for scatter-type figures with more than 1,000 data points: the default `render_mode="auto"` switches from `scatter` to `scattergl` above this threshold. If your exported SVG or PDF unexpectedly contains rasterized markers, this is likely why. To force fully-vector output, pass `render_mode="svg"` to the Plotly Express function, or use `go.Scatter` instead of `go.Scattergl` with graph objects. Note that fully-vector rendering can be slow for figures with many points, which is the reason WebGL is used by default above 1,000 points. See WebGL Rendering for more on this tradeoff.

Specify a Format

In the earlier example, Plotly inferred the image format from the extension of the filename. You can also specify the format explicitly using the `format` parameter.

```
import plotly.express as px
data_canada = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(data_canada, x='year', y='pop')
fig.write_image("fig1", format="png")
```

<!– #region –> ### Write Multiple Images

Kaleido v1 and later

`plotly.io` provides a `write_images` function for writing multiple figures to images. Using `write_images` is faster than calling `fig.write_image` multiple times.

`write_images` takes a list of figure objects or dicts representing figures as its first argument, `fig`. The second argument `file` is a list of paths to export to. These paths can be specified as `str`s or `pathlib.Path` objects.

```
import plotly.graph_objects as go
import plotly.express as px
import plotly.io as pio
```

```
fig1 = go.Figure(
    data=go.Scatter(x=[1, 2, 3], y=[4, 5, 6], mode='lines+markers'),
    layout=go.Layout(title='Line Chart')
)

fig2 = go.Figure(
    data=go.Bar(x=['A', 'B', 'C'], y=[10, 5, 15]),
    layout=go.Layout(title='Bar Chart')
)

fig3 = px.pie(
    values=[30, 20, 10, 40],
    names=['A', 'B', 'C', 'D'],
    title='Pie Chart'
)

pio.write_images(
    fig=[fig1, fig2, fig3],
    file=['export_images/line_chart.png', 'export_images/bar_chart.png', 'export_images/pie_chart.png']
)
```

<!- #endregion ->

Get Image as Bytes

As well as exporting to a file, Plotly figures also support conversion to a bytes object. To convert a figure to a **PNG** bytes object, call the figure's `to_image` method with a `format`

```
import plotly.express as px
data_canada = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(data_canada, x='year', y='pop')
img_bytes = fig.to_image(format="png")
```

Here's the bytes object displayed using `IPython.display.Image`:

```
from IPython.display import Image
Image(img_bytes)
```

Specify Image Dimensions and Scale

In addition to the image format, the `to_image` and `write_image` functions provide arguments to specify the image `width` and `height` in logical pixels. They also provide a `scale` parameter that can be used to increase (`scale` > 1) or decrease (`scale` < 1) the physical resolution of the resulting image.

```
img_bytes = fig.to_image(format="png", width=600, height=350, scale=2)
Image(img_bytes)
```

<!- #region -> ## plotly.io Functions

Previous examples on this page access `write_image` and `to_image` as methods on Plotly Figure objects. This functionality is also available via the `plotly.io` subpackage.

The following example uses the `write_image` function from `plotly.io`. The function takes the figure or a `dict` representing a figure (as shown in the example) as its first argument.

```
import plotly.io as pio


fig = dict({
    "data": [{"type": "bar",
              "x": [1, 2, 3],
              "y": [1, 3, 2]}],
    "layout": {"title": {"text": "A Figure Specified By Python Dictionary"}}
})

pio.write_image(fig, "fig.png")
```

<!– #endregion –>

Image Export Settings

As well as configuring height, width, and other settings by passing arguments when calling `write_image` and `to_image`, you can also set a single default to be used throughout the duration of the program.

How to Configure

Image export settings can be configured by setting their values in `plotly.io.defaults`.

To set the `default_format` to "jpeg":

```
import plotly.io as pio
pio.defaults.default_format = "jpeg"
```

You can also access current defaults. To see the default value for height:

```
import plotly.io as pio
pio.defaults.default_height
```

Available Settings

The following settings are available:

`default_width`: The default pixel width to use on image export.

`default_height`: The default pixel height to use on image export.

`default_scale`: The default image scale factor applied on image export.

`default_format`: The default image format used on export. One of "png", "jpeg", "webp", "svg", or "pdf".

`mathjax`: Location of the MathJax bundle needed to render LaTeX characters. Defaults to a CDN location. If fully offline export is required, set this to a local MathJax bundle.

`plotlyjs`: Location of the Plotly.js bundle to use. Can be a local file path or URL. By default, Kaleido uses the Plotly.js bundle included with Plotly.py.

`topojson`: Location of the topojson files needed to render choropleth traces. Defaults to a CDN location. If fully offline export is required, set this to a local directory containing the Plotly.js topojson files.

`headers`: *New in 6.8.* A dict of HTTP headers Kaleido sends when fetching external resources during image export (for example, when fetching OpenStreetMap tiles for tile maps). Defaults

to `{"X-Requested-With": "plotly.py"}`, which is required to comply with the OpenStreetMap tile usage policy. Requires Kaleido v1.3.0 or later.

Additional Information on Browsers with Kaleido

When exporting images from Plotly.py, Kaleido will attempt to find a version of Chrome or Chromium that it can use for the export. It checks in the operating system's PATH for executables with the following names: "chromium", "chromium-browser", "chrome", "Chrome", "google-chrome" "google-chrome-stable", "Chrome.app", "Google Chrome", "Google Chrome.app", and "Google Chrome for Testing".

Kaleido will also check the following locations:

Windows

- r"c:\Program Files\Google\Chrome\Application\chrome.exe"
- f"c:\Users\{os.environ.get('USER', 'default')}\AppData\"
- "Local\Google\Chrome\Application\chrome.exe"

Linux"

- "/usr/bin/google-chrome-stable"
- "/usr/bin/google-chrome"
- "/usr/bin/chrome"

Mac OS

- "/Applications/Google Chrome.app/Contents/MacOS/Google Chrome"

Most recent versions of Chrome or Chromium should work with Kaleido. Other Chromium-based browsers may also work, though Kaleido won't discover them automatically. You can set a browser to use by setting the path to search using an environment variable called `BROWSER_PATH`. For example:

```
BROWSER_PATH=/Applications/Microsoft\ Edge.app/Contents/MacOS/Microsoft\ Edge
```

Scatter plots

Source: *https://plotly.com/python/line-and-scatter/*

Scatter plots with Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures.

With `px.scatter`, each data point is represented as a marker point, whose location is given by the `x` and `y` columns.

```
# x and y given as array_like objects
import plotly.express as px
fig = px.scatter(x=[0, 1, 2, 3, 4], y=[0, 1, 4, 9, 16])
fig.show()
```

```
# x and y given as DataFrame columns
import plotly.express as px
df = px.data.iris() # iris is a pandas DataFrame
fig = px.scatter(df, x="sepal_width", y="sepal_length")
fig.show()
```

Setting size and color with column names

Scatter plots with variable-sized circular markers are often known as bubble charts. Note that `color` and `size` data are added to hover information. You can add other columns to hover data with the `hover_data` argument of `px.scatter`.

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species",
                 size='petal_length', hover_data=['petal_width'])
fig.show()
```

Color can be continuous as follows, or discrete/categorical as above.

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color='petal_length')
fig.show()
```

The `symbol` argument can be mapped to a column as well. A wide variety of symbols are available.

```
import plotly.express as px
df = px.data.iris()
fig = px.scatter(df, x="sepal_width", y="sepal_length", color="species", symbol="species")
fig.show()
```

Line plots with Plotly Express

```
import plotly.express as px
import numpy as np

t = np.linspace(0, 2*np.pi, 100)

fig = px.line(x=t, y=np.cos(t), labels={'x':'t', 'y':'cos(t)'})
fig.show()
```

```
import plotly.express as px
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x='year', y='lifeExp', color='country')
fig.show()
```

The `markers` argument can be set to `True` to show markers on lines.

```
import plotly.express as px
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x='year', y='lifeExp', color='country', markers=True)
fig.show()
```

The `symbol` argument can be used to map a data field to the marker symbol. A wide variety of symbols are available.

```
import plotly.express as px
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x='year', y='lifeExp', color='country', symbol="country")
fig.show()
```

Line plots on Date axes

Line plots can be made on using any type of cartesian axis, including linear, logarithmic, categorical or date axes. Line plots on date axes are often called time-series charts.

Plotly auto-sets the axis type to a date format when the corresponding data are either ISO-formatted date strings or if they're a date pandas column or datetime NumPy array.

```
import plotly.express as px

df = px.data.stocks()
fig = px.line(df, x='date', y="GOOG")
fig.show()
```

Data Order in Scatter and Line Charts

Plotly line charts are implemented as connected scatterplots (see below), meaning that the points are plotted and connected with lines **in the order they are provided, with no automatic reordering**.

This makes it possible to make charts like the one below, but also means that it may be required to explicitly sort data before passing it to Plotly to avoid lines moving "backwards" across the chart.

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame(dict(
```

```
    x = [1, 3, 2, 4],
    y = [1, 2, 3, 4]
))
fig = px.line(df, x="x", y="y", title="Unsorted Input")
fig.show()

df = df.sort_values(by="x")
fig = px.line(df, x="x", y="y", title="Sorted Input")
fig.show()
```

Connected Scatterplots

In a connected scatterplot, two continuous variables are plotted against each other, with a line connecting them in some meaningful order, usually a time variable. In the plot below, we show the "trajectory" of a pair of countries through a space defined by GDP per Capita and Life Expectancy. Botswana's life expectancy

```
import plotly.express as px

df = px.data.gapminder().query("country in ['Canada', 'Botswana']")

fig = px.line(df, x="lifeExp", y="gdpPercap", color="country", text="year")
fig.update_traces(textposition="bottom right")
fig.show()
```

Swarm (or Beeswarm) Plots

Swarm plots show the distribution of values in a column by giving each entry one dot and adjusting the y-value so that dots do not overlap and appear symmetrically around the y=0 line. They complement histograms, box plots, and violin plots. This example could be generalized to implement a swarm plot for multiple categories by adjusting the y-coordinate for each category.

```
import pandas as pd
import plotly.express as px
import collections


def negative_1_if_count_is_odd(count):
    # if this is an odd numbered entry in its bin, make its y coordinate negative
    # the y coordinate of the first entry is 0, so entries 3, 5, and 7 get
    # negative y coordinates
    if count % 2 == 1:
        return -1
    else:
        return 1


def swarm(
    X_series,
    fig_title,
    point_size=16,
    fig_width=800,
    gap_multiplier=1.2,
    bin_fraction=0.95,  # slightly undersizes the bins to avoid collisions
):
    # sorting will align columns in attractive c-shaped arcs rather than having
    # columns that vary unpredictably in the x-dimension.
    # We also exploit the fact that sorting means we see bins sequentially when
```

```
    # we add collision prevention offsets.
    X_series = X_series.copy().sort_values()

    # we need to reason in terms of the marker size that is measured in px
    # so we need to think about each x-coordinate as being a fraction of the way from the
    # minimum X value to the maximum X value
    min_x = min(X_series)
    max_x = max(X_series)

    list_of_rows = []
    # we will count the number of points in each "bin" / vertical strip of the graph
    # to be able to assign a y-coordinate that avoids overlapping
    bin_counter = collections.Counter()

    for x_val in X_series:
        # assign this x_value to bin number
        # each bin is a vertical strip slightly narrower than one marker
        bin = (((fig_width*bin_fraction*(x_val-min_x))/(max_x-min_x)) // point_size)

        # update the count of dots in that strip
        bin_counter.update([bin])

        # remember the "y-slot" which tells us the number of points in this bin and is sufficient to
        ↪  compute the y coordinate unless there's a collision with the point to its left
        list_of_rows.append(
            {"x": x_val, "y_slot": bin_counter[bin], "bin": bin})

    # iterate through the points and "offset" any that are colliding with a
    # point to their left apply the offsets to all subsequent points in the same bin.
    # this arranges points in an attractive swarm c-curve where the points
    # toward the edges are (weakly) further right.
    bin = 0
    offset = 0
    for row in list_of_rows:
        if bin != row["bin"]:
            # we have moved to a new bin, so we need to reset the offset
            bin = row["bin"]
            offset = 0
        # see if we need to "look left" to avoid a possible collision
        for other_row in list_of_rows:
            if (other_row["bin"] == bin-1):
                # "bubble" the entry up until we find a slot that avoids a collision
                while ((other_row["y_slot"] == row["y_slot"]+offset)
                       and (((fig_width*(row["x"]-other_row["x"]))/(max_x-min_x)
                             // point_size) < 1)):
                    offset += 1
                    # update the bin count so we know whether the number of
                    # *used* slots is even or odd
                    bin_counter.update([bin])

        row["y_slot"] += offset
        # The collision free y coordinate gives the items in a vertical bin
        # y-coordinates to evenly spread their locations above and below the
        # y-axis (we'll make a correction below to deal with even numbers of
        # entries).  For now, we'll assign 0, 1, -1, 2, -2, 3, -3 ... and so on.
        # We scale this by the point_size*gap_multiplier to get a y coordinate
        # in px.
        row["y"] = (row["y_slot"]//2) * \
            negative_1_if_count_is_odd(row["y_slot"])*point_size*gap_multiplier
```

```
    # if the number of points is even, move y-coordinates down to put an equal
    # number of entries above and below the axis
    for row in list_of_rows:
        if bin_counter[row["bin"]] % 2 == 0:
            row["y"] -= point_size*gap_multiplier/2

    df = pd.DataFrame(list_of_rows)
    # One way to make this code more flexible to e.g. handle multiple categories
    # would be to return a list of "swarmified" y coordinates here and then plot
    # outside the function.
    # That generalization would let you "swarmify" y coordinates for each
    # category and add category specific offsets to put the each category in its
    # own row

    fig = px.scatter(
        df,
        x="x",
        y="y",
        title=fig_title,
    )
    # we want to suppress the y coordinate in the hover value because the
    # y-coordinate is irrelevant/misleading
    fig.update_traces(
        marker_size=point_size,
        # suppress the y coordinate because the y-coordinate is irrelevant
        hovertemplate="value: %{x}",
    )
    # we have to set the width and height because we aim to avoid icon collisions
    # and we specify the icon size in the same units as the width and height
    fig.update_layout(width=fig_width, height=(
        point_size*max(bin_counter.values())+200))
    fig.update_yaxes(
        showticklabels=False,  # Turn off y-axis labels
        ticks='',               # Remove the ticks
        title=""
    )
    return fig


df = px.data.iris()  # iris is a pandas DataFrame
fig = swarm(df["sepal_length"], "Sepal length distribution from 150 iris samples")
# The iris data set entries are rounded so there are no collisions.
# a more interesting test case for collision avoidance is:
# fig = swarm(pd.Series([1, 1.5, 1.78, 1.79, 1.85, 2,
#            2, 2, 2, 3, 3, 2.05, 2.1, 2.2, 2.5, 12]))
fig.show()
```

Scatter and line plots with go.Scatter

If Plotly Express does not provide a good starting point, it is possible to use the more generic `go.Scatter` class from `plotly.graph_objects`. Whereas `plotly.express` has two functions `scatter` and `line`, `go.Scatter` can be used both for plotting points (makers) or lines, depending on the value of `mode`. The different options of `go.Scatter` are documented in its reference page.

Simple Scatter Plot

```
import plotly.graph_objects as go
import numpy as np

N = 1000
t = np.linspace(0, 10, 100)
y = np.sin(t)

fig = go.Figure(data=go.Scatter(x=t, y=y, mode='markers'))

fig.show()
```

Line and Scatter Plots

Use `mode` argument to choose between markers, lines, or a combination of both. For more options about line plots, see also the line charts notebook and the filled area plots notebook.

```
import plotly.graph_objects as go

# Create random data with numpy
import numpy as np
np.random.seed(1)

N = 100
random_x = np.linspace(0, 1, N)
random_y0 = np.random.randn(N) + 5
random_y1 = np.random.randn(N)
random_y2 = np.random.randn(N) - 5

fig = go.Figure()

# Add traces
fig.add_trace(go.Scatter(x=random_x, y=random_y0,
                    mode='markers',
                    name='markers'))
fig.add_trace(go.Scatter(x=random_x, y=random_y1,
                    mode='lines+markers',
                    name='lines+markers'))
fig.add_trace(go.Scatter(x=random_x, y=random_y2,
                    mode='lines',
                    name='lines'))

fig.show()
```

Bubble Scatter Plots

In bubble charts, a third dimension of the data is shown through the size of markers. For more examples, see the bubble chart notebook

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Scatter(
    x=[1, 2, 3, 4],
    y=[10, 11, 12, 13],
    mode='markers',
    marker=dict(size=[40, 60, 80, 100],
                color=[0, 1, 2, 3])
```

```
))

fig.show()
```

Style Scatter Plots

```
import plotly.graph_objects as go
import numpy as np


t = np.linspace(0, 10, 100)

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=t, y=np.sin(t),
    name='sin',
    mode='markers',
    marker_color='rgba(152, 0, 0, .8)'
))

fig.add_trace(go.Scatter(
    x=t, y=np.cos(t),
    name='cos',
    marker_color='rgba(255, 182, 193, .9)'
))

# Set options common to all traces with fig.update_traces
fig.update_traces(mode='markers', marker_line_width=2, marker_size=10)
fig.update_layout(title=dict(text='Styled Scatter'),
                  yaxis_zeroline=False, xaxis_zeroline=False)


fig.show()
```

Data Labels on Hover

```
import plotly.graph_objects as go
import pandas as pd

data= pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/2014_usa_states.csv")

fig = go.Figure(data=go.Scatter(x=data['Postal'],
                                y=data['Population'],
                                mode='markers',
                                marker_color=data['Population'],
                                text=data['State'])) # hover text goes here

fig.update_layout(title=dict(text='Population of USA States'))
fig.show()
```

Scatter with a Color Dimension

```
import plotly.graph_objects as go
import numpy as np
```

```
fig = go.Figure(data=go.Scatter(
    y = np.random.randn(500),
    mode='markers',
    marker=dict(
        size=16,
        color=np.random.randn(500), #set color equal to a variable
        colorscale='Viridis', # one of plotly colorscales
        showscale=True
    )
))

fig.show()
```

Trace Zorder

New in 5.21

For many trace types, including `go.Scatter`, you can define the order traces are drawn in by setting a `zorder`. Traces with a higher `zorder` appear at the front, with traces with a lower `zorder` at the back. In this example, we give our trace for 'France' the highest `zorder`, meaning it is drawn in front of the other two traces:

```
import plotly.graph_objects as go
import plotly.data as data

df = data.gapminder()

df_europe = df[df['continent'] == 'Europe']

trace1 = go.Scatter(x=df_europe[df_europe['country'] == 'France']['year'],
                    y=df_europe[df_europe['country'] == 'France']['lifeExp'],
                    mode='lines+markers',
                    zorder=3,
                    name='France',
                    marker=dict(size=15))

trace2 = go.Scatter(x=df_europe[df_europe['country'] == 'Germany']['year'],
                    y=df_europe[df_europe['country'] == 'Germany']['lifeExp'],
                    mode='lines+markers',
                    zorder=1,
                    name='Germany',
                    marker=dict(size=15))

trace3 = go.Scatter(x=df_europe[df_europe['country'] == 'Spain']['year'],
                    y=df_europe[df_europe['country'] == 'Spain']['lifeExp'],
                    mode='lines+markers',
                    zorder=2,
                    name='Spain',
                    marker=dict(size=15))

layout = go.Layout(title=dict(text='Life Expectancy in Europe Over Time'))

fig = go.Figure(data=[trace1, trace2, trace3], layout=layout)

fig.show()
```

Large Data Sets

Now in Plotly you can implement WebGL with `Scattergl()` in place of `Scatter()` for increased speed, improved interactivity, and the ability to plot even more data!

```
import plotly.graph_objects as go
import numpy as np

N = 100000
fig = go.Figure(data=go.Scattergl(
    x = np.random.randn(N),
    y = np.random.randn(N),
    mode='markers',
    marker=dict(
        color=np.random.randn(N),
        colorscale='Viridis',
        line_width=1
    )
))

fig.show()
```

```
import plotly.graph_objects as go
import numpy as np

N = 100000
r = np.random.uniform(0, 1, N)
theta = np.random.uniform(0, 2*np.pi, N)

fig = go.Figure(data=go.Scattergl(
    x = r * np.cos(theta), # non-uniform distribution
    y = r * np.sin(theta), # zoom to see more points at the center
    mode='markers',
    marker=dict(
        color=np.random.randn(N),
        colorscale='Viridis',
        line_width=1
    )
))

fig.show()
```

Reference

See function reference for `px.scatter()` or https://plotly.com/python/reference/scatter/ or https://plotly.com/python/reference/scattergl/ for more information and chart attribute options!

Line charts

Source: https://plotly.com/python/line-charts/

Line Plots with plotly.express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures. With `px.line`, each data point is represented as a vertex (which location is given by the `x` and `y` columns) of a **polyline mark** in 2D space.

For more examples of line plots, see the line and scatter notebook.

```
import plotly.express as px

df = px.data.gapminder().query("country=='Canada'")
fig = px.line(df, x="year", y="lifeExp", title='Life expectancy in Canada')
fig.show()
```

Line Plots with column encoding color

```
import plotly.express as px

df = px.data.gapminder().query("continent=='Oceania'")
fig = px.line(df, x="year", y="lifeExp", color='country')
fig.show()
```

Data Order in Line Charts

Plotly line charts are implemented as connected scatterplots (see below), meaning that the points are plotted and connected with lines **in the order they are provided, with no automatic reordering**.

This makes it possible to make charts like the one below, but also means that it may be required to explicitly sort data before passing it to Plotly to avoid lines moving "backwards" across the chart.

```
import plotly.express as px
import pandas as pd

df = pd.DataFrame(dict(
    x = [1, 3, 2, 4],
    y = [1, 2, 3, 4]
))
fig = px.line(df, x="x", y="y", title="Unsorted Input")
fig.show()

df = df.sort_values(by="x")
fig = px.line(df, x="x", y="y", title="Sorted Input")
fig.show()
```

Connected Scatterplots

In a connected scatterplot, two continuous variables are plotted against each other, with a line connecting them in some meaningful order, usually a time variable. In the plot below, we show the "trajectory" of a pair of countries through a space defined by GDP per Capita and Life Expectancy.

```
import plotly.express as px

df = px.data.gapminder().query("country in ['Canada', 'Botswana']")

fig = px.line(df, x="lifeExp", y="gdpPercap", color="country", text="year")
fig.update_traces(textposition="bottom right")
fig.show()
```

Line charts with markers

The `markers` argument can be set to `True` to show markers on lines.

```
import plotly.express as px
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x='year', y='lifeExp', color='country', markers=True)
fig.show()
```

The `symbol` argument can be used to map a data field to the marker symbol. A wide variety of symbols are available.

```
import plotly.express as px
df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.line(df, x='year', y='lifeExp', color='country', symbol="country")
fig.show()
```

Line plots on Date axes

Line plots can be made on using any type of cartesian axis, including linear, logarithmic, categorical or date axes. Line plots on date axes are often called time-series charts.

Plotly auto-sets the axis type to a date format when the corresponding data are either ISO-formatted date strings or if they're a date pandas column or datetime NumPy array.

```
import plotly.express as px

df = px.data.stocks()
fig = px.line(df, x='date', y="GOOG")
fig.show()
```

Sparklines with Plotly Express

Sparklines are scatter plots inside subplots, with gridlines, axis lines, and ticks removed.

```
import plotly.express as px
df = px.data.stocks(indexed=True)
fig = px.line(df, facet_row="company", facet_row_spacing=0.01, height=200, width=200)

# hide and lock down axes
```

```
fig.update_xaxes(visible=False, fixedrange=True)
fig.update_yaxes(visible=False, fixedrange=True)

# remove facet/subplot labels
fig.update_layout(annotations=[], overwrite=True)

# strip down the rest of the plot
fig.update_layout(
    showlegend=False,
    plot_bgcolor="white",
    margin=dict(t=10,l=10,b=10,r=10)
)

# disable the modebar for such a small plot
fig.show(config=dict(displayModeBar=False))
```

Line Plot with go.Scatter

If Plotly Express does not provide a good starting point, it is possible to use the more generic `go.Scatter` class from `plotly.graph_objects`. Whereas `plotly.express` has two functions `scatter` and `line`, `go.Scatter` can be used both for plotting points (makers) or lines, depending on the value of `mode`. The different options of `go.Scatter` are documented in its reference page.

Simple Line Plot

```
import plotly.graph_objects as go
import numpy as np

x = np.arange(10)

fig = go.Figure(data=go.Scatter(x=x, y=x**2))
fig.show()
```

Line Plot Modes

```
import plotly.graph_objects as go

# Create random data with numpy
import numpy as np
np.random.seed(1)

N = 100
random_x = np.linspace(0, 1, N)
random_y0 = np.random.randn(N) + 5
random_y1 = np.random.randn(N)
random_y2 = np.random.randn(N) - 5

# Create traces
fig = go.Figure()
fig.add_trace(go.Scatter(x=random_x, y=random_y0,
                    mode='lines',
                    name='lines'))
fig.add_trace(go.Scatter(x=random_x, y=random_y1,
                    mode='lines+markers',
```

```
                    name='lines+markers'))
fig.add_trace(go.Scatter(x=random_x, y=random_y2,
                    mode='markers', name='markers'))

fig.show()
```

Style Line Plots

This example styles the color and dash of the traces, adds trace names, modifies line width, and adds plot and axes titles.

```
import plotly.graph_objects as go

# Add data
month = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
         'August', 'September', 'October', 'November', 'December']
high_2000 = [32.5, 37.6, 49.9, 53.0, 69.1, 75.4, 76.5, 76.6, 70.7, 60.6, 45.1, 29.3]
low_2000 = [13.8, 22.3, 32.5, 37.2, 49.9, 56.1, 57.7, 58.3, 51.2, 42.8, 31.6, 15.9]
high_2007 = [36.5, 26.6, 43.6, 52.3, 71.5, 81.4, 80.5, 82.2, 76.0, 67.3, 46.1, 35.0]
low_2007 = [23.6, 14.0, 27.0, 36.8, 47.6, 57.7, 58.9, 61.2, 53.3, 48.5, 31.0, 23.6]
high_2014 = [28.8, 28.5, 37.0, 56.8, 69.7, 79.7, 78.5, 77.8, 74.1, 62.6, 45.3, 39.9]
low_2014 = [12.7, 14.3, 18.6, 35.5, 49.9, 58.0, 60.0, 58.6, 51.7, 45.2, 32.2, 29.1]

fig = go.Figure()
# Create and style traces
fig.add_trace(go.Scatter(x=month, y=high_2014, name='High 2014',
                         line=dict(color='firebrick', width=4)))
fig.add_trace(go.Scatter(x=month, y=low_2014, name = 'Low 2014',
                         line=dict(color='royalblue', width=4)))
fig.add_trace(go.Scatter(x=month, y=high_2007, name='High 2007',
                         line=dict(color='firebrick', width=4,
                              dash='dash') # dash options include 'dash', 'dot', and 'dashdot'
))
fig.add_trace(go.Scatter(x=month, y=low_2007, name='Low 2007',
                         line = dict(color='royalblue', width=4, dash='dash')))
fig.add_trace(go.Scatter(x=month, y=high_2000, name='High 2000',
                         line = dict(color='firebrick', width=4, dash='dot')))
fig.add_trace(go.Scatter(x=month, y=low_2000, name='Low 2000',
                         line=dict(color='royalblue', width=4, dash='dot')))

# Edit the layout
fig.update_layout(
        title=dict(
            text='Average High and Low Temperatures in New York'
        ),
        xaxis=dict(
            title=dict(
                text='Month'
            )
        ),
        yaxis=dict(
            title=dict(
                text='Temperature (degrees F)'
            )
        ),
)


fig.show()
```

Connect Data Gaps

connectgaps determines if missing values in the provided data are shown as a gap in the graph or not. In this tutorial, we showed how to take benefit of this feature and illustrate multiple areas on a tile map.

```
import plotly.graph_objects as go

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=x,
    y=[10, 20, None, 15, 10, 5, 15, None, 20, 10, 10, 15, 25, 20, 10],
    name = 'No Gaps', # Style name/legend entry with html tags
    connectgaps=True # override default to connect the gaps
))
fig.add_trace(go.Scatter(
    x=x,
    y=[5, 15, None, 10, 5, 0, 10, None, 15, 5, 5, 10, 20, 15, 5],
    name='Gaps',
))

fig.show()
```

Interpolation with Line Plots

```
import plotly.graph_objects as go
import numpy as np

x = np.array([1, 2, 3, 4, 5])
y = np.array([1, 3, 2, 3, 1])

fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y, name="linear",
                    line_shape='linear'))
fig.add_trace(go.Scatter(x=x, y=y + 5, name="spline",
                    text=["tweak line smoothnesswith 'smoothing' in line object"],
                    hoverinfo='text+name',
                    line_shape='spline'))
fig.add_trace(go.Scatter(x=x, y=y + 10, name="vhv",
                    line_shape='vhv'))
fig.add_trace(go.Scatter(x=x, y=y + 15, name="hvh",
                    line_shape='hvh'))
fig.add_trace(go.Scatter(x=x, y=y + 20, name="vh",
                    line_shape='vh'))
fig.add_trace(go.Scatter(x=x, y=y + 25, name="hv",
                    line_shape='hv'))

fig.update_traces(hoverinfo='text+name', mode='lines+markers')
fig.update_layout(legend=dict(y=0.5, traceorder='reversed', font_size=16))

fig.show()
```

Label Lines with Annotations

```
import plotly.graph_objects as go
import numpy as np

title = 'Main Source for News'
labels = ['Television', 'Newspaper', 'Internet', 'Radio']
colors = ['rgb(67,67,67)', 'rgb(115,115,115)', 'rgb(49,130,189)', 'rgb(189,189,189)']

mode_size = [8, 8, 12, 8]
line_size = [2, 2, 4, 2]

x_data = np.vstack((np.arange(2001, 2014),)*4)

y_data = np.array([
    [74, 82, 80, 74, 73, 72, 74, 70, 70, 66, 66, 69],
    [45, 42, 50, 46, 36, 36, 34, 35, 32, 31, 31, 28],
    [13, 14, 20, 24, 20, 24, 24, 40, 35, 41, 43, 50],
    [18, 21, 18, 21, 16, 14, 13, 18, 17, 16, 19, 23],
])

fig = go.Figure()

for i in range(0, 4):
    fig.add_trace(go.Scatter(x=x_data[i], y=y_data[i], mode='lines',
        name=labels[i],
        line=dict(color=colors[i], width=line_size[i]),
        connectgaps=True,
    ))

    # endpoints
    fig.add_trace(go.Scatter(
        x=[x_data[i][0], x_data[i][-1]],
        y=[y_data[i][0], y_data[i][-1]],
        mode='markers',
        marker=dict(color=colors[i], size=mode_size[i])
    ))

fig.update_layout(
    xaxis=dict(
        showline=True,
        showgrid=False,
        showticklabels=True,
        linecolor='rgb(204, 204, 204)',
        linewidth=2,
        ticks='outside',
        tickfont=dict(
            family='Arial',
            size=12,
            color='rgb(82, 82, 82)',
        ),
    ),
    yaxis=dict(
        showgrid=False,
        zeroline=False,
        showline=False,
        showticklabels=False,
    ),
    autosize=False,
    margin=dict(
```

```
        autoexpand=False,
        l=100,
        r=20,
        t=110,
    ),
    showlegend=False,
    plot_bgcolor='white'
)

annotations = []

# Adding labels
for y_trace, label, color in zip(y_data, labels, colors):
    # labeling the left_side of the plot
    annotations.append(dict(xref='paper', x=0.05, y=y_trace[0],
                                  xanchor='right', yanchor='middle',
                                  text=label + ' {}%'.format(y_trace[0]),
                                  font=dict(family='Arial',
                                            size=16),
                                  showarrow=False))
    # labeling the right_side of the plot
    annotations.append(dict(xref='paper', x=0.95, y=y_trace[11],
                                  xanchor='left', yanchor='middle',
                                  text='{}%'.format(y_trace[11]),
                                  font=dict(family='Arial',
                                            size=16),
                                  showarrow=False))
# Title
annotations.append(dict(xref='paper', yref='paper', x=0.0, y=1.05,
                              xanchor='left', yanchor='bottom',
                              text='Main Source for News',
                              font=dict(family='Arial',
                                        size=30,
                                        color='rgb(37,37,37)'),
                              showarrow=False))
# Source
annotations.append(dict(xref='paper', yref='paper', x=0.5, y=-0.1,
                              xanchor='center', yanchor='top',
                              text='Source: PewResearch Center & ' +
                                   'Storytelling with data',
                              font=dict(family='Arial',
                                        size=12,
                                        color='rgb(150,150,150)'),
                              showarrow=False))

fig.update_layout(annotations=annotations)

fig.show()
```

Filled Lines

```
import plotly.graph_objects as go
import numpy as np



x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
x_rev = x[::-1]
```

```
# Line 1
y1 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y1_upper = [2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
y1_lower = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
y1_lower = y1_lower[::-1]

# Line 2
y2 = [5, 2.5, 5, 7.5, 5, 2.5, 7.5, 4.5, 5.5, 5]
y2_upper = [5.5, 3, 5.5, 8, 6, 3, 8, 5, 6, 5.5]
y2_lower = [4.5, 2, 4.4, 7, 4, 2, 7, 4, 5, 4.75]
y2_lower = y2_lower[::-1]

# Line 3
y3 = [10, 8, 6, 4, 2, 0, 2, 4, 2, 0]
y3_upper = [11, 9, 7, 5, 3, 1, 3, 5, 3, 1]
y3_lower = [9, 7, 5, 3, 1, -.5, 1, 3, 1, -1]
y3_lower = y3_lower[::-1]


fig = go.Figure()

fig.add_trace(go.Scatter(
    x=x+x_rev,
    y=y1_upper+y1_lower,
    fill='toself',
    fillcolor='rgba(0,100,80,0.2)',
    line_color='rgba(255,255,255,0)',
    showlegend=False,
    name='Fair',
))
fig.add_trace(go.Scatter(
    x=x+x_rev,
    y=y2_upper+y2_lower,
    fill='toself',
    fillcolor='rgba(0,176,246,0.2)',
    line_color='rgba(255,255,255,0)',
    name='Premium',
    showlegend=False,
))
fig.add_trace(go.Scatter(
    x=x+x_rev,
    y=y3_upper+y3_lower,
    fill='toself',
    fillcolor='rgba(231,107,243,0.2)',
    line_color='rgba(255,255,255,0)',
    showlegend=False,
    name='Ideal',
))
fig.add_trace(go.Scatter(
    x=x, y=y1,
    line_color='rgb(0,100,80)',
    name='Fair',
))
fig.add_trace(go.Scatter(
    x=x, y=y2,
    line_color='rgb(0,176,246)',
    name='Premium',
))
```

```
fig.add_trace(go.Scatter(
    x=x, y=y3,
    line_color='rgb(231,107,243)',
    name='Ideal',
))

fig.update_traces(mode='lines')
fig.show()
```

Reference

See function reference for `px.line()` or https://plotly.com/python/reference/scatter/ for more information and chart attribute options!

Bar charts

Source: https://plotly.com/python/bar-charts/

Bar chart with Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures.

With `px.bar`, **each row of the DataFrame is represented as a rectangular mark**. To aggregate multiple data points into the same rectangular mark, please refer to the histogram documentation.

In the example below, there is only a single row of data per year, so a single bar is displayed per year.

```
import plotly.express as px
data_canada = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(data_canada, x='year', y='pop')
fig.show()
```

Bar charts with Long Format Data

Long-form data has one row per observation, and one column per variable. This is suitable for storing and displaying multivariate data i.e. with dimension greater than 2. This format is sometimes called "tidy".

To learn more about how to provide a specific form of column-oriented data to 2D-Cartesian Plotly Express functions such as `px.bar`, see the Plotly Express Wide-Form Support in Python documentation.

For detailed column-input-format documentation, see the Plotly Express Arguments documentation.

```
import plotly.express as px

long_df = px.data.medals_long()

fig = px.bar(long_df, x="nation", y="count", color="medal", title="Long-Form Input")
fig.show()
```

```
long_df
```

Bar charts with Wide Format Data

Wide-form data has one row per value of one of the first variable, and one column per value of the second variable. This is suitable for storing and displaying 2-dimensional data.

```
import plotly.express as px

wide_df = px.data.medals_wide()

fig = px.bar(wide_df, x="nation", y=["gold", "silver", "bronze"], title="Wide-Form Input")
fig.show()
```

```
wide_df
```

Colored Bars

The bar plot can be customized using keyword arguments, for example to use continuous color, as below, or discrete color, as above.

```
import plotly.express as px

df = px.data.gapminder().query("country == 'Canada'")
fig = px.bar(df, x='year', y='pop',
             hover_data=['lifeExp', 'gdpPercap'], color='lifeExp',
             labels={'pop':'population of Canada'}, height=400)
fig.show()
```

```
import plotly.express as px

df = px.data.gapminder().query("continent == 'Oceania'")
fig = px.bar(df, x='year', y='pop',
             hover_data=['lifeExp', 'gdpPercap'], color='country',
             labels={'pop':'population of Oceania'}, height=400)
fig.show()
```

Stacked vs Grouped Bars

When several rows share the same value of `x` (here Female or Male), the rectangles are stacked on top of one another by default.

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color='time')
fig.show()
```

The default stacked bar chart behavior can be changed to grouped (also known as clustered) using the `barmode` argument:

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill",
             color='smoker', barmode='group',
             height=400)
fig.show()
```

Aggregating into Single Colored Bars

As noted above `px.bar()` will result in **one rectangle drawn per row of input**. This can sometimes result in a striped look as in the examples above. To combine these rectangles into one per color per position, you can use `px.histogram()`, which has its own detailed documentation page.

> `px.bar` and `px.histogram` are designed to be nearly interchangeable in their call signatures, so as to be able to switch between aggregated and disaggregated bar representations.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="sex", y="total_bill",
             color='smoker', barmode='group',
             height=400)
fig.show()
```

`px.histogram()` will aggregate `y` values by summing them by default, but the `histfunc` argument can be used to set this to `avg` to create what is sometimes called a "barplot" which summarizes the central tendency of a dataset, rather than visually representing the totality of the dataset.

> Warning: when using `histfuncs` other than `"sum"` or `"count"` it can be very misleading to use a `barmode` other than `"group"`, as stacked bars in effect represent the sum of the bar heights, and summing averages is rarely a reasonable thing to visualize.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="sex", y="total_bill",
             color='smoker', barmode='group',
             histfunc='avg',
             height=400)
fig.show()
```

Bar Charts with Text

New in v5.5

You can add text to bars using the `text_auto` argument. Setting it to `True` will display the values on the bars, and setting it to a `d3-format` formatting string will control the output format.

```
import plotly.express as px
df = px.data.medals_long()

fig = px.bar(df, x="medal", y="count", color="nation", text_auto=True)
fig.show()
```

The `text` argument can be used to display arbitrary text on the bars:

```
import plotly.express as px
df = px.data.medals_long()

fig = px.bar(df, x="medal", y="count", color="nation", text="nation")
fig.show()
```

By default, Plotly will scale and rotate text labels to maximize the number of visible labels, which can result in a variety of text angles and sizes and positions in the same figure. The `textfont`, `textposition` and `textangle` trace attributes can be used to control these.

Here is an example of the default behavior:

```
import plotly.express as px

df = px.data.gapminder().query("continent == 'Europe' and year == 2007 and pop > 2.e6")
fig = px.bar(df, y='pop', x='country', text_auto='.2s',
            title="Default: various text sizes, positions and angles")
fig.show()
```

Here is the same data with less variation in text formatting. Note that `textfont_size` will set the *maximum* size. The `layout.uniformtext` attribute can be used to guarantee that all text labels are the same size. See the documentation on text and annotations for details.

The `cliponaxis` attribute is set to `False` in the example below to ensure that the outside text on the tallest bar is allowed to render outside of the plotting area.

```
import plotly.express as px

df = px.data.gapminder().query("continent == 'Europe' and year == 2007 and pop > 2.e6")
fig = px.bar(df, y='pop', x='country', text_auto='.2s',
            title="Controlled text sizes, positions and angles")
fig.update_traces(textfont_size=12, textangle=0, textposition="outside", cliponaxis=False)
fig.show()
```

Pattern Fills

New in v5.0

Bar charts afford the use of patterns (also known as hatching or texture) in addition to color:

```
import plotly.express as px
df = px.data.medals_long()

fig = px.bar(df, x="medal", y="count", color="nation",
             pattern_shape="nation", pattern_shape_sequence=[".", "x", "+"])
fig.show()
```

Facetted subplots

Use the keyword arguments `facet_row` (resp. `facet_col`) to create facetted subplots, where different rows (resp. columns) correspond to different values of the dataframe column specified in `facet_row`.

```
import plotly.express as px
df = px.data.tips()
fig = px.bar(df, x="sex", y="total_bill", color="smoker", barmode="group",
             facet_row="time", facet_col="day",
             category_orders={"day": ["Thur", "Fri", "Sat", "Sun"],
                              "time": ["Lunch", "Dinner"]})
fig.show()
```

Basic Bar Charts with plotly.graph_objects

If Plotly Express does not provide a good starting point, it is also possible to use the more generic `go.Bar` class from `plotly.graph_objects`.

```
import plotly.graph_objects as go
animals=['giraffes', 'orangutans', 'monkeys']

fig = go.Figure([go.Bar(x=animals, y=[20, 14, 23])])
fig.show()
```

Grouped Bar Chart

Customize the figure using `fig.update`.

```
import plotly.graph_objects as go
animals=['giraffes', 'orangutans', 'monkeys']

fig = go.Figure(data=[
    go.Bar(name='SF Zoo', x=animals, y=[20, 14, 23]),
    go.Bar(name='LA Zoo', x=animals, y=[12, 18, 29])
])
# Change the bar mode
fig.update_layout(barmode='group')
fig.show()
```

Stacked Bar Chart

```
import plotly.graph_objects as go
animals=['giraffes', 'orangutans', 'monkeys']

fig = go.Figure(data=[
    go.Bar(name='SF Zoo', x=animals, y=[20, 14, 23]),
    go.Bar(name='LA Zoo', x=animals, y=[12, 18, 29])
])
# Change the bar mode
fig.update_layout(barmode='stack')
fig.show()
```

Bar Chart with Relative Barmode

With “relative” barmode, the bars are stacked on top of one another, with negative values below the axis and positive values above.

```
import plotly.graph_objects as go
x = [1, 2, 3, 4]

fig = go.Figure()
fig.add_trace(go.Bar(x=x, y=[1, 4, 9, 16]))
fig.add_trace(go.Bar(x=x, y=[6, -8, -4.5, 8]))
fig.add_trace(go.Bar(x=x, y=[-15, -3, 4.5, -8]))
fig.add_trace(go.Bar(x=x, y=[-1, 3, -3, -4]))

fig.update_layout(barmode='relative', title_text='Relative Barmode')
fig.show()
```

Grouped Stacked Bar Chart

Supported in Plotly.py 6.0.0 and later

Use the `offsetgroup` property with `barmode="stacked"` or `barmode="relative"` to create grouped stacked bar charts. Bars that have the same `offsetgroup` will share the same position on the axis. Bars with no `offsetgroup` set will also share the same position on the axis. In the following example, for each quarter, the value for cities that belong to the same `offsetgroup` are stacked together.

```
import plotly.graph_objects as go
```

```
data = [
    go.Bar(
        x=['Q1', 'Q2', 'Q3', 'Q4'],
        y=[150, 200, 250, 300],
        name='New York',
        offsetgroup="USA"
    ),
    go.Bar(
        x=['Q1', 'Q2', 'Q3', 'Q4'],
        y=[180, 220, 270, 320],
        name='Boston',
        offsetgroup="USA"
    ),
    go.Bar(
        x=['Q1', 'Q2', 'Q3', 'Q4'],
        y=[130, 170, 210, 260],
        name='Montreal',
        offsetgroup="Canada"
    ),
    go.Bar(
        x=['Q1', 'Q2', 'Q3', 'Q4'],
        y=[160, 210, 260, 310],
        name='Toronto',
        offsetgroup="Canada"
    )
]

layout = go.Layout(
    title={
        'text': 'Quarterly Sales by City, Grouped by Country'
    },
    xaxis={
        'title': {
            'text': 'Quarter'
        }
    },
    yaxis={
        'title': {
            'text': 'Sales'
        }
    },
    barmode='stack'
)

fig = go.Figure(data=data, layout=layout)

fig.show()
```

Stacked Bar Chart From Aggregating a DataFrame

Stacked bar charts are a powerful way to present results summarizing categories generated using the Pandas aggregate commands. `pandas.DataFrame.agg` produces a wide data set format incompatible with `px.bar`. Transposing and updating the indexes to achieve `px.bar` compatibility is a somewhat involved option. Here is one straightforward alternative, which presents the aggregated data as a stacked bar using plotly.graph_objects.

```
from plotly import graph_objects as go
import pandas as pd

# Get one year of gapminder data
url = 'https://raw.githubusercontent.com/plotly/datasets/master/gapminderDataFiveYear.csv'
df = pd.read_csv(url)
df = df[df['year']==2007]
df["gdp"]=df["pop"]*df['gdpPercap']


# Build the summary of interest
df_summarized = df.groupby("continent", observed=True).agg("sum").reset_index()

df_summarized["percent of world population"]=100*df_summarized["pop"]/df_summarized["pop"].sum()
df_summarized["percent of world GDP"]=100*df_summarized["gdp"]/df_summarized["gdp"].sum()


df = df_summarized[["continent",
"percent of world population",
"percent of world GDP",
]]

# We now have a wide data frame, but it's in the opposite orientation from the one that px is designed to
 ↪  deal with.
# Transposing it and rebuilding the indexes is an option, but iterating through the DF using graph objects
 ↪  is more succinct.

fig=go.Figure()
for category in df_summarized["continent"].values:
    fig.add_trace(go.Bar(
            x=df.columns[1:],
            # We need to get a pandas series that contains just the values to graph;
            # We do so by selecting the right row, selecting the right columns
            # and then transposing and using iloc to convert to a series
            # Here, we assume that the bar element category variable is in column 0
            y=list(df.loc[df["continent"]==category][list(df.columns[1:])].transpose().iloc[:,0]),
            name=str(category)


        )
)
fig.update_layout(barmode="stack")

fig.show()
```

Bar Chart with Hover Text

```
import plotly.graph_objects as go

x = ['Product A', 'Product B', 'Product C']
y = [20, 14, 23]

# Use the hovertext kw argument for hover text
fig = go.Figure(data=[go.Bar(x=x, y=y,
            hovertext=['27% market share', '24% market share', '19% market share'])])
# Customize aspect
fig.update_traces(marker_color='rgb(158,202,225)', marker_line_color='rgb(8,48,107)',
```

```
                  marker_line_width=1.5, opacity=0.6)
fig.update_layout(title_text='January 2013 Sales Report')
fig.show()
```

Bar Chart with Direct Labels

```
import plotly.graph_objects as go

x = ['Product A', 'Product B', 'Product C']
y = [20, 14, 23]

# Use textposition='auto' for direct text
fig = go.Figure(data=[go.Bar(
            x=x, y=y,
            text=y,
            textposition='auto',
        )])

fig.show()
```

Controlling text fontsize with uniformtext

If you want all the text labels to have the same size, you can use the `uniformtext` layout parameter. The `minsize` attribute sets the font size, and the `mode` attribute sets what happens for labels which cannot fit with the desired fontsize: either `hide` them or `show` them with overflow. In the example below we also force the text to be outside of bars with `textposition`.

```
import plotly.express as px

df = px.data.gapminder().query("continent == 'Europe' and year == 2007 and pop > 2.e6")
fig = px.bar(df, y='pop', x='country', text='pop')
fig.update_traces(texttemplate='%{text:.2s}', textposition='outside')
fig.update_layout(uniformtext_minsize=8, uniformtext_mode='hide')
fig.show()
```

Rotated Bar Chart Labels

```
import plotly.graph_objects as go

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
          'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

fig = go.Figure()
fig.add_trace(go.Bar(
    x=months,
    y=[20, 14, 25, 16, 18, 22, 19, 15, 12, 16, 14, 17],
    name='Primary Product',
    marker_color='indianred'
))
fig.add_trace(go.Bar(
    x=months,
    y=[19, 14, 22, 14, 16, 19, 15, 14, 10, 12, 12, 16],
    name='Secondary Product',
```

```
    marker_color='lightsalmon'
))

# Here we modify the tickangle of the xaxis, resulting in rotated labels.
fig.update_layout(barmode='group', xaxis_tickangle=-45)
fig.show()
```

Customizing Individual Bar Colors

```
import plotly.graph_objects as go

colors = ['lightslategray',] * 5
colors[1] = 'crimson'

fig = go.Figure(data=[go.Bar(
    x=['Feature A', 'Feature B', 'Feature C',
       'Feature D', 'Feature E'],
    y=[20, 14, 23, 25, 22],
    marker_color=colors # marker color can be a single color value or an iterable
)])
fig.update_layout(title_text='Least Used Feature')
```

Customizing Individual Bar Widths

```
import plotly.graph_objects as go

fig = go.Figure(data=[go.Bar(
    x=[1, 2, 3, 5.5, 10],
    y=[10, 8, 6, 4, 2],
    width=[0.8, 0.8, 0.8, 3.5, 4] # customize width here
)])

fig.show()
```

Bar charts with custom widths can be used to make mekko charts (also known as marimekko charts, mosaic plots, or variwide charts).

```
import plotly.graph_objects as go
import numpy as np

labels = ["apples","oranges","pears","bananas"]
widths = np.array([10,20,20,50])

data = {
    "South": [50,80,60,70],
    "North": [50,20,40,30]
}

fig = go.Figure()
for key in data:
    fig.add_trace(go.Bar(
        name=key,
        y=data[key],
        x=np.cumsum(widths)-widths,
        width=widths,
```

```
        offset=0,
        customdata=np.transpose([labels, widths*data[key]]),
        texttemplate="%{y} x %{width} =%{customdata[1]}",
        textposition="inside",
        textangle=0,
        textfont_color="white",
        hovertemplate="".join([
            "label: %{customdata[0]}",
            "width: %{width}",
            "height: %{y}",
            "area: %{customdata[1]}",
        ])
    ))

fig.update_xaxes(
    tickvals=np.cumsum(widths)-widths/2,
    ticktext= ["%s%d" % (l, w) for l, w in zip(labels, widths)]
)

fig.update_xaxes(range=[0,100])
fig.update_yaxes(range=[0,100])

fig.update_layout(
    title_text="Marimekko Chart",
    barmode="stack",
    uniformtext=dict(mode="hide", minsize=10),
)
```

Using a scatterplot to wrap long bars into multiple columns

This bar-style pictogram allows readers to focus on the relative sizes of smaller entities by wrapping the bar for largest entries into multiple columns. You could make it even more of a pictogram by using fontawesome to replace the square markers we use below with icons like mortar boards for students.

```
import plotly.graph_objects as go
import pandas as pd
def pictogram_bar(data, title, icon_size, max_icons_per_column=10, units_per_icon=1, unit_description="",
↪  inter_group_spacing=.8,icon_vertical_spacing=0.005):

    fig = go.Figure()
    x_start = 1
    tick_locations = []
    #loop through each group and create a trace with its icons
    for i, (category, value) in enumerate(data.items()):
        # compute the number of icons to use to represent this category.  Depending on your use case, you
        ↪  might replace round with floor or ceiling.
        icon_count = round(value / units_per_icon)
        # compute the number of columns in which to arrange the icons for this category
        # using a double negative sign to convert a floor(division) operation into a ceiling(division)
        ↪  operation
        num_columns = -(-icon_count // max_icons_per_column)

        #create and populate lists of icon coordinates
        x_coordinates, y_coordinates = [], []
        for col in range(num_columns):
            # the number of icons in this column is the lesser of the column height or
```

```
            # the number of icons remaining to place
            column_icons = min(max_icons_per_column, icon_count - col * max_icons_per_column)

            # Create a one item list containing the x-coordinate of this column.
            # Then add column_icons copies of that coordinate to the list of icon x coordinates using list
             ↪ multiplication.
            # Normalizing the width of each within-category column to 1 simplifies the code.
            # We can adjust the visible space between columns by adjusting the total width below.
            x_coordinates.extend([x_start + col] * column_icons)
            # Create a list of sequentially increasing y-coordinates for icons.
            y_coordinates.extend([y + icon_vertical_spacing * y for y in range(1, column_icons + 1)])
        # Add scatter plot for the category
        fig.add_trace(go.Scatter(
            x=x_coordinates,
            y=y_coordinates,
            mode='markers',
            marker=dict(size=icon_size, symbol="square", color= i),
            name=category,
            # Suppress the x and y coordinates in the hover text, since they are irrelevant implementation
             ↪ details.
            hoverinfo="text",
            text=[f"{category}: {value}" for _ in range(len(x_coordinates))]
        ))

        # Add an annotation above the center of each category showing its value
        fig.add_trace(go.Scatter(
            x=[x_start + (num_columns - 1) / 2],  # Compute the location of the center
            y=[max_icons_per_column* (1+icon_vertical_spacing) + 1.15],
            mode="text",
            text=[f"{value}"],
            textfont=dict(size=14, color="black"),
            showlegend=False
        ))
        # Track locations where we will put the text labeling each category
        tick_locations.append(x_start + (num_columns - 1) / 2)
        #compute the left edge of the next category
        x_start += num_columns + inter_group_spacing

    fig.update_layout(
        title=title,
        xaxis=dict(
            tickvals=tick_locations,
            # Label each category
            ticktext=list(data.keys()),
            tickangle=-45,
            showgrid=False,
            title="Categories"
        ),
        yaxis=dict(
            title=f"Each icon represents {units_per_icon:,g} {unit_description}",
            # The y-axis goes above the top icon to make room for the annotations.
            # We set tick values so the axis labeling does not go above the top icon.
            # If you choose a value of max_icons_per_column that is not a multiple of 5, consider changing
             ↪ this.
            tickvals=list(range(0,max_icons_per_column+1,5)),
            showgrid=False,
            zeroline=False,
        ),
        # We have already got all the labeling we need so we suppress the legend.
```

```
        showlegend=False,
        height=700,
        # The x-coordinates scale to fill available space, so adjusting the width of the image is a good
        ↪  way to adjust spacing between columns.
        width=(len(data) * 150 + 50)
    )
    fig.show()

df = pd.DataFrame({
    'School': ["Haverford College", "University of Mary Washington", "Brown University", "Arizona State
    ↪  University"],
    'Enrollment': [1421, 3611, 7226, 65174]
})

pictogram_bar(
    data={row['School']: row['Enrollment'] for _, row in df.iterrows()},
    title="Undergraduate Enrollment at Participating Schools",
    units_per_icon=1000,
    unit_description = "students",
    icon_size=27,
    icon_vertical_spacing=0.05
)
```

Customizing Individual Bar Base

```
import plotly.graph_objects as go

years = ['2016','2017','2018']

fig = go.Figure()
fig.add_trace(go.Bar(x=years, y=[500, 600, 700],
                base=[-500,-600,-700],
                marker_color='crimson',
                name='expenses'))
fig.add_trace(go.Bar(x=years, y=[300, 400, 700],
                base=0,
                marker_color='lightslategrey',
                name='revenue'
                ))

fig.show()
```

Rounded Bars

New in 5.19

You can round the corners on all bar traces in a figure by setting `barcornerradius` on the figure's layout. `barcornerradius` can be a number of pixels or a percentage of the bar width (using a string ending in %, for example "20%").

In this example, we set all bars to have a radius of 15 pixels.

```
import plotly.graph_objects as go
from plotly import data

df = data.medals_wide()
```

```
fig = go.Figure(
    data=[
        go.Bar(x=df.nation, y=df.gold, name="Gold"),
        go.Bar(x=df.nation, y=df.silver, name="Silver"),
        go.Bar(x=df.nation, y=df.bronze, name="Bronze"),
    ],
    layout=dict(
        barcornerradius=15,
    ),
)

fig.show()
```

When you don't want all bar traces in a figure to have the same rounded corners, you can instead configure rounded corners on each trace using `marker.cornerradius`. In this example, which uses subplots, the first trace has a corner radius of 30 pixels, the second trace has a bar corner radius of 30% of the bar width, and the third trace has no rounded corners set.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from plotly import data

df = data.medals_wide()

fig = make_subplots(rows=1, cols=3, shared_yaxes=True)

fig.add_trace(
    go.Bar(x=df.nation, y=df.gold, name="Gold", marker=dict(cornerradius=30)), 1, 1
)
fig.add_trace(
    go.Bar(x=df.nation, y=df.silver, name="Silver", marker=dict(cornerradius="30%")),
    1,
    2,
)

fig.add_trace(
    go.Bar(x=df.nation, y=df.bronze, name="Bronze"),
    1,
    3,
)


fig.show()
```

Colored and Styled Bar Chart

In this example several parameters of the layout are customized, hence it is convenient to use directly the `go.Layout(...)` constructor instead of calling `fig.update`.

```
import plotly.graph_objects as go

years = [1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003,
         2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012]

fig = go.Figure()
fig.add_trace(go.Bar(x=years,
                y=[219, 146, 112, 127, 124, 180, 236, 207, 236, 263,
```

```
                   350, 430, 474, 526, 488, 537, 500, 439],
                name='Rest of world',
                marker_color='rgb(55, 83, 109)'
                ))
fig.add_trace(go.Bar(x=years,
                y=[16, 13, 10, 11, 28, 37, 43, 55, 56, 88, 105, 156, 270,
                   299, 340, 403, 549, 499],
                name='China',
                marker_color='rgb(26, 118, 255)'
                ))

fig.update_layout(
    title=dict(text='US Export of Plastic Scrap'),
    xaxis_tickfont_size=14,
    yaxis=dict(
        title=dict(
            text="USD (millions)",
            font=dict(
                size=16
            )
        ),
    ),
    legend=dict(
        x=0,
        y=1.0,
        bgcolor='rgba(255, 255, 255, 0)',
        bordercolor='rgba(255, 255, 255, 0)'
    ),
    barmode='group',
    bargap=0.15, # gap between bars of adjacent location coordinates.
    bargroupgap=0.1 # gap between bars of the same location coordinate.
)
fig.show()
```

Bar Chart with Sorted or Ordered Categories

Set `categoryorder` to `"category ascending"` or `"category descending"` for the alphanumerical order of the category names or `"total ascending"` or `"total descending"` for numerical order of values. categoryorder for more information. Note that sorting the bars by a particular trace isn't possible right now - it's only possible to sort by the total values. Of course, you can always sort your data *before* plotting it if you need more customization.

This example orders the bar chart alphabetically with `categoryorder: 'category ascending'`

```
import plotly.graph_objects as go

x=['b', 'a', 'c', 'd']
fig = go.Figure(go.Bar(x=x, y=[2,5,1,9], name='Montreal'))
fig.add_trace(go.Bar(x=x, y=[1, 4, 9, 16], name='Ottawa'))
fig.add_trace(go.Bar(x=x, y=[6, 8, 4.5, 8], name='Toronto'))

fig.update_layout(barmode='stack', xaxis={'categoryorder':'category ascending'})
fig.show()
```

This example shows how to customise sort ordering by defining `categoryorder` to "array" to derive the ordering from the attribute `categoryarray`.

```
import plotly.graph_objects as go

x=['b', 'a', 'c', 'd']
fig = go.Figure(go.Bar(x=x, y=[2,5,1,9], name='Montreal'))
fig.add_trace(go.Bar(x=x, y=[1, 4, 9, 16], name='Ottawa'))
fig.add_trace(go.Bar(x=x, y=[6, 8, 4.5, 8], name='Toronto'))

fig.update_layout(barmode='stack', xaxis={'categoryorder':'array', 'categoryarray':['d','a','c','b']})
fig.show()
```

This example orders the bar chart by descending value with `categoryorder: 'total descending'`

```
import plotly.graph_objects as go

x=['b', 'a', 'c', 'd']
fig = go.Figure(go.Bar(x=x, y=[2,5,1,9], name='Montreal'))
fig.add_trace(go.Bar(x=x, y=[1, 4, 9, 16], name='Ottawa'))
fig.add_trace(go.Bar(x=x, y=[6, 8, 4.5, 8], name='Toronto'))

fig.update_layout(barmode='stack', xaxis={'categoryorder':'total descending'})
fig.show()
```

Horizontal Bar Charts

See examples of horizontal bar charts here.

Bar Charts With Multicategory Axis Type

If your traces have arrays for `x` or `y`, then the axis type is automatically inferred to be `multicategory`.

```
import plotly.graph_objects as go
x = [
    ["BB+", "BB+", "BB+", "BB", "BB", "BB"],
    [16, 17, 18, 16, 17, 18,]
]
fig = go.Figure()
fig.add_bar(x=x,y=[1,2,3,4,5,6])
fig.add_bar(x=x,y=[6,5,4,3,2,1])
fig.update_layout(barmode="relative")
fig.show()
```

Reference

See function reference for `px.bar()` or https://plotly.com/python/reference/bar/ for more information and chart attribute options!

Histograms

Source: https://plotly.com/python/histograms/

<!- #region -> In statistics, a histogram is representation of the distribution of numerical data, where the data are binned and the count for each bin is represented. More generally, in Plotly a histogram is an aggregated bar chart, with several possible aggregation functions (e.g. sum, average, count...) which can be used to visualize data on categorical and date axes as well as linear axes.

Alternatives to histogram plots for visualizing distributions include violin plots, box plots, ECDF plots and strip charts.

> If you're looking instead for bar charts, i.e. representing *raw, unaggregated* data with rectangular bar, go to the Bar Chart tutorial.

Histograms with Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures. <!- #endregion ->

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill")
fig.show()
```

```
import plotly.express as px
df = px.data.tips()
# Here we use a column with categorical data
fig = px.histogram(df, x="day")
fig.show()
```

Choosing the number of bins

By default, the number of bins is chosen so that this number is comparable to the typical number of samples in a bin. This number can be customized, as well as the range of values.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", nbins=20)
fig.show()
```

Histograms on Date Data

Plotly histograms will automatically bin date data in addition to numerical data:

```
import plotly.express as px

df = px.data.stocks()
fig = px.histogram(df, x="date")
fig.update_layout(bargap=0.2)
fig.show()
```

Histograms on Categorical Data

Plotly histograms will automatically bin numerical or date data but can also be used on raw categorical data, as in the following example, where the X-axis value is the categorical "day" variable:

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="day", category_orders=dict(day=["Thur", "Fri", "Sat", "Sun"]))
fig.show()
```

Accessing the counts (y-axis) values

JavaScript calculates the y-axis (count) values on the fly in the browser, so it's not accessible in the `fig`. You can manually calculate it using `np.histogram`.

```
import plotly.express as px
import numpy as np

df = px.data.tips()
# create the bins
counts, bins = np.histogram(df.total_bill, bins=range(0, 60, 5))
bins = 0.5 * (bins[:-1] + bins[1:])

fig = px.bar(x=bins, y=counts, labels={'x':'total_bill', 'y':'count'})
fig.show()
```

Type of normalization

The default mode is to represent the count of samples in each bin. With the `histnorm` argument, it is also possible to represent the percentage or fraction of samples in each bin (`histnorm='percent'` or `probability`), or a density histogram (the sum of all bar areas equals the total number of sample points, `density`), or a probability density histogram (the sum of all bar areas equals 1, `probability density`).

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", histnorm='probability density')
fig.show()
```

Aspect of the histogram plot

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill",
                   title='Histogram of bills',
                   labels={'total_bill':'total bill'}, # can specify one label per df column
                   opacity=0.8,
                   log_y=True, # represent bars with log scale
                   color_discrete_sequence=['indianred'] # color of histogram bars
                   )
fig.show()
```

Several histograms for the different values of one column

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", color="sex")
fig.show()
```

Aggregating with other functions than `count`

For each bin of `x`, one can compute a function of data using `histfunc`. The argument of `histfunc` is the dataframe column given as the `y` argument. Below the plot shows that the average tip increases with the total bill.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", y="tip", histfunc='avg')
fig.show()
```

The default `histfunc` is `sum` if `y` is given, and works with categorical as well as binned numeric data on the `x` axis:

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="day", y="total_bill", category_orders=dict(day=["Thur", "Fri", "Sat", "Sun"]))
fig.show()
```

New in v5.0

Histograms afford the use of patterns (also known as hatching or texture) in addition to color:

```
import plotly.express as px

df = px.data.tips()
fig = px.histogram(df, x="sex", y="total_bill", color="sex", pattern_shape="smoker")
fig.show()
```

Visualizing the distribution

With the `marginal` keyword, a marginal is drawn alongside the histogram, visualizing the distribution. See the distplot page for more examples of combined statistical representations.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", color="sex", marginal="rug", # can be `box`, `violin`
                         hover_data=df.columns)
fig.show()
```

Adding text labels

New in v5.5

You can add text to histogram bars using the `text_auto` argument. Setting it to `True` will display the values on the bars, and setting it to a `d3-format` formatting string will control the output format.

```
import plotly.express as px
df = px.data.tips()
fig = px.histogram(df, x="total_bill", y="tip", histfunc="avg", nbins=8, text_auto=True)
fig.show()
```

Histograms with go.Histogram

If Plotly Express does not provide a good starting point, it is also possible to use the more generic `go.Histogram` class from `plotly.graph_objects`. All of the available histogram options are described in the histogram section of the reference page: https://plotly.com/python/reference#histogram.

Basic Histogram

```
import plotly.graph_objects as go

import numpy as np
np.random.seed(1)

x = np.random.randn(500)

fig = go.Figure(data=[go.Histogram(x=x)])
fig.show()
```

Normalized Histogram

```
import plotly.graph_objects as go

import numpy as np

x = np.random.randn(500)
fig = go.Figure(data=[go.Histogram(x=x, histnorm='probability')])

fig.show()
```

Horizontal Histogram

```
import plotly.graph_objects as go

import numpy as np

y = np.random.randn(500)
# Use `y` argument instead of `x` for horizontal histogram

fig = go.Figure(data=[go.Histogram(y=y)])
fig.show()
```

Overlaid Histogram

```
import plotly.graph_objects as go

import numpy as np

x0 = np.random.randn(500)
# Add 1 to shift the mean of the Gaussian distribution
x1 = np.random.randn(500) + 1

fig = go.Figure()
fig.add_trace(go.Histogram(x=x0))
```

```
fig.add_trace(go.Histogram(x=x1))

# Overlay both histograms
fig.update_layout(barmode='overlay')
# Reduce opacity to see both histograms
fig.update_traces(opacity=0.75)
fig.show()
```

Stacked Histograms

```
import plotly.graph_objects as go

import numpy as np

x0 = np.random.randn(2000)
x1 = np.random.randn(2000) + 1

fig = go.Figure()
fig.add_trace(go.Histogram(x=x0))
fig.add_trace(go.Histogram(x=x1))

# The two histograms are drawn on top of another
fig.update_layout(barmode='stack')
fig.show()
```

Styled Histogram

```
import plotly.graph_objects as go

import numpy as np
x0 = np.random.randn(500)
x1 = np.random.randn(500) + 1

fig = go.Figure()
fig.add_trace(go.Histogram(
    x=x0,
    histnorm='percent',
    name='control', # name used in legend and hover labels
    xbins=dict( # bins used for histogram
        start=-4.0,
        end=3.0,
        size=0.5
    ),
    marker_color='#EB89B5',
    opacity=0.75
))
fig.add_trace(go.Histogram(
    x=x1,
    histnorm='percent',
    name='experimental',
    xbins=dict(
        start=-3.0,
        end=4,
        size=0.5
    ),
    marker_color='#330C73',
```

```
    opacity=0.75
))

fig.update_layout(
    title_text='Sampled Results', # title of plot
    xaxis_title_text='Value', # xaxis label
    yaxis_title_text='Count', # yaxis label
    bargap=0.2, # gap between bars of adjacent location coordinates
    bargroupgap=0.1 # gap between bars of the same location coordinates
)

fig.show()
```

Histogram Bar Text

You can add text to histogram bars using the `texttemplate` argument. In this example we add the x-axis values as text following the format `%{variable}`. We also adjust the size of the text using `textfont_size`.

```
import plotly.graph_objects as go

numbers = ["5", "10", "3", "10", "5", "8", "5", "5"]

fig = go.Figure()
fig.add_trace(go.Histogram(x=numbers, name="count", texttemplate="%{x}", textfont_size=20))

fig.show()
```

Cumulative Histogram

```
import plotly.graph_objects as go

import numpy as np

x = np.random.randn(500)
fig = go.Figure(data=[go.Histogram(x=x, cumulative_enabled=True)])

fig.show()
```

Specify Aggregation Function

```
import plotly.graph_objects as go

x = ["Apples","Apples","Apples","Oranges", "Bananas"]
y = ["5","10","3","10","5"]

fig = go.Figure()
fig.add_trace(go.Histogram(histfunc="count", y=y, x=x, name="count"))
fig.add_trace(go.Histogram(histfunc="sum", y=y, x=x, name="sum"))

fig.show()
```

Custom Binning

For custom binning along x-axis, use the attribute `nbinsx`. Please note that the autobin algorithm will choose a 'nice' round bin size that may result in somewhat fewer than `nbinsx` total bins. Alternatively, you can set the exact values for `xbins` along with `autobinx = False`.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

x = ['1970-01-01', '1970-01-01', '1970-02-01', '1970-04-01', '1970-01-02',
     '1972-01-31', '1970-02-13', '1971-04-19']

fig = make_subplots(rows=3, cols=2)

trace0 = go.Histogram(x=x, nbinsx=4)
trace1 = go.Histogram(x=x, nbinsx = 8)
trace2 = go.Histogram(x=x, nbinsx=10)
trace3 = go.Histogram(x=x,
                      xbins=dict(
                      start='1969-11-15',
                      end='1972-03-31',
                      size='M18'), # M18 stands for 18 months
                      autobinx=False
                     )
trace4 = go.Histogram(x=x,
                      xbins=dict(
                      start='1969-11-15',
                      end='1972-03-31',
                      size='M4'), # 4 months bin size
                      autobinx=False
                      )
trace5 = go.Histogram(x=x,
                      xbins=dict(
                      start='1969-11-15',
                      end='1972-03-31',
                      size= 'M2'), # 2 months
                      autobinx = False
                      )

fig.add_trace(trace0, 1, 1)
fig.add_trace(trace1, 1, 2)
fig.add_trace(trace2, 2, 1)
fig.add_trace(trace3, 2, 2)
fig.add_trace(trace4, 3, 1)
fig.add_trace(trace5, 3, 2)

fig.show()
```

See also: Bar Charts

If you want to display information about the individual items within each histogram bar, then create a stacked bar chart with hover information as shown below. Note that this is not technically the histogram chart type, but it will have a similar effect as shown below by comparing the output of `px.histogram` and `px.bar`. For more information, see the tutorial on bar charts.

```
import plotly.express as px
df = px.data.tips()
fig1 = px.bar(df, x='day', y='tip', height=300,
              title='Stacked Bar Chart - Hover on individual items')
```

```
fig2 = px.histogram(df, x='day', y='tip', histfunc='sum', height=300,
                    title='Histogram Chart')
fig1.show()
fig2.show()
```

Share bins between histograms

In this example both histograms have a compatible bin settings using bingroup attribute. Note that traces on the same subplot, and with the same `barmode` ("stack", "relative", "group") are forced into the same `bingroup`, however traces with `barmode = "overlay"` and on different axes (of the same axis type) can have compatible bin settings. Histogram and histogram2d trace can share the same `bingroup`.

```
import plotly.graph_objects as go
import numpy as np

fig = go.Figure(go.Histogram(
    x=np.random.randint(7, size=100),
    bingroup=1))

fig.add_trace(go.Histogram(
    x=np.random.randint(7, size=20),
    bingroup=1))

fig.update_layout(
    barmode="overlay",
    bargap=0.1)

fig.show()
```

Sort Histogram by Category Order

Histogram bars can also be sorted based on the ordering logic of the categorical values using the categoryorder attribute of the x-axis. Sorting of histogram bars using `categoryorder` also works with multiple traces on the same x-axis. In the following examples, the histogram bars are sorted based on the total numerical values.

```
import plotly.express as px

df = px.data.tips()
fig = px.histogram(df, x="day").update_xaxes(categoryorder='total ascending')
fig.show()
```

```
import plotly.express as px

df = px.data.tips()
fig = px.histogram(df, x="day", color="smoker").update_xaxes(categoryorder='total descending')
fig.show()
```

Reference

See function reference for `px.histogram()` or https://plotly.com/python/reference/histogram/ for more information and chart attribute options!

Box plots

Source: https://plotly.com/python/box-plots/

<!– #region –> A box plot is a statistical representation of the distribution of a variable through its quartiles. The ends of the box represent the lower and upper quartiles, while the median (second quartile) is marked by a line inside the box. For other statistical representations of numerical data, see other statistical charts.

Alternatives to box plots for visualizing distributions include histograms, violin plots, ECDF plots and strip charts.

Box Plot with `plotly.express`

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures.

In a box plot created by `px.box`, the distribution of the column given as `y` argument is represented. <!– #endregion –>

```
import plotly.express as px
df = px.data.tips()
fig = px.box(df, y="total_bill")
fig.show()
```

If a column name is given as `x` argument, a box plot is drawn for each value of `x`.

```
import plotly.express as px
df = px.data.tips()
fig = px.box(df, x="time", y="total_bill")
fig.show()
```

Display the underlying data

With the `points` argument, display underlying data points with either all points (`all`), outliers only (`outliers`, default), or none of them (`False`).

```
import plotly.express as px
df = px.data.tips()
fig = px.box(df, x="time", y="total_bill", points="all")
fig.show()
```

Choosing The Algorithm For Computing Quartiles

By default, quartiles for box plots are computed using the `linear` method (for more about linear interpolation, see #10 listed on http://jse.amstat.org/v14n3/langford.html and https://en.wikipedia.org/wiki/Quartile for more details).

However, you can also choose to use an `exclusive` or an `inclusive` algorithm to compute quartiles.

The *exclusive* algorithm uses the median to divide the ordered dataset into two halves. If the sample is odd, it does not include the median in either half. Q1 is then the median of the lower half and Q3 is the median of the upper half.

The *inclusive* algorithm also uses the median to divide the ordered dataset into two halves, but if the sample is odd, it includes the median in both halves. Q1 is then the median of the lower half and Q3 the median of the upper half.

```
import plotly.express as px

df = px.data.tips()

fig = px.box(df, x="day", y="total_bill", color="smoker")
fig.update_traces(quartilemethod="exclusive") # or "inclusive", or "linear" by default
fig.show()
```

Difference Between Quartile Algorithms

It can sometimes be difficult to see the difference between the linear, inclusive, and exclusive algorithms for computing quartiles. In the following example, the same dataset is visualized using each of the three different quartile computation algorithms.

```
import plotly.express as px
import pandas as pd

data = [1,2,3,4,5,6,7,8,9]
df = pd.DataFrame(dict(
    linear=data,
    inclusive=data,
    exclusive=data
)).melt(var_name="quartilemethod")


fig = px.box(df, y="value", facet_col="quartilemethod", color="quartilemethod",
             boxmode="overlay", points='all')

fig.update_traces(quartilemethod="linear", jitter=0, col=1)
fig.update_traces(quartilemethod="inclusive", jitter=0, col=2)
fig.update_traces(quartilemethod="exclusive", jitter=0, col=3)

fig.show()
```

Styled box plot

For the interpretation of the notches, see https://en.wikipedia.org/wiki/Box_plot#Variations.

```
import plotly.express as px
df = px.data.tips()
fig = px.box(df, x="time", y="total_bill", color="smoker",
             notched=True, # used notched shape
             title="Box plot of total bill",
             hover_data=["day"] # add day column to hover data
            )
fig.show()
```

Box plot with go.Box

If Plotly Express does not provide a good starting point, it is also possible to use the more generic `go.Box` class from `plotly.graph_objects`. All available options for `go.Box` are described in the reference page https://plotly.com/python/reference/box/.

Basic Box Plot

```
import plotly.graph_objects as go
import numpy as np
np.random.seed(1)

y0 = np.random.randn(50) - 1
y1 = np.random.randn(50) + 1

fig = go.Figure()
fig.add_trace(go.Box(y=y0))
fig.add_trace(go.Box(y=y1))

fig.show()
```

Basic Horizontal Box Plot

```
import plotly.graph_objects as go
import numpy as np

x0 = np.random.randn(50)
x1 = np.random.randn(50) + 2 # shift mean

fig = go.Figure()
# Use x instead of y argument for horizontal plot
fig.add_trace(go.Box(x=x0))
fig.add_trace(go.Box(x=x1))

fig.show()
```

Box Plot That Displays The Underlying Data

```
import plotly.graph_objects as go

fig = go.Figure(data=[go.Box(y=[0, 1, 1, 2, 3, 5, 8, 13, 21],
            boxpoints='all', # can also be outliers, or suspectedoutliers, or False
            jitter=0.3, # add some jitter for a better separation between points
            pointpos=-1.8 # relative position of points wrt box
              )])

fig.show()
```

Modifying The Algorithm For Computing Quartiles

For an explanation of how each algorithm works, see Choosing The Algorithm For Computing Quartiles.

```
import plotly.graph_objects as go

data = [1, 2, 3, 4, 5, 6, 7, 8, 9]

fig = go.Figure()
fig.add_trace(go.Box(y=data, quartilemethod="linear", name="Linear Quartile Mode"))
fig.add_trace(go.Box(y=data, quartilemethod="inclusive", name="Inclusive Quartile Mode"))
fig.add_trace(go.Box(y=data, quartilemethod="exclusive", name="Exclusive Quartile Mode"))
```

```
fig.update_traces(boxpoints='all', jitter=0)
fig.show()
```

Box Plot With Precomputed Quartiles

You can specify precomputed quartile attributes rather than using a built-in quartile computation algorithm.

This could be useful if you have already pre-computed those values or if you need to use a different algorithm than the ones provided.

```
import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Box(q1=[ 1, 2, 3 ], median=[ 4, 5, 6 ],
                  q3=[ 7, 8, 9 ], lowerfence=[-1, 0, 1],
                  upperfence=[7, 8, 9], mean=[ 2.2, 2.8, 3.2 ],
                  sd=[ 0.2, 0.4, 0.6 ], notchspan=[ 0.2, 0.4, 0.6 ], name="Precompiled Quartiles"))

fig.show()
```

Colored Box Plot

```
import plotly.graph_objects as go
import numpy as np

y0 = np.random.randn(50)
y1 = np.random.randn(50) + 1 # shift mean

fig = go.Figure()
fig.add_trace(go.Box(y=y0, name='Sample A',
                marker_color = 'indianred'))
fig.add_trace(go.Box(y=y1, name = 'Sample B',
                marker_color = 'lightseagreen'))

fig.show()
```

Box Plot Styling Mean & Standard Deviation

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Box(
    y=[2.37, 2.16, 4.82, 1.73, 1.04, 0.23, 1.32, 2.91, 0.11, 4.51, 0.51, 3.75, 1.35, 2.98, 4.50, 0.18,
↪  4.66, 1.30, 2.06, 1.19],
    name='Only Mean',
    marker_color='darkblue',
    boxmean=True # represent mean
))
fig.add_trace(go.Box(
    y=[2.37, 2.16, 4.82, 1.73, 1.04, 0.23, 1.32, 2.91, 0.11, 4.51, 0.51, 3.75, 1.35, 2.98, 4.50, 0.18,
↪  4.66, 1.30, 2.06, 1.19],
    name='Mean & SD',
    marker_color='royalblue',
    boxmean='sd' # represent mean and standard deviation
```

```
))

fig.show()
```

Styling Outliers

The example below shows how to use the `boxpoints` argument. If "outliers", only the sample points lying outside the whiskers are shown. If "suspectedoutliers", the outlier points are shown and points either less than 4Q1-3Q3 or greater than 4Q3-3Q1 are highlighted (using `outliercolor`). If "all", all sample points are shown. If False, only the boxes are shown with no sample points.

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Box(
    y=[0.75, 5.25, 5.5, 6, 6.2, 6.6, 6.80, 7.0, 7.2, 7.5, 7.5, 7.75, 8.15,
       8.15, 8.65, 8.93, 9.2, 9.5, 10, 10.25, 11.5, 12, 16, 20.90, 22.3, 23.25],
    name="All Points",
    jitter=0.3,
    pointpos=-1.8,
    boxpoints='all', # represent all points
    marker_color='rgb(7,40,89)',
    line_color='rgb(7,40,89)'
))

fig.add_trace(go.Box(
    y=[0.75, 5.25, 5.5, 6, 6.2, 6.6, 6.80, 7.0, 7.2, 7.5, 7.5, 7.75, 8.15,
        8.15, 8.65, 8.93, 9.2, 9.5, 10, 10.25, 11.5, 12, 16, 20.90, 22.3, 23.25],
    name="Only Whiskers",
    boxpoints=False, # no data points
    marker_color='rgb(9,56,125)',
    line_color='rgb(9,56,125)'
))

fig.add_trace(go.Box(
    y=[0.75, 5.25, 5.5, 6, 6.2, 6.6, 6.80, 7.0, 7.2, 7.5, 7.5, 7.75, 8.15,
        8.15, 8.65, 8.93, 9.2, 9.5, 10, 10.25, 11.5, 12, 16, 20.90, 22.3, 23.25],
    name="Suspected Outliers",
    boxpoints='suspectedoutliers', # only suspected outliers
    marker=dict(
        color='rgb(8,81,156)',
        outliercolor='rgba(219, 64, 82, 0.6)',
        line=dict(
            outliercolor='rgba(219, 64, 82, 0.6)',
            outlierwidth=2)),
    line_color='rgb(8,81,156)'
))

fig.add_trace(go.Box(
    y=[0.75, 5.25, 5.5, 6, 6.2, 6.6, 6.80, 7.0, 7.2, 7.5, 7.5, 7.75, 8.15,
        8.15, 8.65, 8.93, 9.2, 9.5, 10, 10.25, 11.5, 12, 16, 20.90, 22.3, 23.25],
    name="Whiskers and Outliers",
    boxpoints='outliers', # only outliers
    marker_color='rgb(107,174,214)',
    line_color='rgb(107,174,214)'
))
```

```
fig.update_layout(title_text="Box Plot Styling Outliers")
fig.show()
```

Grouped Box Plots

```
import plotly.graph_objects as go

x = ['day 1', 'day 1', 'day 1', 'day 1', 'day 1', 'day 1',
     'day 2', 'day 2', 'day 2', 'day 2', 'day 2', 'day 2']

fig = go.Figure()

fig.add_trace(go.Box(
    y=[0.2, 0.2, 0.6, 1.0, 0.5, 0.4, 0.2, 0.7, 0.9, 0.1, 0.5, 0.3],
    x=x,
    name='kale',
    marker_color='#3D9970'
))
fig.add_trace(go.Box(
    y=[0.6, 0.7, 0.3, 0.6, 0.0, 0.5, 0.7, 0.9, 0.5, 0.8, 0.7, 0.2],
    x=x,
    name='radishes',
    marker_color='#FF4136'
))
fig.add_trace(go.Box(
    y=[0.1, 0.3, 0.1, 0.9, 0.6, 0.6, 0.9, 1.0, 0.3, 0.6, 0.8, 0.5],
    x=x,
    name='carrots',
    marker_color='#FF851B'
))

fig.update_layout(
    yaxis=dict(
        title=dict(
            text='normalized moisture')
    ),
    boxmode='group' # group together boxes of the different traces for each value of x
)
fig.show()
```

Grouped Horizontal Box Plot

```
import plotly.graph_objects as go

y = ['day 1', 'day 1', 'day 1', 'day 1', 'day 1', 'day 1',
     'day 2', 'day 2', 'day 2', 'day 2', 'day 2', 'day 2']

fig = go.Figure()
fig.add_trace(go.Box(
    x=[0.2, 0.2, 0.6, 1.0, 0.5, 0.4, 0.2, 0.7, 0.9, 0.1, 0.5, 0.3],
    y=y,
    name='kale',
    marker_color='#3D9970'
))
fig.add_trace(go.Box(
```

```
    x=[0.6, 0.7, 0.3, 0.6, 0.0, 0.5, 0.7, 0.9, 0.5, 0.8, 0.7, 0.2],
    y=y,
    name='radishes',
    marker_color='#FF4136'
))
fig.add_trace(go.Box(
    x=[0.1, 0.3, 0.1, 0.9, 0.6, 0.6, 0.9, 1.0, 0.3, 0.6, 0.8, 0.5],
    y=y,
    name='carrots',
    marker_color='#FF851B'
))

fig.update_layout(
    xaxis=dict(title=dict(text='normalized moisture'), zeroline=False),
    boxmode='group'
)

fig.update_traces(orientation='h') # horizontal box plots
fig.show()
```

Rainbow Box Plots

```
import plotly.graph_objects as go
import numpy as np

N = 30     # Number of boxes

# generate an array of rainbow colors by fixing the saturation and lightness of the HSL
# representation of colour and marching around the hue.
# Plotly accepts any CSS color format, see e.g. http://www.w3schools.com/cssref/css_colors_legal.asp.
c = ['hsl('+str(h)+',50%'+',50%)' for h in np.linspace(0, 360, N)]

# Each box is represented by a dict that contains the data, the type, and the colour.
# Use list comprehension to describe N boxes, each with a different colour and with different randomly
↪  generated data:
fig = go.Figure(data=[go.Box(
    y=3.5 * np.sin(np.pi * i/N) + i/N + (1.5 + 0.5 * np.cos(np.pi*i/N)) * np.random.rand(10),
    marker_color=c[i]
    ) for i in range(int(N))])

# format the layout
fig.update_layout(
    xaxis=dict(showgrid=False, zeroline=False, showticklabels=False),
    yaxis=dict(zeroline=False, gridcolor='white'),
    paper_bgcolor='rgb(233,233,233)',
    plot_bgcolor='rgb(233,233,233)',
)

fig.show()
```

Fully Styled Box Plots

```
import plotly.graph_objects as go
import numpy as np

x_data = ['Carmelo Anthony', 'Dwyane Wade',
```

```
          'Deron Williams', 'Brook Lopez',
          'Damian Lillard', 'David West',]

N = 50

y0 = (10 * np.random.randn(N) + 30).astype(int)
y1 = (13 * np.random.randn(N) + 38).astype(int)
y2 = (11 * np.random.randn(N) + 33).astype(int)
y3 = (9 * np.random.randn(N) + 36).astype(int)
y4 = (15 * np.random.randn(N) + 31).astype(int)
y5 = (12 * np.random.randn(N) + 40).astype(int)

y_data = [y0, y1, y2, y3, y4, y5]

colors = ['rgba(93, 164, 214, 0.5)', 'rgba(255, 144, 14, 0.5)', 'rgba(44, 160, 101, 0.5)',
          'rgba(255, 65, 54, 0.5)', 'rgba(207, 114, 255, 0.5)', 'rgba(127, 96, 0, 0.5)']

fig = go.Figure()

for xd, yd, cls in zip(x_data, y_data, colors):
        fig.add_trace(go.Box(
            y=yd,
            name=xd,
            boxpoints='all',
            jitter=0.5,
            whiskerwidth=0.2,
            fillcolor=cls,
            marker_size=2,
            line_width=1)
        )

fig.update_layout(
    title=dict(text='Points Scored by the Top 9 Scoring NBA Players in 2012'),
    yaxis=dict(
        autorange=True,
        showgrid=True,
        zeroline=True,
        dtick=5,
        gridcolor='rgb(255, 255, 255)',
        gridwidth=1,
        zerolinecolor='rgb(255, 255, 255)',
        zerolinewidth=2,
    ),
    margin=dict(
        l=40,
        r=30,
        b=80,
        t=100,
    ),
    paper_bgcolor='rgb(243, 243, 243)',
    plot_bgcolor='rgb(243, 243, 243)',
    showlegend=False
)

fig.show()
```

Box Plot With Only Points

A strip chart is like a box plot with points showing, and no box:

```
import plotly.express as px
df = px.data.tips()
fig = px.strip(df, x='day', y='tip')
fig.show()
```

Reference

See function reference for `px.box()` or https://plotly.com/python/reference/box/ for more information and chart attribute options!

Heatmaps

Source: https://plotly.com/python/heatmaps/

The term “heatmap” usually refers to a Cartesian plot with data visualized as colored rectangular tiles, which is the subject of this page. It is also sometimes used to refer to actual maps with density data displayed as color intensity.

Plotly supports two different types of colored-tile heatmaps:

1. **Matrix Heatmaps** accept a 2-dimensional matrix or array of data and visualizes it directly. This type of heatmap is the subject of this page.
2. **Density Heatmaps** accept data as a list and visualizes aggregated quantities like counts or sums of this data. Please refer to the 2D Histogram documentation for this kind of figure.

Heatmaps with Plotly Express

Plotly Express is the easy-to-use, high-level interface to Plotly, which operates on a variety of types of data and produces easy-to-style figures. With `px.imshow`, each value of the input array or data frame is represented as a heatmap pixel.

<!- #region -> The `px.imshow()` function can be used to display heatmaps (as well as full-color images, as its name suggests). It accepts both array-like objects like lists of lists and `numpy` or `xarray` arrays, as well as supported DataFrame objects.

> For more examples using `px.imshow`, including examples of faceting and animations, as well as full-color image display, see the the `imshow` documentation page. <!- #endregion ->

```
import plotly.express as px

fig = px.imshow([[1, 20, 30],
                 [20, 1, 60],
                 [30, 60, 1]])
fig.show()
```

```
import plotly.express as px

df = px.data.medals_wide(indexed=True)
fig = px.imshow(df)
fig.show()
```

<!- #region -> ### Displaying Text on Heatmaps

New in v5.5

You can add the values to the figure as text using the `text_auto` argument. Setting it to `True` will display the values on the bars, and setting it to a `d3-format` formatting string will control the output format. <!- #endregion ->

```
import plotly.express as px

z = [[.1, .3, .5, .7, .9],
     [1, .8, .6, .4, .2],
     [.2, 0, .5, .7, .9],
     [.9, .8, .4, .2, 0],
```

```
     [.3, .4, .5, .7, 1]]

fig = px.imshow(z, text_auto=True)
fig.show()
```

Controlling Aspect Ratio

By default, `px.imshow()` produces heatmaps with square tiles, but setting the `aspect` argument to `"auto"` will instead fill the plotting area with the heatmap, using non-square tiles.

```
import plotly.express as px

z = [[.1, .3, .5, .7, .9],
     [1, .8, .6, .4, .2],
     [.2, 0, .5, .7, .9],
     [.9, .8, .4, .2, 0],
     [.3, .4, .5, .7, 1]]

fig = px.imshow(z, text_auto=True, aspect="auto")
fig.show()
```

Customizing the axes and labels on a heatmap

You can use the `x`, `y` and `labels` arguments to customize the display of a heatmap, and use `.update_xaxes()` to move the x axis tick labels to the top:

```
import plotly.express as px
data=[[1, 25, 30, 50, 1], [20, 1, 60, 80, 30], [30, 60, 1, 5, 20]]
fig = px.imshow(data,
                labels=dict(x="Day of Week", y="Time of Day", color="Productivity"),
                x=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],
                y=['Morning', 'Afternoon', 'Evening']
               )
fig.update_xaxes(side="top")
fig.show()
```

Display an xarray image with px.imshow

xarrays are labeled arrays (with labeled axes and coordinates). If you pass an xarray image to `px.imshow`, its axes labels and coordinates will be used for axis titles. If you don't want this behavior, you can pass `img.values` which is a NumPy array if `img` is an xarray. Alternatively, you can override axis titles hover labels and colorbar title using the `labels` attribute, as above.

```
import plotly.express as px
import xarray as xr
# Load xarray from dataset included in the xarray tutorial
airtemps = xr.tutorial.open_dataset('air_temperature').air.sel(lon=250.0)
fig = px.imshow(airtemps.T, color_continuous_scale='RdBu_r', origin='lower')
fig.show()
```

Basic Heatmap with `plotly.graph_objects`

If Plotly Express does not provide a good starting point, it is also possible to use the more generic `go.Heatmap` class from `plotly.graph_objects`.

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Heatmap(
                    z=[[1, 20, 30],
                      [20, 1, 60],
                      [30, 60, 1]]))
fig.show()
```

Heatmap with Categorical Axis Labels

In this example we also show how to ignore hovertext when we have missing values in the data by setting the hoverongaps to False.

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Heatmap(
                   z=[[1, None, 30, 50, 1], [20, 1, 60, 80, 30], [30, 60, 1, -10, 20]],
                   x=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],
                   y=['Morning', 'Afternoon', 'Evening'],
                   hoverongaps = False))
fig.show()
```

Heatmap with Unequal Block Sizes

```
import plotly.graph_objects as go
import numpy as np

# Build the rectangles as a heatmap
# specify the edges of the heatmap squares
phi = (1 + np.sqrt(5) )/2. # golden ratio
xe = [0, 1, 1+(1/(phi**4)), 1+(1/(phi**3)), phi]
ye = [0, 1/(phi**3), 1/phi**3+1/phi**4, 1/(phi**2), 1]

z = [ [13,3,3,5],
      [13,2,1,5],
      [13,10,11,12],
      [13,8,8,8]
    ]

fig = go.Figure(data=go.Heatmap(
          x = np.sort(xe),
          y = np.sort(ye),
          z = z,
          type = 'heatmap',
          colorscale = 'Viridis'))

# Add spiral line plot

def spiral(th):
    a = 1.120529
```

```
    b = 0.306349
    r = a*np.exp(-b*th)
    return (r*np.cos(th), r*np.sin(th))

theta = np.linspace(-np.pi/13,4*np.pi,1000); # angle
(x,y) = spiral(theta)

fig.add_trace(go.Scatter(x= -x+x[0], y= y-y[0],
     line =dict(color='white',width=3)))

axis_template = dict(range = [0,1.6], autorange = False,
              showgrid = False, zeroline = False,
              linecolor = 'black', showticklabels = False,
              ticks = '' )

fig.update_layout(margin = dict(t=200,r=200,b=200,l=200),
    xaxis = axis_template,
    yaxis = axis_template,
    showlegend = False,
    width = 700, height = 700,
    autosize = False )

fig.show()
```

Heatmap with Datetime Axis

```
import plotly.graph_objects as go
import datetime
import numpy as np
np.random.seed(1)

programmers = ['Alex','Nicole','Sara','Etienne','Chelsea','Jody','Marianne']

base = datetime.datetime.today()
dates = base - np.arange(180) * datetime.timedelta(days=1)
z = np.random.poisson(size=(len(programmers), len(dates)))

fig = go.Figure(data=go.Heatmap(
        z=z,
        x=dates,
        y=programmers,
        colorscale='Viridis'))

fig.update_layout(
    title=dict(text='GitHub commits per day'),
    xaxis_nticks=36)

fig.show()
```

Text on Heatmap Points

In this example we add text to heatmap points using `texttemplate`. We use the values from the `text` attribute for the text. We also adjust the font size using `textfont`.

```
import plotly.graph_objects as go

fig = go.Figure(data=go.Heatmap(
                    z=[[1, 20, 30],
                      [20, 1, 60],
                      [30, 60, 1]],
                    text=[['one', 'twenty', 'thirty'],
                          ['twenty', 'one', 'sixty'],
                          ['thirty', 'sixty', 'one']],
                    texttemplate="%{text}",
                    textfont={"size":20}))

fig.show()
```

Heatmap and datashader

Arrays of rasterized values build by datashader can be visualized using plotly's heatmaps, as shown in the plotly and datashader tutorial.

Reference

See function reference for `px.(imshow)` or https://plotly.com/python/reference/heatmap/ for more information and chart attribute options!

Candlestick charts

Source: https://plotly.com/python/candlestick-charts/

The candlestick chart is a style of financial chart describing open, high, low and close for a given `x` coordinate (most likely time). The boxes represent the spread between the `open` and `close` values and the lines represent the spread between the `low` and `high` values. Sample points where the close value is higher (lower) then the open value are called increasing (decreasing). By default, increasing candles are drawn in green whereas decreasing are drawn in red.

Simple Candlestick with Pandas

```
import plotly.graph_objects as go

import pandas as pd
from datetime import datetime

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=[go.Candlestick(x=df['Date'],
                open=df['AAPL.Open'],
                high=df['AAPL.High'],
                low=df['AAPL.Low'],
                close=df['AAPL.Close'])])

fig.show()
```

Candlestick without Rangeslider

```
import plotly.graph_objects as go
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=[go.Candlestick(x=df['Date'],
                open=df['AAPL.Open'], high=df['AAPL.High'],
                low=df['AAPL.Low'], close=df['AAPL.Close'])
                     ])

fig.update_layout(xaxis_rangeslider_visible=False)
fig.show()
```

Adding Customized Text and Annotations

```
import plotly.graph_objects as go
import pandas as pd


df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')
```

```
fig = go.Figure(data=[go.Candlestick(x=df['Date'],
                open=df['AAPL.Open'], high=df['AAPL.High'],
                low=df['AAPL.Low'], close=df['AAPL.Close'])
                      ])

fig.update_layout(
    title=dict(text='The Great Recession'),
    yaxis=dict(
      title=dict(
        text='AAPL Stock'
        )
    ),
    shapes = [dict(
        x0='2016-12-09', x1='2016-12-09', y0=0, y1=1, xref='x', yref='paper',
        line_width=2)],
    annotations=[dict(
        x='2016-12-09', y=0.05, xref='x', yref='paper',
        showarrow=False, xanchor='left', text='Increase Period Begins')]
)

fig.show()
```

Custom Candlestick Colors

```
import plotly.graph_objects as go
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=[go.Candlestick(
    x=df['Date'],
    open=df['AAPL.Open'], high=df['AAPL.High'],
    low=df['AAPL.Low'], close=df['AAPL.Close'],
    increasing_line_color= 'cyan', decreasing_line_color= 'gray'
)])

fig.show()
```

Simple Example with `datetime` Objects

```
import plotly.graph_objects as go
from datetime import datetime

open_data = [33.0, 33.3, 33.5, 33.0, 34.1]
high_data = [33.1, 33.3, 33.6, 33.2, 34.8]
low_data = [32.7, 32.7, 32.8, 32.6, 32.8]
close_data = [33.0, 32.9, 33.3, 33.1, 33.1]
dates = [datetime(year=2013, month=10, day=10),
         datetime(year=2013, month=11, day=10),
         datetime(year=2013, month=12, day=10),
         datetime(year=2014, month=1, day=10),
         datetime(year=2014, month=2, day=10)]

fig = go.Figure(data=[go.Candlestick(x=dates,
```

```
                      open=open_data, high=high_data,
                      low=low_data, close=close_data)])

fig.show()
```

Reference

For more information on candlestick attributes, see: https://plotly.com/python/reference/candlestick/

OHLC charts

Source: https://plotly.com/python/ohlc-charts/

The OHLC chart (for open, high, low and close) is a style of financial chart describing open, high, low and close values for a given `x` coordinate (most likely time). The tip of the lines represent the `low` and `high` values and the horizontal segments represent the `open` and `close` values. Sample points where the close value is higher (lower) then the open value are called increasing (decreasing). By default, increasing items are drawn in green whereas decreasing are drawn in red.

See also Candlestick Charts and other financial charts.

Simple OHLC Chart with Pandas

```
import plotly.graph_objects as go
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=go.Ohlc(x=df['Date'],
                    open=df['AAPL.Open'],
                    high=df['AAPL.High'],
                    low=df['AAPL.Low'],
                    close=df['AAPL.Close']))
fig.show()
```

OHLC Chart without Rangeslider

```
import plotly.graph_objects as go

import pandas as pd


df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=go.Ohlc(x=df['Date'],
                open=df['AAPL.Open'],
                high=df['AAPL.High'],
                low=df['AAPL.Low'],
                close=df['AAPL.Close']))
fig.update(layout_xaxis_rangeslider_visible=False)
fig.show()
```

Adding Customized Text and Annotations

```
import plotly.graph_objects as go
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=go.Ohlc(x=df['Date'],
```

```
                open=df['AAPL.Open'],
                high=df['AAPL.High'],
                low=df['AAPL.Low'],
                close=df['AAPL.Close']))

fig.update_layout(
    title=dict(text='The Great Recession'),
    yaxis=dict(title=dict(text='AAPL Stock')),
    shapes = [dict(
        x0='2016-12-09', x1='2016-12-09', y0=0, y1=1, xref='x', yref='paper',
        line_width=2)],
    annotations=[dict(
        x='2016-12-09', y=0.05, xref='x', yref='paper',
        showarrow=False, xanchor='left', text='Increase Period Begins')]
)

fig.show()
```

Custom OHLC Colors

```
import plotly.graph_objects as go
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=[go.Ohlc(
    x=df['Date'],
    open=df['AAPL.Open'], high=df['AAPL.High'],
    low=df['AAPL.Low'], close=df['AAPL.Close'],
    increasing_line_color= 'cyan', decreasing_line_color= 'gray'
)])
fig.show()
```

Simple OHLC with `datetime` Objects

```
import plotly.graph_objects as go

from datetime import datetime

open_data = [33.0, 33.3, 33.5, 33.0, 34.1]
high_data = [33.1, 33.3, 33.6, 33.2, 34.8]
low_data = [32.7, 32.7, 32.8, 32.6, 32.8]
close_data = [33.0, 32.9, 33.3, 33.1, 33.1]
dates = [datetime(year=2013, month=10, day=10),
         datetime(year=2013, month=11, day=10),
         datetime(year=2013, month=12, day=10),
         datetime(year=2014, month=1, day=10),
         datetime(year=2014, month=2, day=10)]

fig = go.Figure(data=[go.Ohlc(x=dates,
                          open=open_data, high=high_data,
                          low=low_data, close=close_data)])
fig.show()
```

Custom Hovertext

```
import plotly.graph_objects as go

import pandas as pd
from datetime import datetime

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

hovertext=[]
for i in range(len(df['AAPL.Open'])):
    hovertext.append('Open: '+str(df['AAPL.Open'][i])+'Close: '+str(df['AAPL.Close'][i]))

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/finance-charts-apple.csv')

fig = go.Figure(data=go.Ohlc(x=df['Date'],
                open=df['AAPL.Open'],
                high=df['AAPL.High'],
                low=df['AAPL.Low'],
                close=df['AAPL.Close'],
                text=hovertext,
                hoverinfo='text'))
fig.show()
```

Reference

For more information on candlestick attributes, see: https://plotly.com/python/reference/ohlc/

API quick reference

Generated from the docstrings of the installed Plotly 7.1.0 package, which are the text the official API reference pages are built from. Full reference: https://plotly.com/python-api-reference/

Plotly Express

plotly.express.line

```
plotly.express.line(data_frame=None, x=None, y=None, line_group=None, color=None, line_dash=None,
↪  symbol=None, hover_name=None, hover_data=None, custom_data=None, text=None, facet_row=None,
↪  facet_col=None, facet_col_wrap=0, facet_row_spacing=None, facet_col_spacing=None, error_x=None,
↪  error_x_minus=None, error_y=None, error_y_minus=None, animation_frame=None, animation_group=None,
↪  category_orders=None, labels=None,  ...)
```

In a 2D line plot, each row of data_frame is represented as a vertex of a polyline mark in 2D space.

Parameters

`data_frame: DataFrame or array-like or dict` This argument needs to be passed for column names (and not keyword names) to be used. Array-like and dict are transformed internally to a pandas DataFrame. Optional: if missing, a DataFrame gets constructed under the hood using the other arguments.

`x: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the x axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

`y: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the y axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

`line_group: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to group rows of data_frame into lines.

`color: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign color to marks.

`line_dash: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign dash-patterns to lines.

`symbol: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign symbols to marks.

hover_name: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in bold in the hover tooltip.

hover_data: str, or list of str or int, or Series or array-like, or dict Either a name or list of names of columns in data_frame, or pandas Series, or array_like objects or a dict with column names as keys, with values True (for default formatting) False (in order to remove this column from hover information), or a formatting string, for example ':.3f' or '|%a' or list-like data to appear in the hover tooltip or tuples with a bool or formatting string as first element, and list-like data to appear in hover as second element Values from these columns appear as extra data in the hover tooltip.

custom_data: str, or list of str or int, or Series or array-like Either name or list of names of columns in data_frame, or pandas Series, or array_like objects Values from these columns are extra data, to be used in widgets or Dash callbacks for example. This data is not user-visible but is included in events emitted by the figure (lasso selection etc.)

text: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in the figure as text labels.

facet_row: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the vertical direction.

facet_col: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the horizontal direction.

facet_col_wrap: int Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if 0, and forced to 0 if facet_row or a marginal is set.

facet_row_spacing: float between 0 and 1 Spacing between facet rows, in paper units. Default is 0.03 or 0.07 when facet_col_wrap is used.

facet_col_spacing: float between 0 and 1 Spacing between facet columns, in paper units Default is 0.02.

error_x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars. If error_x_minus is None, error bars will be symmetrical, otherwise error_x is used for the positive direction only.

error_x_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars in the negative direction. Ignored if error_x is None.

error_y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars. If error_y_minus is None, error bars will be symmetrical, otherwise error_y is used for the positive direction only.

error_y_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars in the negative direction. Ignored if error_y is None.

animation_frame: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to animation frames.

animation_group: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to provide object-constancy across animation frames: rows with matching 'animation_group's will be treated as if they describe the same object in each frame.

category_orders: dict with str keys and list of str values (default `{}`) By default, in Python 3.6+, the order of categorical values in axes, legends and facets depends on the order in which these values are first encountered in data_frame (and no order is guaranteed by default in Python below 3.6). This parameter is used to force a specific ordering of values per column. The keys of this dict should correspond to column names, and the values should be lists of strings corresponding to the specific display order desired.

labels: dict with str keys and str values (default `{}`) By default, column names are used in the figure for axis titles, legend entries and hovers. This parameter allows this to be overridden. The keys of this dict should correspond to column names, and the values should correspond to the desired label to be displayed.

orientation: str, one of `'h'` for horizontal or `'v'` for vertical. (default 'v' if x and y are provided and both continuous or both categorical, otherwise 'v'('h') if x(y) is categorical and y(x) is continuous, otherwise 'v'('h') if only x(y) is provided)

color_discrete_sequence: list of str Strings should define valid CSS-colors. When color is set and the values in the corresponding column are not numeric, values in that column are assigned colors by cycling through color_discrete_sequence in the order described in category_orders, unless the value of color is a key in color_discrete_map. Various useful color sequences are available in the plotly.express.colors submodules, specifically plotly.express.colors.qualitative.

color_discrete_map: dict with str keys and str values (default `{}`) String values should define valid CSS-colors Used to override color_discrete_sequence to assign a specific colors to marks corresponding with specific values. Keys in color_discrete_map should be values in the column denoted by color. Alternatively, if the values of color are valid colors, the string 'identity' may be passed to cause them to be used directly.

line_dash_sequence: list of str Strings should define valid plotly.js dash-patterns. When line_dash is set, values in that column are assigned dash-patterns by cycling through line_dash_sequence in the order described in category_orders, unless the value of line_dash is a key in line_dash_map.

line_dash_map: dict with str keys and str values (default `{}`) Strings values define plotly.js dash-patterns. Used to override line_dash_sequences to assign a specific dash-patterns to lines corresponding with specific values. Keys in line_dash_map should be values in the column denoted by line_dash. Alternatively, if the values of line_dash are valid line-dash names, the string 'identity' may be passed to cause them to be used directly.

symbol_sequence: list of str Strings should define valid plotly.js symbols. When symbol is set, values in that column are assigned symbols by cycling through symbol_sequence in the order described in category_orders, unless the value of symbol is a key in symbol_map.

symbol_map: dict with str keys and str values (default `{}`) String values should define plotly.js symbols Used to override symbol_sequence to assign a specific symbols to marks corresponding with specific values. Keys in symbol_map should be values in the column denoted by symbol. Alternatively, if the values of symbol are valid symbol names, the string 'identity' may be passed to cause them to be used directly.

markers: boolean (default `False`) If True, markers are shown on lines.

log_x: boolean (default `False`) If True, the x-axis is log-scaled in cartesian coordinates.

log_y: boolean (default `False`) If True, the y-axis is log-scaled in cartesian coordinates.

range_x: list of two numbers If provided, overrides auto-scaling on the x-axis in cartesian coordinates.

range_y: list of two numbers If provided, overrides auto-scaling on the y-axis in cartesian coordinates.

line_shape: str (default `'linear'`) One of 'linear', 'spline', 'hv', 'vh', 'hvh', or 'vhv'

render_mode: str One of 'auto', 'svg' or 'webgl', default 'auto' Controls the browser API used to draw marks. 'svg' is appropriate for figures of less than 1000 data points, and will allow for fully-vectorized output. 'webgl' is likely necessary for acceptable performance above 1000 points but rasterizes part of the output. 'auto' uses heuristics to choose the mode.

title: str The figure title.

subtitle: str The figure subtitle.

template: str or dict or plotly.graph_objects.layout.Template instance The figure template name (must be a key in plotly.io.templates) or definition.

width: int (default `None`) The figure width in pixels.

height: int (default `None`) The figure height in pixels.

Returns

plotly.graph_objects.Figure

plotly.express.scatter

```
plotly.express.scatter(data_frame=None, x=None, y=None, color=None, symbol=None, size=None,
↪  hover_name=None, hover_data=None, custom_data=None, text=None, facet_row=None, facet_col=None,
↪  facet_col_wrap=0, facet_row_spacing=None, facet_col_spacing=None, error_x=None, error_x_minus=None,
↪  error_y=None, error_y_minus=None, animation_frame=None, animation_group=None, category_orders=None,
↪  labels=None, orientation=None, colo ...)
```

In a scatter plot, each row of data_frame is represented by a symbol mark in 2D space.

Parameters

data_frame: DataFrame or array-like or dict This argument needs to be passed for column names (and not keyword names) to be used. Array-like and dict are transformed internally to a pandas DataFrame. Optional: if missing, a DataFrame gets constructed under the hood using the other arguments.

x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the x axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the y axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

color: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign color to marks.

symbol: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign symbols to marks.

size: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign mark sizes.

hover_name: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in bold in the hover tooltip.

hover_data: str, or list of str or int, or Series or array-like, or dict Either a name or list of names of columns in data_frame, or pandas Series, or array_like objects or a dict with column names as keys, with values True (for default formatting) False (in order to remove this column from hover information), or a formatting string, for example ':.3f' or '|%a' or list-like data to appear in the hover tooltip or tuples with a bool or formatting string as first element, and list-like data to appear in hover as second element Values from these columns appear as extra data in the hover tooltip.

custom_data: str, or list of str or int, or Series or array-like Either name or list of names of columns in data_frame, or pandas Series, or array_like objects Values from these columns are extra data, to be used in widgets or Dash callbacks for example. This data is not user-visible but is included in events emitted by the figure (lasso selection etc.)

text: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in the figure as text labels.

facet_row: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the vertical direction.

facet_col: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the horizontal direction.

facet_col_wrap: int Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if 0, and forced to 0 if facet_row or a marginal is set.

facet_row_spacing: float between 0 and 1 Spacing between facet rows, in paper units. Default is 0.03 or 0.07 when facet_col_wrap is used.

facet_col_spacing: float between 0 and 1 Spacing between facet columns, in paper units Default is 0.02.

error_x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars. If error_x_minus is None, error bars will be symmetrical, otherwise error_x is used for the positive direction only.

error_x_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars in the negative direction. Ignored if error_x is None.

error_y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars. If error_y_minus is None, error bars will be symmetrical, otherwise error_y is used for the positive direction only.

error_y_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars in the negative direction. Ignored if error_y is None.

animation_frame: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to animation frames.

animation_group: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to provide object-constancy across animation frames: rows with matching 'animation_group's will be treated as if they describe the same object in each frame.

category_orders: dict with str keys and list of str values (default `{}`) By default, in Python 3.6+, the order of categorical values in axes, legends and facets depends on the order in which these values are first encountered in data_frame (and no order is guaranteed by default in Python below 3.6). This parameter is used to force a specific ordering of values per column. The keys of this dict should correspond to column names, and the values should be lists of strings corresponding to the specific display order desired.

labels: dict with str keys and str values (default `{}`) By default, column names are used in the figure for axis titles, legend entries and hovers. This parameter allows this to be overridden. The keys of this dict should correspond to column names, and the values should correspond to the desired label to be displayed.

orientation: str, one of `'h'` for horizontal or `'v'` for vertical. (default 'v' if x and y are provided and both continuous or both categorical, otherwise 'v'('h') if x(y) is categorical and y(x) is continuous, otherwise 'v'('h') if only x(y) is provided)

color_discrete_sequence: list of str Strings should define valid CSS-colors. When color is set and the values in the corresponding column are not numeric, values in that column are assigned colors by cycling through color_discrete_sequence in the order described in category_orders, unless the value of color is a key in color_discrete_map. Various useful color sequences are available in the plotly.express.colors submodules, specifically plotly.express.colors.qualitative.

color_discrete_map: dict with str keys and str values (default `{}`) String values should define valid CSS-colors Used to override color_discrete_sequence to assign a specific colors to marks corresponding with specific values. Keys in color_discrete_map should be values in the column denoted by color. Alternatively, if the values of color are valid colors, the string 'identity' may be passed to cause them to be used directly.

color_continuous_scale: list of str Strings should define valid CSS-colors This list is used to build a continuous color scale when the column denoted by color contains numeric data. Various useful color scales are available in the plotly.express.colors submodules, specifically plotly.express.colors.sequential, plotly.express.colors.diverging and plotly.express.colors.cyclical.

range_color: list of two numbers If provided, overrides auto-scaling on the continuous color scale.

color_continuous_midpoint: number (default `None`) If set, computes the bounds of the continuous color scale to have the desired midpoint. Setting this value is recommended when using plotly.express.colors.diverging color scales as the inputs to color_continuous_scale.

symbol_sequence: list of str Strings should define valid plotly.js symbols. When symbol is set, values in that column are assigned symbols by cycling through symbol_sequence in the order described in category_orders, unless the value of symbol is a key in symbol_map.

symbol_map: dict with str keys and str values (default `{}`) String values should define plotly.js symbols Used to override symbol_sequence to assign a specific symbols to marks

corresponding with specific values. Keys in symbol_map should be values in the column denoted by symbol. Alternatively, if the values of symbol are valid symbol names, the string 'identity' may be passed to cause them to be used directly.

opacity: float Value between 0 and 1. Sets the opacity for markers.

size_max: int (default `20`) Set the maximum mark size when using size.

marginal_x: str One of 'rug', 'box', 'violin', or 'histogram'. If set, a horizontal subplot is drawn above the main plot, visualizing the x-distribution.

marginal_y: str One of 'rug', 'box', 'violin', or 'histogram'. If set, a vertical subplot is drawn to the right of the main plot, visualizing the y-distribution.

trendline: str One of 'ols', 'lowess', 'rolling', 'expanding' or 'ewm'. If 'ols', an Ordinary Least Squares regression line will be drawn for each discrete-color/symbol group. If 'lowess', a Locally Weighted Scatterplot Smoothing line will be drawn for each discrete-color/symbol group. If 'rolling', a Rolling (e.g. rolling average, rolling median) line will be drawn for each discrete-color/symbol group. If 'expanding', an Expanding (e.g. expanding average, expanding sum) line will be drawn for each discrete-color/symbol group. If 'ewm', an Exponentially Weighted Moment (e.g. exponentially-weighted moving average) line will be drawn for each discrete-color/symbol group. See the docstrings for the functions in plotly.express.trendline_functions for more details on these functions and how to configure them with the trendline_options argument.

trendline_options: dict Options passed as the first argument to the function from plotly.express.trendline_functions named in the trendline argument.

trendline_color_override: str Valid CSS color. If provided, and if trendline is set, all trendlines will be drawn in this color rather than in the same color as the traces from which they draw their inputs.

trendline_scope: str (one of `'trace'` or `'overall'`, default `'trace'`) If 'trace', then one trendline is drawn per trace (i.e. per color, symbol, facet, animation frame etc) and if 'overall' then one trendline is computed for the entire dataset, and replicated across all facets.

log_x: boolean (default `False`) If True, the x-axis is log-scaled in cartesian coordinates.

log_y: boolean (default `False`) If True, the y-axis is log-scaled in cartesian coordinates.

range_x: list of two numbers If provided, overrides auto-scaling on the x-axis in cartesian coordinates.

range_y: list of two numbers If provided, overrides auto-scaling on the y-axis in cartesian coordinates.

render_mode: str One of 'auto', 'svg' or 'webgl', default 'auto' Controls the browser API used to draw marks. 'svg' is appropriate for figures of less than 1000 data points, and will allow for fully-vectorized output. 'webgl' is likely necessary for acceptable performance above 1000 points but rasterizes part of the output. 'auto' uses heuristics to choose the mode.

title: str The figure title.

subtitle: str The figure subtitle.

template: str or dict or plotly.graph_objects.layout.Template instance The figure template name (must be a key in plotly.io.templates) or definition.

width: int (default `None`) The figure width in pixels.

height: int (default `None`) The figure height in pixels.

Returns

plotly.graph_objects.Figure

plotly.express.bar

```
plotly.express.bar(data_frame=None, x=None, y=None, color=None, pattern_shape=None, facet_row=None,
↪  facet_col=None, facet_col_wrap=0, facet_row_spacing=None, facet_col_spacing=None, hover_name=None,
↪  hover_data=None, custom_data=None, text=None, base=None, error_x=None, error_x_minus=None,
↪  error_y=None, error_y_minus=None, animation_frame=None, animation_group=None, category_orders=None,
↪  labels=None, color_discrete_ ...)
```

In a bar plot, each row of data_frame is represented as a rectangular mark.

Parameters

data_frame: DataFrame or array-like or dict This argument needs to be passed for column names (and not keyword names) to be used. Array-like and dict are transformed internally to a pandas DataFrame. Optional: if missing, a DataFrame gets constructed under the hood using the other arguments.

x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the x axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the y axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

color: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign color to marks.

pattern_shape: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign pattern shapes to marks.

facet_row: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the vertical direction.

facet_col: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the horizontal direction.

facet_col_wrap: int Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if 0, and forced to 0 if facet_row or a marginal is set.

facet_row_spacing: float between 0 and 1 Spacing between facet rows, in paper units. Default is 0.03 or 0.07 when facet_col_wrap is used.

facet_col_spacing: float between 0 and 1 Spacing between facet columns, in paper units Default is 0.02.

hover_name: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in bold in the hover tooltip.

hover_data: str, or list of str or int, or Series or array-like, or dict Either a name or list of names of columns in data_frame, or pandas Series, or array_like objects or a dict with column names as keys, with values True (for default formatting) False (in order to remove

this column from hover information), or a formatting string, for example ':.3f' or '|%a' or list-like data to appear in the hover tooltip or tuples with a bool or formatting string as first element, and list-like data to appear in hover as second element Values from these columns appear as extra data in the hover tooltip.

custom_data: str, or list of str or int, or Series or array-like Either name or list of names of columns in data_frame, or pandas Series, or array_like objects Values from these columns are extra data, to be used in widgets or Dash callbacks for example. This data is not user-visible but is included in events emitted by the figure (lasso selection etc.)

text: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in the figure as text labels.

base: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position the base of the bar.

error_x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars. If error_x_minus is None, error bars will be symmetrical, otherwise error_x is used for the positive direction only.

error_x_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size x-axis error bars in the negative direction. Ignored if error_x is None.

error_y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars. If error_y_minus is None, error bars will be symmetrical, otherwise error_y is used for the positive direction only.

error_y_minus: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to size y-axis error bars in the negative direction. Ignored if error_y is None.

animation_frame: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to animation frames.

animation_group: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to provide object-constancy across animation frames: rows with matching 'animation_group's will be treated as if they describe the same object in each frame.

category_orders: dict with str keys and list of str values (default `{}`) By default, in Python 3.6+, the order of categorical values in axes, legends and facets depends on the order in which these values are first encountered in data_frame (and no order is guaranteed by default in Python below 3.6). This parameter is used to force a specific ordering of values per column. The keys of this dict should correspond to column names, and the values should be lists of strings corresponding to the specific display order desired.

labels: dict with str keys and str values (default `{}`) By default, column names are used in the figure for axis titles, legend entries and hovers. This parameter allows this to be overridden. The keys of this dict should correspond to column names, and the values should correspond to the desired label to be displayed.

color_discrete_sequence: list of str Strings should define valid CSS-colors. When color is set and the values in the corresponding column are not numeric, values in that column are assigned colors by cycling through color_discrete_sequence in the order described in category_orders, unless the value of color is a key in color_discrete_map. Various

useful color sequences are available in the plotly.express.colors submodules, specifically plotly.express.colors.qualitative.

color_discrete_map: dict with str keys and str values (default `{}`) String values should define valid CSS-colors Used to override color_discrete_sequence to assign a specific colors to marks corresponding with specific values. Keys in color_discrete_map should be values in the column denoted by color. Alternatively, if the values of color are valid colors, the string 'identity' may be passed to cause them to be used directly.

color_continuous_scale: list of str Strings should define valid CSS-colors This list is used to build a continuous color scale when the column denoted by color contains numeric data. Various useful color scales are available in the plotly.express.colors submodules, specifically plotly.express.colors.sequential, plotly.express.colors.diverging and plotly.express.colors.cyclical.

pattern_shape_sequence: list of str Strings should define valid plotly.js patterns-shapes. When pattern_shape is set, values in that column are assigned patterns-shapes by cycling through pattern_shape_sequence in the order described in category_orders, unless the value of pattern_shape is a key in pattern_shape_map.

pattern_shape_map: dict with str keys and str values (default `{}`) Strings values define plotly.js patterns-shapes. Used to override pattern_shape_sequences to assign a specific patterns-shapes to lines corresponding with specific values. Keys in pattern_shape_map should be values in the column denoted by pattern_shape. Alternatively, if the values of pattern_shape are valid patterns-shapes names, the string 'identity' may be passed to cause them to be used directly.

range_color: list of two numbers If provided, overrides auto-scaling on the continuous color scale.

color_continuous_midpoint: number (default `None`) If set, computes the bounds of the continuous color scale to have the desired midpoint. Setting this value is recommended when using plotly.express.colors.diverging color scales as the inputs to color_continuous_scale.

opacity: float Value between 0 and 1. Sets the opacity for markers.

orientation: str, one of `'h'` for horizontal or `'v'` for vertical. (default 'v' if x and y are provided and both continuous or both categorical, otherwise 'v'('h') if x(y) is categorical and y(x) is continuous, otherwise 'v'('h') if only x(y) is provided)

barmode: str (default `'relative'`) One of 'group', 'overlay' or 'relative' In 'relative' mode, bars are stacked above zero for positive values and below zero for negative values. In 'overlay' mode, bars are drawn on top of one another. In 'group' mode, bars are placed beside each other.

log_x: boolean (default `False`) If True, the x-axis is log-scaled in cartesian coordinates.

log_y: boolean (default `False`) If True, the y-axis is log-scaled in cartesian coordinates.

range_x: list of two numbers If provided, overrides auto-scaling on the x-axis in cartesian coordinates.

range_y: list of two numbers If provided, overrides auto-scaling on the y-axis in cartesian coordinates.

text_auto: bool or string (default `False`) If True or a string, the x or y or z values will be displayed as text, depending on the orientation A string like '.2f' will be interpreted as a texttemplate numeric formatting directive.

title: str The figure title.

subtitle: str The figure subtitle.

template: str or dict or plotly.graph_objects.layout.Template instance The figure template name (must be a key in plotly.io.templates) or definition.

width: int (default `None`) The figure width in pixels.

height: int (default `None`) The figure height in pixels.

Returns

plotly.graph_objects.Figure

plotly.express.histogram

```
plotly.express.histogram(data_frame=None, x=None, y=None, color=None, pattern_shape=None, facet_row=None,
↪  facet_col=None, facet_col_wrap=0, facet_row_spacing=None, facet_col_spacing=None, hover_name=None,
↪  hover_data=None, animation_frame=None, animation_group=None, category_orders=None, labels=None,
↪  color_discrete_sequence=None, color_discrete_map=None, pattern_shape_sequence=None,
↪  pattern_shape_map=None, marginal=None,  ...)
```

In a histogram, rows of data_frame are grouped together into a rectangular mark to visualize the 1D distribution of an aggregate function histfunc (e.g. the count or sum) of the value y (or x if orientation is 'h').

Parameters

data_frame: DataFrame or array-like or dict This argument needs to be passed for column names (and not keyword names) to be used. Array-like and dict are transformed internally to a pandas DataFrame. Optional: if missing, a DataFrame gets constructed under the hood using the other arguments.

x: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the x axis in cartesian coordinates. If orientation is 'h', these values are used as inputs to histfunc. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

y: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the y axis in cartesian coordinates. If orientation is 'v', these values are used as inputs to histfunc. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

color: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign color to marks.

pattern_shape: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign pattern shapes to marks.

facet_row: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the vertical direction.

facet_col: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the horizontal direction.

facet_col_wrap: int Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if 0, and forced to 0 if facet_row or a marginal is set.

facet_row_spacing: float between 0 and 1 Spacing between facet rows, in paper units. Default is 0.03 or 0.07 when facet_col_wrap is used.

facet_col_spacing: float between 0 and 1 Spacing between facet columns, in paper units Default is 0.02.

hover_name: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in bold in the hover tooltip.

hover_data: str, or list of str or int, or Series or array-like, or dict Either a name or list of names of columns in data_frame, or pandas Series, or array_like objects or a dict with column names as keys, with values True (for default formatting) False (in order to remove this column from hover information), or a formatting string, for example ':.3f' or '|%a' or list-like data to appear in the hover tooltip or tuples with a bool or formatting string as first element, and list-like data to appear in hover as second element Values from these columns appear as extra data in the hover tooltip.

animation_frame: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to animation frames.

animation_group: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to provide object-constancy across animation frames: rows with matching 'animation_group's will be treated as if they describe the same object in each frame.

category_orders: dict with str keys and list of str values (default `{}`) By default, in Python 3.6+, the order of categorical values in axes, legends and facets depends on the order in which these values are first encountered in data_frame (and no order is guaranteed by default in Python below 3.6). This parameter is used to force a specific ordering of values per column. The keys of this dict should correspond to column names, and the values should be lists of strings corresponding to the specific display order desired.

labels: dict with str keys and str values (default `{}`) By default, column names are used in the figure for axis titles, legend entries and hovers. This parameter allows this to be overridden. The keys of this dict should correspond to column names, and the values should correspond to the desired label to be displayed.

color_discrete_sequence: list of str Strings should define valid CSS-colors. When color is set and the values in the corresponding column are not numeric, values in that column are assigned colors by cycling through color_discrete_sequence in the order described in category_orders, unless the value of color is a key in color_discrete_map. Various useful color sequences are available in the plotly.express.colors submodules, specifically plotly.express.colors.qualitative.

color_discrete_map: dict with str keys and str values (default `{}`) String values should define valid CSS-colors Used to override color_discrete_sequence to assign a specific colors to marks corresponding with specific values. Keys in color_discrete_map should be values in the column denoted by color. Alternatively, if the values of color are valid colors, the string 'identity' may be passed to cause them to be used directly.

pattern_shape_sequence: list of str Strings should define valid plotly.js patterns-shapes. When pattern_shape is set, values in that column are assigned patterns-shapes by cycling through pattern_shape_sequence in the order described in category_orders, unless the value of pattern_shape is a key in pattern_shape_map.

pattern_shape_map: dict with str keys and str values (default `{}`) Strings values define plotly.js patterns-shapes. Used to override pattern_shape_sequences to assign a specific patterns-shapes to lines corresponding with specific values. Keys in pattern_shape_map should be values in the column denoted by pattern_shape. Alternatively, if the values

of pattern_shape are valid patterns-shapes names, the string 'identity' may be passed to cause them to be used directly.

marginal: str One of 'rug', 'box', 'violin', or 'histogram'. If set, a subplot is drawn alongside the main plot, visualizing the distribution.

opacity: float Value between 0 and 1. Sets the opacity for markers.

orientation: str, one of `'h'` for horizontal or `'v'` for vertical. (default 'v' if x and y are provided and both continuous or both categorical, otherwise 'v'('h') if x(y) is categorical and y(x) is continuous, otherwise 'v'('h') if only x(y) is provided)

barmode: str (default `'relative'`) One of 'group', 'overlay' or 'relative' In 'relative' mode, bars are stacked above zero for positive values and below zero for negative values. In 'overlay' mode, bars are drawn on top of one another. In 'group' mode, bars are placed beside each other.

barnorm: str (default `None`) One of 'fraction' or 'percent'. If 'fraction', the value of each bar is divided by the sum of all values at that location coordinate. 'percent' is the same but multiplied by 100 to show percentages. None will stack up all values at each location coordinate.

histnorm: str (default `None`) One of 'percent', 'probability', 'density', or 'probability density' If None, the output of histfunc is used as is. If 'probability', the output of histfunc for a given bin is divided by the sum of the output of histfunc for all bins. If 'percent', the output of histfunc for a given bin is divided by the sum of the output of histfunc for all bins and multiplied by 100. If 'density', the output of histfunc for a given bin is divided by the size of the bin. If 'probability density', the output of histfunc for a given bin is normalized such that it corresponds to the probability that a random event whose distribution is described by the output of histfunc will fall into that bin.

log_x: boolean (default `False`) If True, the x-axis is log-scaled in cartesian coordinates.

log_y: boolean (default `False`) If True, the y-axis is log-scaled in cartesian coordinates.

range_x: list of two numbers If provided, overrides auto-scaling on the x-axis in cartesian coordinates.

range_y: list of two numbers If provided, overrides auto-scaling on the y-axis in cartesian coordinates.

histfunc: str (default `'count'` if no arguments are provided, else `'sum'`) One of 'count', 'sum', 'avg', 'min', or 'max'. Function used to aggregate values for summarization (note: can be normalized with histnorm). The arguments to this function are the values of y (x) if orientation is 'v' ('h').

cumulative: boolean (default `False`) If True, histogram values are cumulative.

nbins: int Positive integer. Sets the number of bins.

text_auto: bool or string (default `False`) If True or a string, the x or y or z values will be displayed as text, depending on the orientation A string like '.2f' will be interpreted as a texttemplate numeric formatting directive.

title: str The figure title.

subtitle: str The figure subtitle.

template: str or dict or plotly.graph_objects.layout.Template instance The figure template name (must be a key in plotly.io.templates) or definition.

width: int (default `None`) The figure width in pixels.

height: int (default `None`) The figure height in pixels.

Returns

plotly.graph_objects.Figure

plotly.express.box

```
plotly.express.box(data_frame=None, x=None, y=None, color=None, facet_row=None, facet_col=None,
↪  facet_col_wrap=0, facet_row_spacing=None, facet_col_spacing=None, hover_name=None, hover_data=None,
↪  custom_data=None, animation_frame=None, animation_group=None, category_orders=None, labels=None,
↪  color_discrete_sequence=None, color_discrete_map=None, orientation=None, boxmode=None, log_x=False,
↪  log_y=False, range_x=None ...)
```

In a box plot, rows of data_frame are grouped together into a box-and-whisker mark to visualize their distribution.

Each box spans from quartile 1 (Q1) to quartile 3 (Q3). The second quartile (Q2) is marked by a line inside the box. By default, the whiskers correspond to the box' edges +/- 1.5 times the interquartile range (IQR: Q3-Q1), see ”points” for other options.

Parameters

`data_frame: DataFrame or array-like or dict` This argument needs to be passed for column names (and not keyword names) to be used. Array-like and dict are transformed internally to a pandas DataFrame. Optional: if missing, a DataFrame gets constructed under the hood using the other arguments.

`x: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the x axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

`y: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to position marks along the y axis in cartesian coordinates. Either x or y can optionally be a list of column references or array_likes, in which case the data will be treated as if it were 'wide' rather than 'long'.

`color: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign color to marks.

`facet_row: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the vertical direction.

`facet_col: str or int or Series or array-like` Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to facetted subplots in the horizontal direction.

`facet_col_wrap: int` Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if 0, and forced to 0 if facet_row or a marginal is set.

`facet_row_spacing: float between 0 and 1` Spacing between facet rows, in paper units. Default is 0.03 or 0.07 when facet_col_wrap is used.

`facet_col_spacing: float between 0 and 1` Spacing between facet columns, in paper units Default is 0.02.

hover_name: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like appear in bold in the hover tooltip.

hover_data: str, or list of str or int, or Series or array-like, or dict Either a name or list of names of columns in data_frame, or pandas Series, or array_like objects or a dict with column names as keys, with values True (for default formatting) False (in order to remove this column from hover information), or a formatting string, for example ':.3f' or '|%a' or list-like data to appear in the hover tooltip or tuples with a bool or formatting string as first element, and list-like data to appear in hover as second element Values from these columns appear as extra data in the hover tooltip.

custom_data: str, or list of str or int, or Series or array-like Either name or list of names of columns in data_frame, or pandas Series, or array_like objects Values from these columns are extra data, to be used in widgets or Dash callbacks for example. This data is not user-visible but is included in events emitted by the figure (lasso selection etc.)

animation_frame: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to assign marks to animation frames.

animation_group: str or int or Series or array-like Either a name of a column in data_frame, or a pandas Series or array_like object. Values from this column or array_like are used to provide object-constancy across animation frames: rows with matching 'animation_group's will be treated as if they describe the same object in each frame.

category_orders: dict with str keys and list of str values (default `{}`) By default, in Python 3.6+, the order of categorical values in axes, legends and facets depends on the order in which these values are first encountered in data_frame (and no order is guaranteed by default in Python below 3.6). This parameter is used to force a specific ordering of values per column. The keys of this dict should correspond to column names, and the values should be lists of strings corresponding to the specific display order desired.

labels: dict with str keys and str values (default `{}`) By default, column names are used in the figure for axis titles, legend entries and hovers. This parameter allows this to be overridden. The keys of this dict should correspond to column names, and the values should correspond to the desired label to be displayed.

color_discrete_sequence: list of str Strings should define valid CSS-colors. When color is set and the values in the corresponding column are not numeric, values in that column are assigned colors by cycling through color_discrete_sequence in the order described in category_orders, unless the value of color is a key in color_discrete_map. Various useful color sequences are available in the plotly.express.colors submodules, specifically plotly.express.colors.qualitative.

color_discrete_map: dict with str keys and str values (default `{}`) String values should define valid CSS-colors Used to override color_discrete_sequence to assign a specific colors to marks corresponding with specific values. Keys in color_discrete_map should be values in the column denoted by color. Alternatively, if the values of color are valid colors, the string 'identity' may be passed to cause them to be used directly.

orientation: str, one of `'h'` for horizontal or `'v'` for vertical. (default 'v' if x and y are provided and both continuous or both categorical, otherwise 'v'('h') if x(y) is categorical and y(x) is continuous, otherwise 'v'('h') if only x(y) is provided)

boxmode: str (default `'group'`) One of 'group' or 'overlay' In 'overlay' mode, boxes are on drawn top of one another. In 'group' mode, boxes are placed beside each other.

log_x: boolean (default `False`) If True, the x-axis is log-scaled in cartesian coordinates.

log_y: boolean (default `False`) If True, the y-axis is log-scaled in cartesian coordinates.

range_x: list of two numbers If provided, overrides auto-scaling on the x-axis in cartesian coordinates.

range_y: list of two numbers If provided, overrides auto-scaling on the y-axis in cartesian coordinates.

points: str or boolean (default `'outliers'`) One of 'outliers', 'suspectedoutliers', 'all', or False. If 'outliers', only the sample points lying outside the whiskers are shown. If 'suspectedoutliers', all outlier points are shown and those less than 4*Q1-3*Q3 or greater than 4*Q3-3*Q1 are highlighted with the marker's 'outliercolor'. If 'outliers', only the sample points lying outside the whiskers are shown. If 'all', all sample points are shown. If False, no sample points are shown and the whiskers extend to the full range of the sample.

notched: boolean (default `False`) If True, boxes are drawn with notches.

title: str The figure title.

subtitle: str The figure subtitle.

template: str or dict or plotly.graph_objects.layout.Template instance The figure template name (must be a key in plotly.io.templates) or definition.

width: int (default `None`) The figure width in pixels.

height: int (default `None`) The figure height in pixels.

Returns

plotly.graph_objects.Figure

plotly.express.imshow

```
plotly.express.imshow(img, zmin=None, zmax=None, origin=None, labels={}, x=None, y=None,
↪  animation_frame=None, facet_col=None, facet_row=None, facet_col_wrap=None, facet_col_spacing=None,
↪  facet_row_spacing=None, color_continuous_scale=None, color_continuous_midpoint=None,
↪  range_color=None, title=None, template=None, width=None, height=None, aspect=None,
↪  contrast_rescaling=None, binary_string=None, binary_backend='auto ...)
```

Display an image, i.e. data on a 2D regular raster.

Parameters

img: array-like image, or xarray The image data. Supported array shapes are

- (M, N): an image with scalar data. The data is visualized using a colormap.
- (M, N, 3): an image with RGB values.
- (M, N, 4): an image with RGBA values, i.e. including transparency.

zmin, zmax : *scalar or iterable, optional* zmin and zmax define the scalar range that the colormap covers. By default, zmin and zmax correspond to the min and max values of the datatype for integer datatypes (ie [0-255] for uint8 images, [0, 65535] for uint16 images, etc.). For a multichannel image of floats, the max of the image is computed and zmax is the smallest power of 256 (1, 255, 65535) greater than this max value, with a 5% tolerance. For a single-channel image, the max of the image is used. Overridden by range_color.

origin : *str, 'upper' or 'lower' (default 'upper')* position of the [0, 0] pixel of the image array, in the upper left or lower left corner. The convention 'upper' is typically used for matrices and images.

labels : *dict with str keys and str values (default '{}')* Sets names used in the figure for axis titles (keys `x` and `y`), colorbar title and hoverlabel (key `color`). The values should correspond to the desired label to be displayed. If `img` is an xarray, dimension names are used

for axis titles, and long name for the colorbar title (unless overridden in `labels`). Possible keys are: x, y, and color.

`x, y: list-like, optional` x and y are used to label the axes of single-channel heatmap visualizations and their lengths must match the lengths of the second and first dimensions of the img argument. They are auto-populated if the input is an xarray.

`animation_frame: int or str, optional (default None)` axis number along which the image array is sliced to create an animation plot. If img is an xarray, animation_frame can be the name of one the dimensions.

`facet_col: int or str, optional (default None)` axis number along which the image array is sliced to create a facetted plot. If img is an xarray, facet_col can be the name of one the dimensions.

`facet_row: int or str, optional (default None)` axis number along which the image array is sliced to create a vertically facetted plot. If img is an xarray, facet_row can be the name of one the dimensions.

`facet_col_wrap: int` Maximum number of facet columns. Wraps the column variable at this width, so that the column facets span multiple rows. Ignored if facet_col is None or if facet_row is set.

`facet_col_spacing: float between 0 and 1` Spacing between facet columns, in paper units. Default is 0.02.

`facet_row_spacing: float between 0 and 1` Spacing between facet rows created when `facet_col_wrap` is used, in paper units. Default is 0.0.7.

`color_continuous_scale : str or list of str` colormap used to map scalar data to colors (for a 2D image). This parameter is not used for RGB or RGBA images. If a string is provided, it should be the name of a known color scale, and if a list is provided, it should be a list of CSS-compatible colors.

`color_continuous_midpoint : number` If set, computes the bounds of the continuous color scale to have the desired midpoint. Overridden by range_color or zmin and zmax.

`range_color : list of two numbers` If provided, overrides auto-scaling on the continuous color scale, including overriding color_continuous_midpoint. Also overrides zmin and zmax. Used only for single-channel images.

`title : str` The figure title.

`template : str or dict or plotly.graph_objects.layout.Template instance` The figure template name or definition.

`width : number` The figure width in pixels.

`height: number` The figure height in pixels.

`aspect: 'equal', 'auto', or None`

- 'equal': Ensures an aspect ratio of 1 or pixels (square pixels)
- 'auto': The axes is kept fixed and the aspect ratio of pixels is adjusted so that the data fit in the axes. In general, this will result in non-square pixels.
- if None, 'equal' is used for numpy arrays and 'auto' for xarrays (which have typically heterogeneous coordinates)

`contrast_rescaling: 'minmax', 'infer', or None` how to determine data values corresponding to the bounds of the color range, when zmin or zmax are not passed. If minmax, the min and max values of the image are used. If infer, a heuristic based on the image data type is used.

`binary_string: bool, default None` if True, the image data are first rescaled and encoded as uint8 and then passed to plotly.js as a b64 PNG string. If False, data are passed unchanged as a numerical array. Setting to True may lead to performance gains, at the cost of a loss of

precision depending on the original data type. If None, use_binary_string is set to True for multichannel (eg) RGB arrays, and to False for single-channel (2D) arrays. 2D arrays are represented as grayscale and with no colorbar if use_binary_string is True.

binary_backend: str, 'auto' (default), 'pil' or 'pypng' Third-party package for the transformation of numpy arrays to png b64 strings. If 'auto', Pillow is used if installed, otherwise pypng.

binary_compression_level: int, between 0 and 9 (default 4) png compression level to be passed to the backend when transforming an array to a png b64 string. Increasing binary_compression decreases the size of the png string, but the compression step takes more time. For most images it is not worth using levels greater than 5, but it's possible to test len(fig.data[0].source) and to time the execution of imshow to tune the level of compression. 0 means no compression (not recommended).

binary_format: str, 'png' (default) or 'jpg' compression format used to generate b64 string. 'png' is recommended since it uses lossless compression, but 'jpg' (lossy) compression can result if smaller binary strings for natural images.

text_auto: bool or str (default `False`) If True or a string, single-channel img values will be displayed as text. A string like '.2f' will be interpreted as a texttemplate numeric formatting directive.

Returns

fig : *graph_objects.Figure containing the displayed image*

See also: `plotly.graph_objects.Image`, `plotly.graph_objects.Heatmap`

Notes

In order to update and customize the returned figure, use go.Figure.update_traces or go.Figure.update_layout.

If an xarray is passed, dimensions names and coordinates are used for axes labels and ticks.

Graph objects and layout

plotly.graph_objects.Figure

```
plotly.graph_objects.Figure(data=None, layout=None, frames=None, skip_invalid=False, **kwargs)
```

plotly.graph_objects.Figure.update_layout

```
plotly.graph_objects.Figure.update_layout(self, dict1=None, overwrite=False, **kwargs) -> 'Figure'
```

Update the properties of the figure's layout with a dict and/or with keyword arguments.

This recursively updates the structure of the original layout with the values in the input dict / keyword arguments.

Parameters

dict1 : *dict* Dictionary of properties to be updated

overwrite: bool If True, overwrite existing properties. If False, apply updates to existing properties recursively, preserving existing properties that are not specified in the update operation.

kwargs Keyword/value pair of properties to be updated

Returns

BaseFigure The Figure object that the update_layout method was called on

plotly.graph_objects.Figure.update_traces

```
plotly.graph_objects.Figure.update_traces(self, patch=None, selector=None, row=None, col=None,
↪  secondary_y=None, overwrite=False, **kwargs) -> 'Figure'
```

Perform a property update operation on all traces that satisfy the specified selection criteria

Parameters

patch: dict or None (default None) Dictionary of property updates to be applied to all traces that satisfy the selection criteria.

selector: dict, function, int, str or None (default None) Dict to use as selection criteria. Traces will be selected if they contain properties corresponding to all of the dictionary's keys, with values that exactly match the supplied values. If None (the default), all traces are selected. If a function, it must be a function accepting a single argument and returning a boolean. The function will be called on each trace and those for which the function returned True will be in the selection. If an int N, the Nth trace matching row and col will be selected (N can be negative). If a string S, the selector is equivalent to dict(type=S).

row, col: int or None (default None) Subplot row and column index of traces to select. To select traces by row and column, the Figure must have been created using plotly.subplots.make_subplots. If None (the default), all traces are selected.

secondary_y: boolean or None (default None)

- If True, only select traces associated with the secondary y-axis of the subplot.
- If False, only select traces associated with the primary y-axis of the subplot.
- If None (the default), do not filter traces based on secondary y-axis.

To select traces by secondary y-axis, the Figure must have been created using plotly.subplots.make_subplots. See the docstring for the specs argument to make_subplots for more info on creating subplots with secondary y-axes.

overwrite: bool If True, overwrite existing properties. If False, apply updates to existing properties recursively, preserving existing properties that are not specified in the update operation.

****kwargs** Additional property updates to apply to each selected trace. If a property is specified in both patch and in **kwargs then the one in**kwargs takes precedence.

Returns

self Returns the Figure object that the method was called on

plotly.graph_objects.Figure.add_trace

```
plotly.graph_objects.Figure.add_trace(self, trace, row=None, col=None, secondary_y=None,
↪  exclude_empty_subplots=False) -> 'Figure'
```

Add a trace to the figure

Parameters

trace : *BaseTraceType or dict* **Either:**

- An instances of a trace classe from the plotly.graph_objs package (e.g plotly.graph_objs.Scatter, plotly.graph_objs.Bar)

- or a dicts where:

  - The 'type' property specifies the trace type (e.g. 'scatter', 'bar', 'area', etc.). If the dict has no 'type' property then 'scatter' is assumed.
  - All remaining properties are passed to the constructor of the specified trace type.

row : *'all', int or None (default)* Subplot row index (starting from 1) for the trace to be added. Only valid if figure was created using plotly.tools.make_subplots. If 'all', addresses all rows in the specified column(s).

col : *'all', int or None (default)* Subplot col index (starting from 1) for the trace to be added. Only valid if figure was created using plotly.tools.make_subplots. If 'all', addresses all columns in the specified row(s).

secondary_y: boolean or None (default None) If True, associate this trace with the secondary y-axis of the subplot at the specified row and col. Only valid if all of the following conditions are satisfied: * The figure was created using plotly.subplots.make_subplots. * The row and col arguments are not None * The subplot at the specified row and col has type xy (which is the default) and secondary_y True. These properties are specified in the specs argument to make_subplots. See the make_subplots docstring for more info. * The trace argument is a 2D cartesian trace (scatter, bar, etc.)

exclude_empty_subplots: boolean If True, the trace will not be added to subplots that don't already have traces.

Returns

BaseFigure The Figure that add_trace was called on

Examples

```
>>> from plotly import subplots
```

Add two Scatter traces to a figure

```
>>> fig = go.Figure()
>>> fig.add_trace(go.Scatter(x=[1,2,3], y=[2,1,2]))
Figure(...)
>>> fig.add_trace(go.Scatter(x=[1,2,3], y=[2,1,2]))
Figure(...)
```

Add two Scatter traces to vertically stacked subplots

```
>>> fig = subplots.make_subplots(rows=2)
>>> fig.add_trace(go.Scatter(x=[1,2,3], y=[2,1,2]), row=1, col=1)
Figure(...)
>>> fig.add_trace(go.Scatter(x=[1,2,3], y=[2,1,2]), row=2, col=1)
Figure(...)
...
```

plotly.subplots.make_subplots

```
plotly.subplots.make_subplots(rows=1, cols=1, shared_xaxes=False, shared_yaxes=False,
 ↪  start_cell='top-left', print_grid=False, horizontal_spacing=None, vertical_spacing=None,
 ↪  subplot_titles=None, column_widths=None, row_heights=None, specs=None, insets=None,
 ↪  column_titles=None, row_titles=None, x_title=None, y_title=None, figure=None, **kwargs) ->
 ↪  plotly.graph_objs._figure.Figure
```

Return an instance of plotly.graph_objs.Figure with predefined subplots configured in 'layout'.

Parameters

rows: int (default 1) Number of rows in the subplot grid. Must be greater than zero.

cols: int (default 1) Number of columns in the subplot grid. Must be greater than zero.

shared_xaxes: boolean or str (default False) Assign shared (linked) x-axes for 2D cartesian subplots

- True or 'columns': Share axes among subplots in the same column
- 'rows': Share axes among subplots in the same row
- 'all': Share axes across all subplots in the grid.

shared_yaxes: boolean or str (default False) Assign shared (linked) y-axes for 2D cartesian subplots

- 'columns': Share axes among subplots in the same column
- True or 'rows': Share axes among subplots in the same row
- 'all': Share axes across all subplots in the grid.

start_cell: 'bottom-left' or 'top-left' (default 'top-left') Choose the starting cell in the subplot grid used to set the domains_grid of the subplots.

- **'top-left': Subplots are numbered with (1, 1) in the top** left corner
- **'bottom-left': Subplots are numbererd with (1, 1) in the bottom** left corner

print_grid: boolean (default False): If True, prints a string representation of the plot grid. Grid may also be printed using the Figure.print_grid() method on the resulting figure.

horizontal_spacing: float (default 0.2 / cols) Space between subplot columns in normalized plot coordinates. Must be a float between 0 and 1.

Applies to all columns (use 'specs' subplot-dependents spacing)

vertical_spacing: float (default 0.3 / rows) Space between subplot rows in normalized plot coordinates. Must be a float between 0 and 1.

Applies to all rows (use 'specs' subplot-dependents spacing)

subplot_titles: list of str or None (default None) Title of each subplot as a list in row-major ordering.

Empty strings (”“) can be included in the list if no subplot title is desired in that space so that the titles are properly indexed.

specs: list of lists of dict or None (default None) Per subplot specifications of subplot type, row/column spanning, and spacing.

ex1: specs=[[{}, {}], [{'colspan': 2}, None]]

ex2: specs=[[{'rowspan': 2}, {}], [None, {}]]

- Indices of the outer list correspond to subplot grid rows starting from the top, if start_cell='top-left', or bottom, if start_cell='bottom-left'. The number of rows in 'specs' must be equal to 'rows'.
- Indices of the inner lists correspond to subplot grid columns starting from the left. The number of columns in 'specs' must be equal to 'cols'.

- Each item in the 'specs' list corresponds to one subplot in a subplot grid. (N.B. The subplot grid has exactly 'rows' times 'cols' cells.)
- Use None for a blank a subplot cell (or to move past a col/row span).
- Note that specs[0][0] has the specs of the 'start_cell' subplot.
- **Each item in 'specs' is a dictionary.**

The available keys are:

```
type (string, default \'xy\'): Subplot type. One of

:   -   \'xy\': 2D Cartesian subplot type for scatter, bar, etc.

    -   \'scene\': 3D Cartesian subplot for scatter3d, cone, etc.

    -   \'polar\': Polar subplot for scatterpolar, barpolar, etc.

    -   \'ternary\': Ternary subplot for scatterternary

    -   \'map\': Map subplot for scattermap

    -

        \'domain\': Subplot type for traces that are individually

        :   positioned. pie, parcoords, parcats, etc.

    -

        trace type: A trace type which will be used to determine

        :   the appropriate subplot type for that trace
```

  - **secondary_y (bool, default False): If True, create a secondary** y-axis positioned on the right side of the subplot. Only valid if type='xy'.
  - **colspan (int, default 1): number of subplot columns** for this subplot to span.
  - **rowspan (int, default 1): number of subplot rows** for this subplot to span.
  - l (float, default 0.0): padding left of cell
  - r (float, default 0.0): padding right of cell
  - t (float, default 0.0): padding right of cell
  - b (float, default 0.0): padding bottom of cell
- Note: Use 'horizontal_spacing' and 'vertical_spacing' to adjust the spacing in between the subplots.

insets: list of dict or None (default None): Inset specifications. Insets are subplots that overlay grid subplots

- **Each item in 'insets' is a dictionary.** The available keys are:
  - **cell (tuple, default=(1,1)): (row, col) index of the** subplot cell to overlay inset axes onto.

- type (string, default 'xy'): Subplot type
- **l (float, default=0.0): padding left of inset** in fraction of cell width
- **w (float or 'to_end', default='to_end') inset width** in fraction of cell width ('to_end': to cell right edge)
- **b (float, default=0.0): padding bottom of inset** in fraction of cell height
- **h (float or 'to_end', default='to_end') inset height** in fraction of cell height ('to_end': to cell top edge)

column_widths: list of numbers or None (default None) list of length cols of the relative widths of each column of subplots. Values are normalized internally and used to distribute overall width of the figure (excluding padding) among the columns.

For backward compatibility, may also be specified using the column_width keyword argument.

row_heights: list of numbers or None (default None) list of length rows of the relative heights of each row of subplots. If start_cell='top-left' then row heights are applied top to bottom. Otherwise, if start_cell='bottom-left' then row heights are applied bottom to top.

For backward compatibility, may also be specified using the row_width kwarg. If specified as row_width, then the width values are applied from bottom to top regardless of the value of start_cell. This matches the legacy behavior of the row_width argument.

column_titles: list of str or None (default None) list of length cols of titles to place above the top subplot in each column.

row_titles: list of str or None (default None) list of length rows of titles to place on the right side of each row of subplots. If start_cell='top-left' then row titles are applied top to bottom. Otherwise, if start_cell='bottom-left' then row titles are applied bottom to top.

x_title: str or None (default None) Title to place below the bottom row of subplots, centered horizontally

y_title: str or None (default None) Title to place to the left of the left column of subplots, centered vertically

figure: go.Figure or None (default None) If None, a new go.Figure instance will be created and its axes will be populated with those corresponding to the requested subplot geometry and this new figure will be returned. If a go.Figure instance, the axes will be added to the layout of this figure and this figure will be returned. If the figure already contains axes, they will be overwritten.

Examples

Example 1:

```
>>> # Stack two subplots vertically, and add a scatter trace to each
>>> from plotly.subplots import make_subplots
>>> fig = make_subplots(rows=2)
```

This is the format of your plot grid: [(1,1) xaxis1,yaxis1] [(2,1) xaxis2,yaxis2]

```
>>> fig.add_scatter(y=[2, 1, 3], row=1, col=1)
Figure(...)
>>> fig.add_scatter(y=[1, 3, 2], row=2, col=1)
Figure(...)
```

or see Figure.add_trace ...

Output

plotly.io.write_html

```
plotly.io.write_html(fig, file, config=None, auto_play=True, include_plotlyjs=True,
↪  include_mathjax=False, post_script=None, full_html=True, animation_opts=None, validate=True,
↪  default_width='100%', default_height='100%', auto_open=False, div_id=None)
```

Write a figure to an HTML file representation

Parameters

fig: Figure object or dict representing a figure

file: str or writeable A string representing a local file path or a writeable object (e.g. a pathlib.Path object or an open file descriptor)

config: dict or None (default None) Plotly.js figure config options

auto_play: bool (default=True) Whether to automatically start the animation sequence on page load if the figure contains frames. Has no effect if the figure does not contain frames.

include_plotlyjs: bool or string (default True) Specifies how the plotly.js library is included/loaded in the output div string.

If True, a script tag containing the plotly.js source code (~3MB) is included in the output. HTML files generated with this option are fully self-contained and can be used offline.

If 'cdn', a script tag that references the plotly.js CDN is included in the output. The url used is versioned to match the bundled plotly.js. HTML files generated with this option are about 3MB smaller than those generated with include_plotlyjs=True, but they require an active internet connection in order to load the plotly.js library.

If 'directory', a script tag is included that references an external plotly.min.js bundle that is assumed to reside in the same directory as the HTML file. If file is a string to a local file path and full_html is True, then the plotly.min.js bundle is copied into the directory of the resulting HTML file. If a file named plotly.min.js already exists in the output directory then this file is left unmodified and no copy is performed. HTML files generated with this option can be used offline, but they require a copy of the plotly.min.js bundle in the same directory. This option is useful when many figures will be saved as HTML files in the same directory because the plotly.js source code will be included only once per output directory, rather than once per output file.

If a string that ends in '.js', a script tag is included that references the specified path. This approach can be used to point the resulting HTML file to an alternative CDN or local bundle.

If False, no script tag referencing plotly.js is included. This is useful when the resulting div string will be placed inside an HTML document that already loads plotly.js. This option is not advised when full_html=True as it will result in a non-functional html file.

include_mathjax: bool or string (default False) Specifies how the MathJax.js library is included in the output html div string. MathJax is required in order to display labels with LaTeX typesetting.

If False, no script tag referencing MathJax.js will be included in the output.

If 'cdn', a script tag that references a MathJax CDN location will be included in the output. HTML div strings generated with this option will be able to display LaTeX typesetting as long as internet access is available.

If a string that ends in '.js', a script tag is included that references the specified path. This approach can be used to point the resulting HTML div string to an alternative CDN.

post_script: str or list or None (default None) JavaScript snippet(s) to be included in the resulting div just after plot creation. The string(s) may include '{plot_id}' placeholders that will then be replaced by the id of the div element that the plotly.js figure is associated with. One application for this script is to install custom plotly.js event handlers.

full_html: bool (default True) If True, produce a string containing a complete HTML document starting with an <html> tag. If False, produce a string containing a single element.

animation_opts: dict or None (default None) dict of custom animation parameters to be passed to the function Plotly.animate in Plotly.js. See https://github.com/plotly/plotly.js/blob/master/src/plots/animation_attributes.js for available options. Has no effect if the figure does not contain frames, or auto_play is False.

default_width, default_height: number or str (default '100%') The default figure width/height to use if the provided figure does not specify its own layout.width/layout.height property. May be specified in pixels as an integer (e.g. 500), or as a css width style string (e.g. '500px', '100%').

validate: bool (default True) True if the figure should be validated before being converted to JSON, False otherwise.

auto_open: bool (default True) If True, open the saved file in a web browser after saving. This argument only applies if full_html is True.

div_id: str (default None) If provided, this is the value of the id attribute of the div tag. If None, the id attribute is a UUID.

Returns

None

plotly.io.write_image

```
plotly.io.write_image(fig: Union[dict, plotly.graph_objs._figure.Figure], file: Union[str,
↪  pathlib._local.Path], format: Optional[str] = None, scale: Union[int, float, NoneType] = None, width:
↪  Optional[int] = None, height: Optional[int] = None, validate: bool = True)
```

Convert a figure to a static image and write it to a file or writeable object

Parameters

fig: Figure object or dict representing a figure

file: str or writeable A string representing a local file path or a writeable object (e.g. a pathlib.Path object or an open file descriptor)

format: str (optional) The desired image format. One of:

- 'png'
- 'jpg' or 'jpeg'
- 'webp'
- 'svg'
- 'pdf'

Defaults to the file extension of file, if given; otherwise defaults to plotly.io.defaults.default_format.

width: int (optional) The width of the exported image in layout pixels. If the scale property is 1.0, this will also be the width of the exported image in physical pixels. Defaults to plotly.io.defaults.default_width.

height: int (optional) The height of the exported image in layout pixels. If the scale property is 1.0, this will also be the height of the exported image in physical pixels. Defaults to plotly.io.defaults.default_height.

scale: int or float (optional) The scale factor to use when exporting the figure. A scale factor larger than 1.0 will increase the image resolution with respect to the figure's layout pixel dimensions, while a scale factor of less than 1.0 will decrease the image resolution. Defaults to plotly.io.defaults.default_scale.

validate: bool (optional) True if the figure should be validated before being converted to an image, False otherwise. Defaults to True.

Returns

None

plotly.io.show

```
plotly.io.show(fig, renderer=None, validate=True, **kwargs)
```

Show a figure using either the default renderer(s) or the renderer(s) specified by the renderer argument

Parameters

fig: dict of Figure The Figure object or figure dict to display

renderer: str or None (default None) A string containing the names of one or more registered renderers (separated by '+' characters) or None. If None, then the default renderers specified in plotly.io.renderers.default are used.

validate: bool (default True) True if the figure should be validated before being shown, False otherwise.

width: int or float An integer or float that determines the number of pixels wide the plot is. The default is set in plotly.js.

height: int or float An integer or float specifying the height of the plot in pixels. The default is set in plotly.js.

config: dict A dict of parameters to configure the figure. The defaults are set in plotly.js.

Returns

None